

Eingereicht von

Jonas Nils Oliver Elspe

Matrikelnummer: 313235

jonas.elsper@htwg-konstanz.de

M

C

Masterthesis

Data-Science-Agent

S

Konstanz, 15.04.2026

Masterthesis

Data-Science-Agent

von

Jonas Nils Oliver Elster

zur Erlangung des akademischen Grades

Master of Science

im Studiengang Informatik

an der Hochschule Konstanz für Technik, Wirtschaft und Gestaltung.

Matrikelnummer: 313235

Startdatum: 16.10.2025

Abgabedatum: 15.04.2026

Erstprüfer: Prof. Dr. Oliver Dürr

Zweitprüfer: Christoph Ulrich

Hiermit erkläre ich, Jonas Nils Oliver Elsper, geboren am 20.08.2001 in Rotthalmünster,

(1) dass ich meine Masterthesis mit dem Titel:

„Data-Science-Agent“

bei SEITENBAU GmbH unter Anleitung von Prof. Dr. Oliver Dürr selbständig und ohne fremde Hilfe angefertigt habe und keine anderen als die angeführten Hilfen benutzt habe.

(2) dass ich die Übernahme wörtlicher Zitate, von Tabellen, Zeichnungen, Bildern und Programmen aus der Literatur oder anderen Quellen (Internet) sowie die Verwendung der Gedanken anderer Autoren an den entsprechenden Stellen innerhalb der Arbeit gekennzeichnet habe.

(3) dass die eingereichten Abgabe-Exemplare in Papierform und im PDF-Format vollständig übereinstimmen.

Ich bin mir bewusst, dass eine falsche Erklärung rechtliche Folgen haben wird.

Konstanz, 15.04.2026

Verwendung von KI-Werkzeugen

Bei der Erstellung dieser Arbeit wurden folgende KI-gestützte Werkzeuge eingesetzt:

Anthropic Claude Opus & Sonnet 4.6	Zur Unterstützung bei der Erstellung von Analyse- und Visualisierungscode. Zur sprachlichen Korrektur, Überarbeitung und Ausformulierung einzelner Textpassagen.
OpenAI GPT-5	Zur Unterstützung bei der Erstellung von Analyse- und Visualisierungscode. Zur sprachlichen Korrektur und Überarbeitung einzelner Textpassagen.
Perplexity Sonar	Zur Unterstützung bei der Auffindung von Dokumentationsressourcen und Community-Beiträgen zu den verwendeten Frameworks.
DeepL	Zur Unterstützung bei der Übersetzung von Textpassagen.
PRISM GPT-5.2	Zur Korrektur von Typst Code.

Alle durch KI-Werkzeuge unterstützten Inhalte wurden vom Verfasser kritisch geprüft, fachlich validiert und bei Bedarf überarbeitet. Die Verantwortung für die endgültige Fassung liegt vollständig beim Verfasser.

Abstract

Open-Data-Plattformen wie das GovData-Portal stellen eine Vielzahl tabellarischer Datensätze öffentlich bereit. Die explorative Erstanalyse dieser Daten erfordert jedoch weiterhin erheblichen manuellen Aufwand, was insbesondere für nicht-technisch-versierte Nutzergruppen eine Zugangshürde darstellt.

Die vorliegende Arbeit untersucht, wie ein LLM-basiertes Multi-Agent-System zur autonomen Exploration tabellarischer Datensätze konzipiert und evaluiert werden kann. Aufbauend auf dem LIDA-Framework wird der Data-Science-Agent vorgestellt, der für beliebige tabellarische Datensätze automatisiert natürlichsprachliche Zusammenfassungen, Spaltenerläuterungen und relevante Visualisierungen erzeugt. Das System ist als Grundlage einer datengestützten Vorschaufunktion für Open-Data-Plattformen konzipiert.

Zur Bewertung der Ergebnisqualität werden die Metriken Visualization Error Rate (VER) und Self-Evaluated Visualization Quality (SEVQ) verwendet und erweitert. Im Rahmen einer empirischen Evaluation auf Datensätzen des GovData-Portals sowie einer begleitenden Studie mit erfahrenen Teilnehmenden werden fünf Forschungsfragen untersucht: die Eignung von SEVQ als automatisierte Evaluierungsmetrik, der Einfluss von Prompt- und Programmiersprache auf die Ergebnisqualität, die Generalisierbarkeit des Systems auf heterogene Datensätze, der Effekt zusätzlicher Metadaten sowie die Stabilität der Ergebnisse über wiederholte Ausführungen.

Die Ergebnisse zeigen, dass die automatisierte SEVQ-Bewertung durch LLM-Judges eine moderate ordinale Konsistenz aufweist, die absolute Übereinstimmung mit menschlichen Urteilen jedoch begrenzt bleibt – die Metrik eignet sich daher als Indikator für relative Qualitätsunterschiede, nicht als Ersatz einer menschlichen Bewertung. Englischsprachige Prompts erzielen signifikant bessere Ergebnisse als deutsche, während die Wahl der Programmiersprache einen untergeordneten Einfluss hat. Der Agent kann mit generischen tabellarischen Datensätzen umgehen, so lang sie offiziellen Standards folgen. Die Ergebnisse sind über wiederholte Ausführungen stabil und die Abweichung der Qualität gering.

Sämtliche in dieser Arbeit verwendeten Datensätze, Rohdaten der Experimente und der Online-Studie, Auswertungsskripte, Prompt-Templates sowie der vollständige Quellcode des Data-Science-Agenten sind im öffentlich zugänglichen GitHub-Repository¹ hinterlegt. Eine vollständige Reproduktion der Ergebnisse ist damit möglich.

Die durchgeführte Online-Studie wurde unter Einhaltung der geltenden Datenschutzbestimmungen (DSGVO) konzipiert. Die Teilnahme erfolgte freiwillig und anonym; personenbezogene Daten wurden nicht erhoben. Die verwendeten Datensätze entstammen dem öffentlich zugänglichen GovData-Portal und unterliegen keinen Zugangsbeschränkungen.

¹<https://github.com/Elsper01/data-science-agent>

Inhaltsverzeichnis

Abkürzungsverzeichnis	vii
1. Einleitung	1
1.1. Motivation	1
1.2. Überblick	1
2. Grundlagen	3
2.1. Large Language Models (LLM)	3
2.2. Agentic AI	4
2.3. Patterns für Agentic AI	5
2.4. LLM-as-a-Judge	8
2.5. LIDA	11
3. Data-Science-Agent	14
3.1. Umsetzung des Data-Science-Agenten	14
3.2. Evaluationsmetriken	19
3.3. Abgrenzung von LIDA	20
4. Versuchsaufbau	20
4.1. Datensatz	21
4.2. Umsetzung der Studie	22
4.3. Analyseverfahren	27
4.4. Beeinträchtigung der Validität	32
5. Ergebnisse	33
5.1. RQ-1: Validität der automatisierten Evaluation	33
5.2. RQ-2: Konfigurationseffekte	37
5.3. RQ-3: Generalisierbarkeit	39
5.4. RQ-4: Einfluss von Metadaten	41
5.5. RQ-5: Ergebnisstabilität	42
6. Diskussion	43
6.1. RQ-1: Validität der automatisierten Evaluation	43
6.2. RQ-2: Konfigurationseffekte	45
6.3. RQ-3: Generalisierbarkeit	45
6.4. RQ-4: Einfluss von Metadaten	46
6.5. RQ-5: Ergebnisstabilität	46
7. Schlussfolgerung	47
Literatur	49
Anhang A: Data-Science-Agent	i
Anhang B: Ergebnisse	vi

Abkürzungsverzeichnis

AIR	Average Iterations Required
API	Application Programming Interface
BLEU	Bilingual Evaluation Understudy
CKAN	Comprehensive Knowledge Archive Network
CER	Convergence Error Rate
CV	Variationskoeffizient
DC	Dataset Coverage
DSGVO	Datenschutz-Grundverordnung
DTO	Data Transfer Object
ICC	Intraclass Correlation Coefficient
IGM	Image Generation Model
KI	Künstliche Intelligenz
LLM	Large Language Model
LOO	Leave-One-Out
MAE	Mean Absolute Error
MAS	Multi-Agent-System
MBR	Metadata Benefit Rate
MCP	Model Context Protocol
METEOR	Metric for Evaluation of Translation with Explicit Ordering
NLP	Natural Language Processing
OOD	Out-of-Distribution
RAG	Retrieval-Augmented Generation
RDF	Resource Description Framework
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
ROUGE	Recall-Oriented Understudy for Gisting Evaluation
RQ	Forschungsfrage (Research Question)
SD	Standardabweichung
SEVQ	Self-Evaluated Visualization Quality
VER	Visualization Error Rate
VLM	Visual Language Model

1. Einleitung

1.1. Motivation

Daten sind im Zuge der Digitalisierung allgegenwärtig: Wetterstationen messen kontinuierlich Parameter für statistische Vorhersagen, Fitness-Tracker unterstützen Nutzende bei der Optimierung ihres Trainings. Als konkretes Beispiel sei der Pegelstand des Bodensees genannt – eine Information, die für Schifffahrt, Umweltbehörden und Wissenschaft gleichermaßen relevant ist. Gemeinsam ist diesen Gruppen, dass sie auf öffentliche Datensätze angewiesen sind, um fundierte Entscheidungen treffen zu können.

Sowohl die Europäische Union (EU) als auch die Bundesregierung haben die Relevanz frei zugänglicher Daten erkannt. Mit der Richtlinie (EU) 2019/1024 [1] verpflichtete die EU ihre Mitgliedsstaaten, Datensätze des öffentlichen Sektors als Open Data bereitzustellen. Im Zuge dessen beschloss die Bundesregierung im Juli 2021 die „Open-Data-Strategie der Bundesregierung“, [2], in deren Rahmen das GovData-Portal² als zentrale Open-Data-Plattform für Deutschland geschaffen wurde.

Obwohl Nutzende damit einfach auf diese Datensätze zugreifen können, erfordert eine erste explorative Analyse weiterhin erheblichen manuellen Aufwand: Um zu prüfen, ob ein Datensatz für den eigenen Anwendungsfall geeignet ist, muss er heruntergeladen, lokal verarbeitet und manuell ausgewertet werden. Dieser Prozess ist zeitaufwendig und fehleranfällig. Mithilfe von auf Large Language Models (LLMs) basierenden Agentensystemen lässt sich dieser Prozess automatisieren.

Eine konkrete Ausprägung dieses Automatisierungsansatzes ist eine in Open-Data-Plattformen integrierbare **datengestützte Vorschaufunktion**: Rufen Nutzende auf dem GovData-Portal einen Datensatz auf, so erhalten sie unmittelbar – ohne Download oder manuelle Aufbereitung – automatisch generierte Einblicke. Diese umfassen eine natürlichsprachliche Zusammenfassung, eine Erläuterung der enthaltenen Spalten sowie relevante Visualisierungen. Die Vorschaufunktion ermöglicht es, schnell einzuschätzen, ob ein Datensatz für den eigenen Anwendungsfall geeignet ist – ein entscheidender Mehrwert insbesondere für Nutzende ohne Programmierkenntnisse.

Ein LLM-basierter Agent ist ein autonomes Softwaresystem, das ein LLM als Steuereinheit einsetzt, um selbstständig Aufgaben zu lösen, eigene Entscheidungen zu treffen und angebundene Werkzeuge einzusetzen [3],[4]. Ein aktueller Entwicklungstrend sind Multi-Agent-Systeme, in denen spezialisierte Subagenten unter Koordination eines Orchestrierungsagenten zusammenwirken [5],[6]. V. Dibia [7] stellten mit LIDA erstmals ein solches System vor, das für beliebige tabellarische Datensätze automatisiert Visualisierungen erzeugt.

1.2. Überblick

In dieser Arbeit wird untersucht, wie ein Multi-Agent-System zur autonomen Exploration tabellarischer Datensätze konzipiert und evaluiert werden kann, das als Grundlage für eine solche

²<https://www.govdata.de/>

Vorschaufunktion einer Open-Data-Plattform dienen soll. Aufbauend auf den Ideen und Erkenntnissen von LIDA [7] wird der Data-Science-Agent als konkrete Umsetzung vorgestellt: Gegeben einen tabellarischen Datensatz, erzeugt das System automatisch relevante Visualisierungen.

Das übergeordnete Ziel ist die autonome Generierung qualitativ hochwertiger Visualisierungen für beliebige Datensätze. Zur Bewertung der Ergebnisqualität und zur gezielten Weiterentwicklung des Systems werden die folgenden Forschungsfragen untersucht:

- **RQ-1 (Evaluierungsmetriken):** Eignen sich die von V. Dibia [7] eingeführten Metriken zur automatisierten Bewertung der Ergebnisqualität des Data-Science-Agenten?
- **RQ-2 (Konfiguration):** Welchen Einfluss haben die verwendete Promptsprache und die Programmiersprache im Codegenerierungsschritt auf die Ergebnisqualität?
- **RQ-3 (Generalisierbarkeit):** Ist der Data-Science-Agent für beliebige tabellarische Datensätze geeignet?
- **RQ-4 (Metadaten):** Verbessert die Integration von Metadaten die Qualität der erzeugten Visualisierungen?
- **RQ-5 (Stabilität):** Erzielt der Data-Science-Agent bei wiederholter Ausführung auf denselben Datensätzen konsistent gute Ergebnisse?

Zur empirischen Untersuchung dieser Forschungsfragen wurden Experimente sowie eine Online-Studie mit 14 erfahrenen Teilnehmenden durchgeführt. Die Ergebnisse zeigen, dass die automatisierte SEVQ-Bewertung durch LLM-Judges eine moderate ordinale Konsistenz aufweist, die absolute Übereinstimmung mit menschlichen Urteilen jedoch begrenzt bleibt – die Metrik eignet sich daher als Indikator für relative Qualitätsunterschiede, nicht als Ersatz einer menschlichen Bewertung. Englischsprachige Prompts erzielen signifikant bessere Ergebnisse als deutsche, während die Wahl der Programmiersprache einen untergeordneten Einfluss hat. Der Agent kann mit generischen tabellarischen Datensätzen umgehen, solange diese offiziellen Standards folgen. Die Ergebnisse sind über wiederholte Ausführungen stabil und die Veränderung der Evaluierungsergebnisse ist gering.

Die Beiträge dieser Arbeit lauten im Einzelnen wie folgt:

- Konzeption und Implementierung des Data-Science-Agenten als explizites Multi-Agent-System, das die konzeptionellen Grundlagen von LIDA [7] in den Dimensionen Architektur, Evaluationsstrategie und Konfigurierbarkeit erweitert.
- Erweiterung der Evaluationsmethodik durch iterative VER-Berechnung und einen Multi-LLM-Judge-Ansatz mit fünf Sprachmodellen zur Reduktion systematischer Bewertungsverzerrungen.
- Empirische Untersuchung der Eignung automatisierter Metriken, des Einflusses von Prompt- und Programmiersprache, der Generalisierbarkeit, des Metadaten-Effekts sowie der Ergebnisstabilität auf Basis von 50 realen Open-Data-Datensätzen und einer begleitenden Online-Studie.
- Statistische Analyse der erhobenen Daten zur Beantwortung der fünf Forschungsfragen und kritische Diskussion der Ergebnisse.

2. Grundlagen

Der Anwendungsbereich von Multi-Agent-Systemen ist sehr vielfältig und erstreckt sich vom Gesundheitswesen über den Finanzsektor bis hin zu adaptiven Softwaresystemen [5]. Diese Arbeit konzentriert sich auf den Teilbereich der automatisierten Datenzusammenfassung, -analyse und -visualisierung. In diesem Kapitel werden zunächst die konzeptionellen Grundlagen in den Bereichen LLMs, Agentic AI, Agentic AI Patterns und LLM-as-a-Judge erläutert. Anschließend wird mit LIDA das für diese Arbeit zentrale verwandte System vorgestellt, das die konzeptionelle Grundlage des Data-Science-Agenten bildet.

2.1. Large Language Models (LLM)

LLMs sind auf großen Textmengen vortrainierte neuronale Sprachmodelle auf Basis der **Transformer-Architektur** [8], die Textsequenzen token-weise autoregressiv modellieren. Der zentrale Mechanismus der Transformer-Architektur ist die **Self-Attention**, die es dem Modell ermöglicht, bei der Verarbeitung eines Tokens gewichtete Beziehungen zu allen anderen Tokens der Eingabesequenz herzustellen. Durch die Parallelisierung mehrerer Attention-Köpfe (**Multi-Head Attention**) kann das Modell unterschiedliche Beziehungsmuster zwischen Tokens gleichzeitig erfassen [8].

Durch Instruction Tuning und Reinforcement Learning from Human Feedback (RLHF) werden vortrainierte Sprachmodelle zu Chat-Modellen verfeinert, die natürlichsprachliche Anweisungen zuverlässig befolgen [9]. Besonders relevant für Multi-Agent-Systeme ist die Fähigkeit moderner LLMs zum **Tool Use**: Modelle können strukturierte Funktionsaufrufe erzeugen und damit externe Werkzeuge in ihren Verarbeitungsprozess einbinden [10]. Zudem ermöglicht Few-Shot-Learning die Anpassung an neue Aufgaben allein durch Beispiele im Kontext, ohne erneutes Training [11].

Der **Temperaturparameter** T steuert die Schärfe der Wahrscheinlichkeitsverteilung bei der Token-Generierung: Bei $T \rightarrow 0$ konvergiert die Ausgabe deterministisch zum wahrscheinlichsten Token, bei $T > 1$ wird die Verteilung gleichmäßiger und die Ausgabe diverser. In der Praxis haben sich stochastische Sampling-Verfahren wie **Nucleus Sampling** (Top- p -Sampling) etabliert, die qualitativ hochwertigere Texte erzeugen als deterministische Dekodierung [12]. Für agentische Systeme ist die Wahl der Sampling-Strategie direkt relevant: Ein deterministisches Setting ($T = 0$) ermöglicht reproduzierbare Ergebnisse, während höhere Temperaturwerte die Varianz der Agentenausgaben erhöhen.

Eine zentrale Einschränkung aktueller LLMs ist die begrenzte Länge des **Kontextfensters**, die sich aus dem quadratischen Aufwand des Self-Attention-Mechanismus ergibt [8]. Für agentische Systeme bedeutet dies, dass lange Konversationshistorien oder umfangreiche Tool-Ergebnisse das Kontextfenster überschreiten können. Strategien wie selektives Zusammenfassen oder gezieltes Filtern sind daher notwendig, um eine konsistente Informationsbasis zu gewährleisten. [13]

2.2. Agentic AI

Agentic AI bezeichnet ein Paradigma autonomer KI-Systeme, die komplexe Aufgaben mit minimalem menschlichem Eingriff bewältigen [5]. Im Unterschied zu klassischen Systemen, die strukturierte Daten, strikte Überwachung und fest definierte Aufgabenrahmen voraussetzen, zeichnen sich agentische Systeme durch Adaptivität, autonome Entscheidungsfindung und flexibles Verhalten in dynamischen Umgebungen aus. Sie verbinden dabei regelbasierte Ansätze mit den Fähigkeiten generativer Modelle – insbesondere kontextabhängiger Sprachverarbeitung und Problemlösung [5].

Die konzeptionellen Grundlagen agentischer Systeme wurden bereits in den frühen 1990er Jahren gelegt. [M. Wooldridge und N. R. Jennings \[3\]](#) definierten vier Kerneigenschaften, die einen Agenten charakterisieren:

- **Autonomie:** Ein Agent handelt ohne direkte menschliche Eingriffe und kontrolliert selbstständig seine Aktionen und seinen internen Zustand.
- **Soziale Fähigkeiten:** Agenten können mit anderen Agenten und mit Menschen kommunizieren und kooperieren.
- **Reaktionsfähigkeit:** Agenten nehmen ihre Umgebung wahr und passen ihr Verhalten dynamisch an veränderte Bedingungen an.
- **Proaktivität:** Agenten reagieren nicht nur auf Umgebungsreize, sondern verfolgen aktiv eigene Ziele.

Zeitraum	Phase	Schlüsseltechnologien	Einschränkungen
1980er-1990er	Konzeptionelle Grundlagen	Symbolisches Denken, regelbasierte Systeme, erste Multi-Agenten-Konzepte	Begrenzte Lernfähigkeit, hoher manueller Modellierungsaufwand
2001-2010	Lernbasierte Agenten	RNNs, LSTMs, NLP, Reinforcement Learning, überwachtes Lernen	Eingeschränkte Generalisierung, hoher Datenbedarf
2011-2017	Deep Reinforcement Learning	Deep RL, CNNs, spielbasierte Lernumgebungen	Aufwendiges Training, Overfitting, fehlende Sprachverarbeitung
2018-2021	Generalistische Agenten	Transformer, BERT, GPT, emergente Fähigkeiten, Transfer Learning	Eingeschränkte Autonomie, kein echtes Reasoning, Alignment-Probleme
2022-heute	Multimodale kollaborative Agenten	LLM-Frameworks, Multi-Agent-Systeme, Tool Use, Memory & Planning, RAG	Alignment, Stabilität, robuste Langzeitplanung

Tabelle 1: Entwicklungsphasen agentischer KI-Systeme von den konzeptionellen Grundlagen der 1980er Jahre bis zur aktuellen Generation multimodaler Multi-Agent-Systeme. Jede Phase ist durch charakteristische Schlüsseltechnologien sowie die jeweils verbleibenden Herausforderungen geprägt. Quelle: in Anlehnung an [U. Nisa et al. \[14\]](#).

Die praktische Umsetzung dieser Konzepte vollzog sich schrittweise in mehreren Phasen, die in [Tabelle 1](#) zusammengefasst sind [14].

Ein zentraler Aspekt der aktuellen Phase sind **Multi-Agent-Systeme**, in denen mehrere spezialisierte Agenten in arbeitsteiliger Koordination an komplexen Aufgaben zusammenwirken. Ein Orchestrierungsagent koordiniert dabei den Prozess, delegiert Teilaufgaben an geeignete spezialisierte Agenten (z.B. einen Daten-Explorations-Agenten oder einen Visualisierungs-Agenten), integriert die Zwischenergebnisse und erzeugt ein strukturiertes Gesamtergebnis [5],[6]. Dieser Ansatz bildet die architektonische Grundlage des in dieser Arbeit vorgestellten Data-Science-Agenten und wird im Kontext der automatisierten Datenanalyse und -visualisierung in [Abschnitt 2.5](#) und [Abschnitt 3](#) näher erläutert.

Der Begriff „Agentic AI“ selbst ist gegenwärtig im Wandel. In der aktuellen Forschung wird zunehmend diskutiert, ob die alleinige Betrachtung von Autonomie zur Charakterisierung agentischer Systeme ausreicht, oder ob darüber hinaus Dimensionen wie Intentionalität, strategische Planung und kontextuelles Reasoning - zusammengefasst unter dem erweiterten Konzept der Agency - berücksichtigt werden müssen. [15]

2.3. Patterns für Agentic AI

Um die Komplexität agentischer Systeme beherrschbar zu gestalten, haben sich wiederkehrende Entwurfsmuster (**Agentic Design Patterns**) [15] etabliert, die analog zu klassischen Software-Entwurfsmustern abstrakte Lösungsstrategien beschreiben [16],[17]. Im Folgenden werden die vier für diese Arbeit relevanten Muster (**Reflexion**, **Tool Use**, **ReAct** und **Multi-Agent-System**) erläutert.

Reflexion

Das **Reflexion**-Muster beschreibt eine Architekturstrategie, bei der ein Agent seinen eigenen Output iterativ bewertet und auf Basis dieses Feedbacks gezielt verbessert. Die Kernidee besteht darin, die Ausgabequalität nicht durch zusätzliches Training, sondern durch mehrfache, aufgabenspezifische Selbstevaluation zur Laufzeit zu steigern. [18],[19]

Die theoretische Grundlage bildet das von [N. Shinn et al. \[20\]](#) eingeführte Framework. Darin lernen Sprachagenten durch **Verbal Reinforcement Learning**: Anstatt Modellgewichte anzupassen, verarbeitet der Agent sprachliches Feedback aus vorherigen Ausführungen und speichert es in einem Speicher, der in nachfolgenden Iterationen als Kontext dient [20].

Der Reflexionsprozess ist in [Abbildung 1](#) dargestellt und läuft iterativ ab: Der **Actor** erzeugt ein Ergebnis (*trajectory*), der **Evaluator** bewertet es, und das **Self-Reflection-Modell** formuliert daraus verbales Feedback, das im Speicher abgelegt wird. In jeder weiteren Iteration greift der Actor auf dieses Feedback zurück. Die Schleife terminiert, sobald der Evaluator die Lösung als korrekt einstuft oder eine maximale Iterationsanzahl n erreicht ist. Um die Kontextlängenbeschränkung einzuhalten, wird der Speicher auf typischerweise ein bis drei Einträge begrenzt. [18]

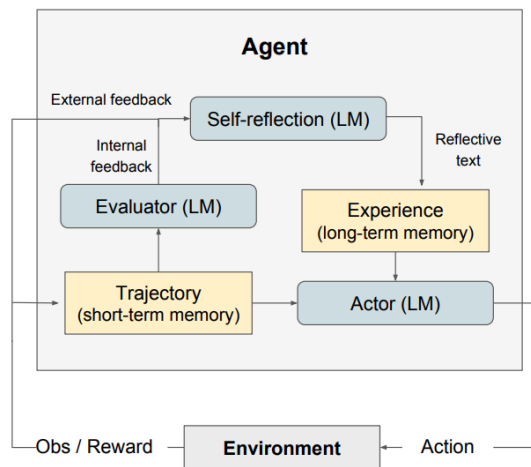


Abbildung 1: Schematischer Ablauf des Reflexion-Musters nach N. Shinn *et al.* [20]. Der Actor erzeugt eine Lösung (Trajectory), die vom Evaluator bewertet wird. Das Self-Reflection-Modell leitet aus der Bewertung verbales Feedback ab, das im Experience-Speicher abgelegt wird. In jeder weiteren Iteration greift der Actor auf dieses Feedback zurück, um sein Verhalten iterativ zu verbessern. Die Schleife terminiert, sobald der Evaluator die Lösung als korrekt einstuft oder eine maximale Iterationsanzahl erreicht ist.

Die Wirksamkeit wurde empirisch durch N. Shinn *et al.* [20] nachgewiesen. Z. Gou *et al.* [21] konnten bestätigen, dass werkzeuggestützte Selbstkorrektur die Ausgabequalität verbessern kann.

Das Muster eignet sich insbesondere für Aufgaben, bei denen Ausgabequalität Vorrang vor Ausführungsgeschwindigkeit hat – etwa in der Codegenerierung oder Datenanalyse. Ein wesentlicher Nachteil ist der erhöhte Ressourcenaufwand durch zusätzliche LLM-Aufrufe pro Iteration. Zudem können sich systematische Modellverzerrungen (vgl. Tabelle 2) in den iterativen Verbesserungen fortschreiben [18].

Tool Using

Als probabilistische Modelle sind Sprachmodelle weder für exakte Berechnungen konzipiert noch verfügen sie über aktuelle Informationen. T. Schick *et al.* [10] stellten erstmals ein Sprachmodell vor, das trainiert wurde, in geeigneten Situationen auf externe Werkzeuge zurückzugreifen. Dieser Ansatz hat sich etabliert und wird heute von allen gängigen Sprachmodellen unterstützt.

Der zugrundeliegende Mechanismus basiert auf *Function Calling*: Das LLM erhält eine strukturierte Beschreibung verfügbarer Werkzeuge und entscheidet autonom, welches Werkzeug eingesetzt werden soll. Es erzeugt einen Funktionsaufruf, der vom umgebenden System ausgeführt wird, und das Ergebnis wird im Kontext gespeichert [10]. Beispiele für häufig genutzte Werkzeuge sind Suchmaschinen, Code-Interpreter und Datenbankabfragen. Mit der Einführung des **Model Context Protocol** (MCP) [22] wurde die Kommunikation zwischen Sprachmodell und Werkzeug weiter vereinheitlicht. Das Tool-Using-Muster bildet einen fundamentalen Baustein für komplexere agentische Entwurfsmuster; das ReAct-Muster baut direkt darauf auf.

ReAct: Reasoning and Acting

Ein essenzielles Muster für LLM-basierte Agenten ist das von [S. Yao et al. \[23\]](#) eingeführte ReAct-Muster (**Reasoning and Acting**). Die Grundidee besteht darin, explizite Denk-Schritte (*Reasoning Traces*) mit Tool-Using zu verknüpfen und dadurch Probleme mehrstufig zu lösen. Anders als beim Chain-of-Thought-Prompting [\[24\]](#), das nur mit sprachlicher Schlussfolgerung arbeitet, ermöglicht ReAct dem Agenten, externe Werkzeuge dynamisch aufzurufen und deren Ergebnisse in den weiteren Reasoning-Prozess einzubeziehen. Dadurch können Abhängigkeiten zwischen Werkzeugaufrufen explizit modelliert und Aufgaben zielgerichtet und autonom gelöst werden.

Das ReAct-Muster ist in [Abbildung 2](#) dargestellt und setzt sich aus folgenden Elementen zusammen: Im **Think**-Schritt formuliert das Reasoning-LLM einen Gedankengang zum weiteren Vorgehen. Im **Act**-Schritt wählt es ein passendes Tool aus und ruft es auf. Im **Observe**-Schritt wird das Ergebnis im Agenten-Kontext gespeichert. In der **Repeat**-Phase entscheidet der Agent, ob weitere Schritte notwendig sind. Die Schleife terminiert (**Stop**), sobald hinreichende Informationen vorliegen oder eine maximale Iterationsanzahl erreicht ist.

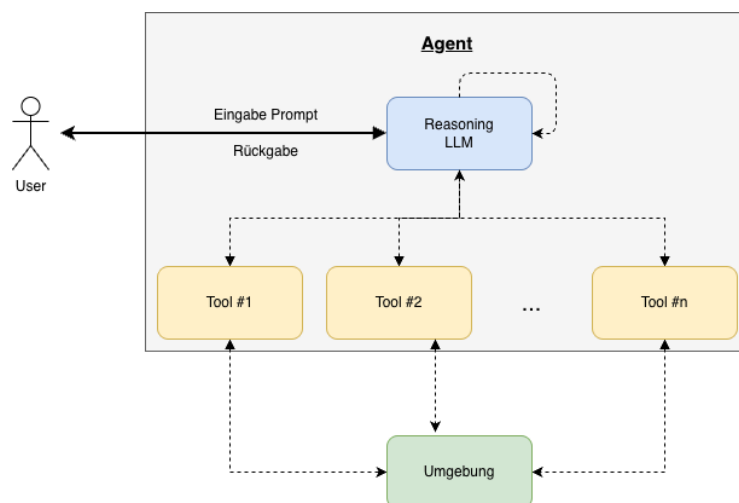


Abbildung 2: Schematischer Ablauf des ReAct-Musters. Der Agent erhält durch den Eingabe-Prompt der Nutzenden eine Aufgabe. Das Reasoning-LLM denkt schrittweise nach und verwendet nach eigenem Ermessen Tools, um mit der Umgebung zu interagieren. Die Ergebnisse der Tool-Using-Schritte werden in das weitere Reasoning einbezogen. Sobald der Agent das Problem als gelöst erachtet, gibt er ein Ergebnis an die Nutzenden zurück. Gestrichelte Kanten bezeichnen für Nutzende nicht sichtbare Kommunikationsschritte, durchgezogene Kanten Kommunikationsschritte mit den Nutzenden.

[S. Yao et al. \[23\]](#) zeigten empirisch, dass ReAct gegenüber Standard-Prompting, Chain-of-Thought und isoliertem Tool-Using Halluzinationen reduziert und bessere Ergebnisse erzielt. Als Nachteile sind die längere Ausführungszeit, höhere Kosten durch zusätzliche Token sowie die Abhängigkeit von der Qualität der Tool-Dokumentation zu nennen. Das Muster eignet sich besonders für Aufgaben mit unbekanntem Lösungsweg, wie mehrstufige Informationsrecherchen oder explorative Datenanalysen.

Multi-Agent-System

Ein Multi-Agent-System (MAS) ist eine Architektur, in der mehrere spezialisierte Agenten kooperieren, um ein übergeordnetes Ziel zu erreichen [6],[25]. Die grundlegende Erkenntnis ist, dass einzelne Agenten bei komplexen, vielschichtigen Aufgaben an Grenzen stoßen: Die erforderliche Breite an Fachwissen, Werkzeugen und Kontextinformationen übersteigt häufig die Kapazität eines einzelnen Agenten [6]. MAS adressieren dieses Problem durch Arbeitsteilung. Jeder Agent ist auf eine Teilaufgabe spezialisiert und die Gesamtlösung entsteht durch Koordination dieser Spezialisten [26].

Die Architektur eines MAS umfasst zwei zentrale Rollen [25],[26]: Der **Orchestrierungs-Agent** zerlegt das übergeordnete Ziel in Teilaufgaben, weist diese Spezialisten zu und integriert die Teilergebnisse. Die **Spezialisten-Agenten** sind für definierte Rollen optimiert und verfügen über aufgabenspezifische System-Prompts sowie spezialisierte Werkzeuge. Alle Agenten operieren auf einem gemeinsamen Zustandsgraphen (*Shared State*) [25].

T. Guo *et al.* [6] unterscheiden drei Kommunikationstopologien: Bei der **layered**-Topologie durchlaufen die Agenten eine feste, sequenzielle Pipeline – geeignet für deterministische Workflows wie Datenanalysepipelines. Die **decentralized**-Topologie erlaubt gleichberechtigte Kommunikation ohne zentrale Steuerung. Die **centralized**-Topologie nutzt einen zentralen Agenten, der Aufgaben dynamisch verteilt und die Strategie zur Laufzeit anpasst [6].

Die Vorteile des MAS-Musters liegen in der Modularität, der Skalierbarkeit sowie der verbesserten Aufgabenqualität durch Spezialisierung [6],[25]. Gleichzeitig steigt die Koordinationskomplexität mit der Anzahl der Agenten, Fehler können sich kaskadenartig auswirken, und die Inferenzkosten erhöhen sich proportional [6],[26].

2.4. LLM-as-a-Judge

Die automatisierte Evaluierung von natural-language Text ist seit langer Zeit ein zentrales Problem der Computerlinguistik und hat im Kontext von Large Language Models neue Aufmerksamkeit erhalten [27]. Lange Zeit beschränkte man sich auf überlappungsbasierte Metriken wie BLEU [28], ROUGE [29] und METEOR [30], die ursprünglich für Aufgaben wie maschinelle Übersetzung und Textzusammenfassung entwickelt wurden. Diese Metriken quantifizieren, wie stark der generierte Text dem erwarteten Referenztext ähnelt. Doch Large Language Models lösen Aufgaben in einer Vielfalt und Komplexität, für die sich Referenzüberlappungen als unzureichend erweisen. Zudem ist es bei vielen Anwendungsfällen praktisch unmöglich, umfangreiche manuelle Referenzmengen zu erstellen. Des Weiteren ist es im Kontext von LLMs sinnvoll, Dimensionen wie Faithfulness, Informativeness, Coherence, u.a. zu bewerten, da ein LLM nur ein probabilistisches Modell ist, das durchaus zu Halluzination im OOD-Bereich tendiert [31]. Aus diesem Grund wurden zeitweise die Ergebnisse von LLMs nicht durch automatisch berechenbare Metriken, sondern durch menschliches Feedback bewertet [32].

Ein vielversprechender Ansatz zur Überwindung dieser Limitationen ist die Verwendung eines LLM als Evaluator - ein Verfahren, das als **LLM-as-a-Judge** bekannt ist. Die zentrale Idee ist vielversprechend: Wenn ein LLM in der Lage ist, komplexe Aufgaben zu lösen, sollte es auch in der Lage sein, die Qualität von Lösungen zu beurteilen. S. Kadavath *et al.* [33] zeigten empirisch,

dass hinreichend große Language Models dazu fähig sind, ihren eigenen Output und den anderer Modelle zuverlässig zu bewerten - oft besser als klassische automatisierte Metriken. Zwischen der grundlegenden Erkenntnis, dass LLMs zur Bewertung eigener Ausgaben fähig sind, und einem methodisch fundierten LLM-as-a-Judge-Ansatz lag jedoch erheblicher Forschungsbedarf. Eine der ersten und umfassendsten kritischen Untersuchungen hierzu wurde von [L. Zheng et al. \[32\]](#) durchgeführt, die den LLM-as-a-Judge-Ansatz systematisch evaluierten und als vielversprechende Alternative zur menschlichen Bewertung etablierten. Der größte Vorteil liegt darin, dass ein LLM-as-a-Judge skalierbarer, automatisierbarer und einfacher nachvollziehbarer (liefert nicht nur eine Bewertung, sondern auch eine Begründung) Ansatz ist, als die bisherigen sehr aufwändigen menschlichen Evaluationen und trotzdem sehr gute Ergebnisse liefert [\[32\]](#). In manchen Anwendungsbereichen, wie etwa bei Textzusammenfassungen, erreicht dieser Ansatz bereits hohe Übereinstimmungsquoten (> 80%) mit menschlichen Bewertungen [\[31\],\[32\]](#).

Trotz der genannten Vorteile sind LLM-as-a-Judge-Systeme nicht frei von systematischen Verzerrungen. In [Tabelle 2](#) sind die von [L. Zheng et al. \[32\]](#) vorgestellten Biases in komprimierter Form aufgelistet und empirisch am Beispiel von GPT-4 als Sprachmodell untersucht worden.

Bias	Beschreibung	Empirischer Befund
Position Bias	Der Judge bevorzugt Antworten an bestimmter Position in der Eingabe, unabhängig vom Inhalt.	Alle getesteten Modelle zeigen starken Positionsbias; nur GPT-4 liefert in >60 % der Fälle konsistente Urteile.
Verbosity Bias	Der Judge bevorzugt längere, ausschweifende Antworten, auch wenn diese nicht qualitativ besser sind.	Alle Modelle sind anfällig; GPT-4 widersteht dem Angriff deutlich besser als kleinere Modelle.
Self-Enhancement Bias	Der Judge neigt dazu, Antworten des eigenen Modells höher zu bewerten.	GPT-4 bevorzugt sich selbst um ~10 %, Claude-v1 um ~25 %; der Effekt ist statistisch nicht eindeutig belegt.

Tabelle 2: Übersicht über systematische Biases von LLM-as-a-Judge-Systemen und deren empirischen Befund. Zusammenfassung der Untersuchung von [L. Zheng et al. \[32\]](#).

Trotz dieser Limitationen und den Biases zeigte [L. Zheng et al. \[32\]](#), dass die Übereinstimmung zwischen LLM-Judge und menschlichen Bewertern insgesamt hoch bleibt, sofern geeignete Gegenmaßnahmen wie Multi-LLM Evaluation ergriffen werden.

Besonders relevant wird der LLM-as-a-Judge Ansatz bei der Evaluierung von Multi-Agent-Systemen, da diese Systeme besonders komplex sind. Die Komplexität kommt durch die Vielzahl miteinander zusammenarbeitender Agenten, daher stellt sich die Frage, auf welcher Granularität die Qualitätsbewertung erfolgen soll. [J. Gu et al. \[34\]](#) identifizieren zwei zentrale Strategien: Erstens die Bewertung des Gesamtergebnisses des Systems am Ende der Verarbeitungskette, zweitens die Evaluierung von Zwischenschritten während der Ausführung. Jede Variante hat ihre Berechtigung - Gesamtsystembewertungen erfassen die Endqualität, während Zwischenevaluierungen ermöglichen, Fehler frühzeitig zu detektieren und den Prozess zu optimieren.

Ergänzend zur Frage der Granularität spielt auch der konkrete Evaluationsmodus eine wichtige Rolle. In der zusammenfassenden Arbeit von [H. Li et al. \[35\]](#) werden drei grundlegende Ansätze

unterschieden: Bei der **pointwise** Evaluation wird jedes Ergebnis unabhängig anhand definierter Kriterien mit einem Score versehen - einfach anwendbar, aber ohne Berücksichtigung relativer Qualitätsunterschiede. Die **pairwise** Evaluation vergleicht zwei Ergebnisse direkt miteinander und entscheidet, welches hinsichtlich der Kriterien besser abschneidet; dieser Ansatz eignet sich besonders, wenn absolute Bewertungen schwer zu treffen, relative Unterschiede jedoch gut erkennbar sind. Die **listwise** Evaluation bewertet schließlich eine gesamte Menge von Kandidaten simultan und erzeugt eine Rangordnung, was insbesondere in Ranking-Szenarien relevant ist. Im Kontext von Multi-Agent-Systemen ergänzen diese Evaluationsmodi die Granularitätsdimension: Pointwise Evaluation eignet sich zur Beurteilung einzelner Agenten-Outputs, pairwise Evaluation zum Vergleich alternativer Lösungsvorschläge, und listwise Evaluation zur gemeinsamen Priorisierung mehrerer Ergebnisse oder Agentenstrategien. [32]

Um sicherzustellen, dass ein LLM-Judge tatsächlich zuverlässige und valide Bewertungen liefert, ist es essenziell, die Qualität des Judges selbst zu validieren. Das zentralste Evaluationskriterium ist hierfür weiterhin die Übereinstimmung zwischen den Bewertungen des LLM-as-a-Judge und menschlichen Bewertern, gemessen durch die Agreement Metrik. Diese wird definiert als der Anteil der Datenpunkte, bei denen Modell und Mensch zum selben Urteil gelangen:

$$(1) \quad \text{Agreement} = \frac{\sum_{i \in D} I(S_{\text{llm},i} = S_{\text{human},i})}{|D|}$$

Dabei bezeichnet D den Evaluierungsdatensatz, $S_{\text{llm},i}$ die Bewertung des LLM für Beispiel i , $S_{\text{human},i}$ die menschliche Referenzbewertung, und $I(\cdot)$ die Indikatorfunktion, die 1 bei Übereinstimmung und 0 sonst zurückgibt. Die Metrik liegt zwischen 0 und 1, wobei höhere Werte bessere Übereinstimmung anzeigen. Neben Agreement werden häufig Korrelationsmaße wie Cohen's Kappa oder Spearman-Rangkorrelation eingesetzt, um subtilere Unterschiede in der Bewertungsqualität zu erfassen.

Ein wichtiger praktischer Befund ist, dass die Qualität von LLM-Judges stark vom zugrunde liegenden Sprachmodell abhängt. Verschiedene LLMs treffen unterschiedliche Bewertungsentscheidungen, teilweise aufgrund unterschiedlicher Trainingsdaten oder Architekturen, teilweise aber auch systematischer Biases. Dies hat zu dem Ansatz der **Multi-LLM Evaluation** geführt, bei dem mehrere spezialisierte Judges ein System gemeinsam evaluieren. H. Li *et al.* [35] argumentieren, dass ein solches Ensemble-Verfahren Biases reduziert, Robustheit erhöht und insgesamt zuverlässigere Urteile liefert. Die Aggregation der Bewertungen mehrerer Judges - sei es durch Mehrheitsvotum, gewichtete Mittelwertbildung oder andere Mechanismen - ermöglicht es, stabilere und weniger voreingenommene Qualitätsbewertungen zu erzielen. [35]

Neben der Wahl der Evaluationsstrategie spielt auch die Gestaltung der Prompts eine entscheidende Rolle im LLM-as-a-Judge Ansatz. Prompt Engineering bestimmt maßgeblich, wie das Modell Bewertungsaufgaben interpretiert und beeinflusst damit direkt die Qualität, Konsistenz und Nachvollziehbarkeit der Ergebnisse. Verschiedene Prompting-Strategien haben sich etabliert, die unterschiedliche Aspekte der Evaluation adressieren (siehe [Tabelle 3](#)).

Strategie	Beschreibung
In-Context Learning	Nutzung von Beispielen im Prompt zur Anpassung an spezifische Evaluationsaufgaben; flexibel, aber anfällig für Bias durch Beispielauswahl.
Step-by-Step Prompting	Zerlegung der Evaluation in mehrere Schritte zur Verbesserung von Nachvollziehbarkeit und Konsistenz der Bewertung. Sehr ähnlich zu Chain-of-Thought.
Definition Augmentation	Explizite Erweiterung und Präzisierung von Bewertungskriterien, ggf. unter Einbezug externen Wissens.
Multi-turn Optimization	Iterative Verfeinerung der Evaluation durch mehrstufige Interaktion und Feedbackschleifen.
Allgemeines Prompt Engineering	Strukturierte und klare Formulierung von Prompts zur Verbesserung von Qualität und Stabilität der Bewertung.

Tabelle 3: Übersicht über Prompt-Engineering-Strategien für LLM-as-a-Judge-Systeme nach [H. Li et al. \[35\]](#). Die Strategien adressieren unterschiedliche Aspekte der Evaluation und können kombiniert eingesetzt werden.

Ein wichtiger Hinweis für die praktische Umsetzung eines LLM-as-a-Judge ist es, stets feste Seeds zu verwenden und `temperature = 0` im Sprachmodell zu konfigurieren, um die Ergebnisse reproduzieren zu können.

Als konkretes Anwendungsbeispiel des pointwise Evaluationsmodus beschreiben [V. Dibia \[7\]](#) eine Architektur, in der ein spezialisierter Evaluator-Agent am Ende der Visualisierungsgenerierung die Gesamtqualität beurteilt (siehe [Abschnitt 3.2](#)).

Zusammenfassend bietet der LLM-as-a-Judge Ansatz eine flexible, skalierbare Alternative zu klassischen Evaluierungsmetriken, ist aber an die Bedingung geknüpft, dass die Validität des Judges selbst durch Human-Agreement-Studien nachgewiesen wird. Multi-Judge-Systeme versprechen zusätzliche Robustheit und reduzieren das Risiko systematischer Fehlbewertungen.

2.5. LIDA

Die automatisierte Ableitung von Visualisierungen aus Daten – im Folgenden als Data2Vis bezeichnet – ist ein aktives Forschungsfeld an der Schnittstelle von Datenanalyse und natürlicher Sprachverarbeitung. Eine zentrale Erkenntnis dieses Feldes ist, dass sich aus natürlichsprachlichen Beschreibungen effektiv Visualisierungen generieren lassen. Daraus ergibt sich ein mehrstufiger Ansatz: Zunächst wird aus tabellarischen Daten eine Zusammenfassung in natürlicher Sprache erzeugt, aus der sich Explorationsziele – also konkrete analytische Fragestellungen – ableiten lassen. Deklarative Visualisierungsgrammatiken wie Vega-Lite [\[36\]](#) und Altair [\[37\]](#) ermöglichen es, zu diesen Explorationszielen präzise Visualisierungsspezifikationen zu formulieren.

Victor Dibia hat diese Forschungsrichtung über mehrere Arbeiten hinweg maßgeblich vorangetrieben. In [B. Saket et al. \[38\]](#) wurden Grundlagen für die datengetriebene Ableitung von Visualisierungen erarbeitet. Mit [V. Dibia und Ç. Demiralp \[39\]](#) stellte Dibia einen konkreten Ansatz vor, der neuronale Sequenz-zu-Sequenz-Modelle zur Übersetzung von Daten in Visualisierungsspezifikationen einsetzt. LIDA [\[7\]](#) stellt die erste Umsetzung dieser Ideen als modulares System

dar, das Large Language Models in einer mehrstufigen Pipeline zur Visualisierungsgenerierung nutzt.

LIDA (visualizations and infographics) ist ein von V. Dibia [7] entwickeltes System zur automatischen Generierung von Visualisierungen und Infografiken aus tabellarischen Datensätzen. LIDA bildet die Grundlage des in dieser Arbeit entwickelten Data-Science-Agenten. Im Folgenden wird die Architektur von LIDA im Detail vorgestellt.

Die zentrale Idee besteht darin, Visualisierungsgenerierung als mehrstufiges Textgenerierungsproblem zu formulieren, das durch den kombinierten Einsatz von LLM und Image Generation Models (IGM) lösbar ist. Die Architektur gliedert sich in vier spezialisierte Module, deren Zusammenspiel in [Abbildung 3](#) veranschaulicht ist.

Der **Summarizer** wandelt einen tabellarischen Datensatz in einen kompakten natürlichsprachlichen Text um, der in LIDA als *Kontext* bezeichnet wird und als Grundlage für alle nachfolgenden Module dient. Die Erzeugung erfolgt zweistufig: Zunächst werden regelbasiert Dateneigenschaften wie Datentypen, statistische Kennzahlen und Stichproben extrahiert. Anschließend generiert ein LLM auf Basis dieser Informationen eine Zusammenfassung des Datensatzes einschließlich einer Erklärung der enthaltenen Variablen.

Der **Goal Explorer** erzeugt auf Basis des Kontexts eine Menge von Explorationszielen, wobei jedes Ziel aus einer Frage (Hypothese), einer zugehörigen Visualisierungsform und einer Begründung (Rationale) besteht. Das explizite Anfordern einer Begründung hat sich empirisch als entscheidend für die semantische Kohärenz der generierten Ziele erwiesen [7].

Der **VizGenerator** ist das komplexeste Modul und setzt sich aus drei Untermodulen zusammen: einem **Code Scaffold Constructor**, der grammatikspezifische Code-Gerüste für verschiedene Visualisierungsbibliotheken (z.B. Matplotlib, Seaborn, Altair) bereitstellt, einem **Code Generator**, der einen LLM im Fill-in-the-Middle-Modus einsetzt, um n Kandidaten-Spezifikationen zu erzeugen, sowie einem **Code Executor**, der die Kandidaten filtert, ausführt und nach dem Self-Consistency-Prinzip den optimalen Code selektiert [7]. Zusätzlich implementiert der VizGenerator einen integrierten **Selbstevaluierungs- und Reparaturmechanismus**: Ein LLM bewertet die generierten Visualisierungen anhand von sechs Dimensionen (Code-Korrektheit, Datentransformation, Zielkonformität, Visualisierungstyp, Datenkodierung und Ästhetik) und kann fehlerhafte Visualisierungen automatisch reparieren.

Der **Infographer** erzeugt schließlich stilisierte Infografiken aus den generierten Visualisierungen unter Verwendung text-konditionierter IGMs.

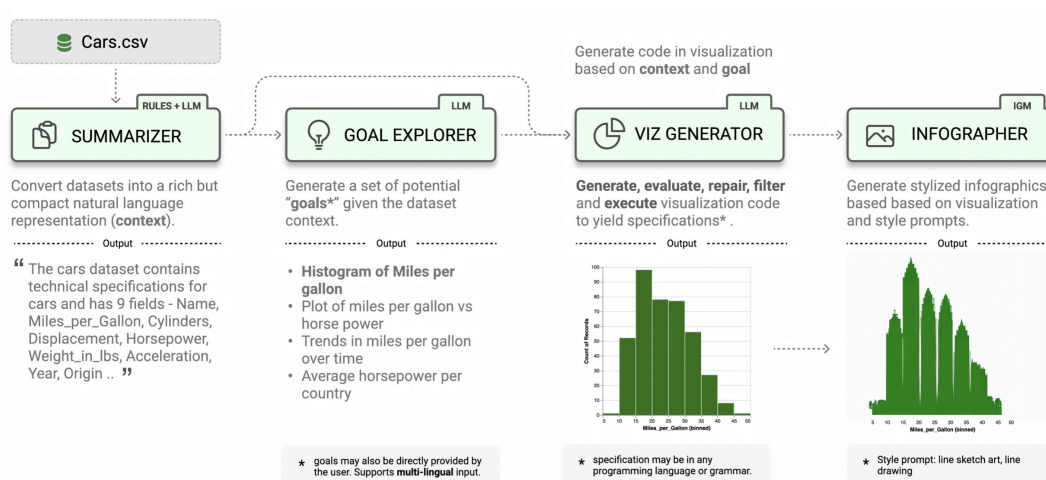


Abbildung 3: Aufbau und Verarbeitungsablauf von LIDA nach V. Dibia [7]. Die Pipeline setzt sich aus vier spezialisierten Modulen zusammen: Der Summarizer erzeugt einen kompakten natürlichsprachlichen Kontext, der Goal Explorer leitet daraus Explorationsziele mit Begründung ab, der VizGenerator erzeugt, evaluiert und repariert Visualisierungscode, und der Infographer generiert stilisierte Infografiken. Für jedes Modul ist ein exemplarischer Output dargestellt.

Obwohl V. Dibia [7] LIDA nicht explizit als Multi-Agent-System bezeichnen, erfüllt die Architektur die von [3] definierten Kerneigenschaften eines Agenten: Jedes der vier Module agiert autonom innerhalb seines spezialisierten Aufgabenbereichs, nimmt einen definierten Input wahr und produziert einen strukturierten Output für das nachfolgende Modul. Der Kontrollfluss – realisiert durch ein übergeordnetes Skript – übernimmt dabei die Rolle eines Orchestrierungsagenten, der die Informationsweitergabe deterministisch koordiniert. LIDA lässt sich damit als implizites Multi-Agent-System einordnen, das Modularität, Spezialisierung und sequenzielle Koordination realisiert, ohne eine explizite Agenten-Abstraktion zu verwenden.

Für den Data-Science-Agenten sind insbesondere die Evaluationsmetriken relevant, welche in LIDA vorgestellt wurden. Zum Einen setzt der VizGenerator zur Selbstevaluation einen LLM-as-a-Judge ein, der Visualisierungen pointwise bewertet und neben einem numerischen Score auch eine Begründung liefert – V. Dibia [7] bezeichnen diese Metrik als **SEVQ**. Zusätzlich wird mit der **VER** eine Reliabilitätsmetrik der Pipeline eingeführt, welche den Anteil der Visualisierungen mit Kompilierfehlern angibt. Beide Metriken – SEVQ und VER – bilden die konzeptionelle Grundlage für die in dieser Arbeit entwickelte Evaluierungsmethodik und werden in Abschnitt 3.2 aufgegriffen.

Einordnung gegenüber alternativen Systemen

Die Wahl von LIDA als konzeptionelle Grundlage ergibt sich aus einer systematischen Abwägung gegenüber alternativen Ansätzen. ChartGPT [40] verfolgt einen monolithischen Ansatz ohne strukturierte Zwischenrepräsentationen oder Selbstbewertungsmechanismen, was die für ein Multi-Agent-System erforderliche funktionale Segmentierung nicht ermöglicht. Die früheren Systeme NL4DV [41] und Voyager [42],[43] lieferten konzeptionelle Grundlagen – insbesondere die semantische Vorverarbeitung natürlicher Sprache und das Prinzip der kandidatenbasierten Generierung –, die im Data-Science-Agenten aufgegriffen werden.

LIDA vereint diese Erkenntnisse mit LLMs als kontextadaptiver Wissensquelle und integriert eine skalierbare interne Qualitätsbewertung. Die explizite Modulsegmentierung ermöglicht eine direkte Abbildung auf spezialisierte Agenten und bildet die Grundlage des in [Abschnitt 3](#) vorgestellten Systems.

3. Data-Science-Agent

In diesem Kapitel wird der Data-Science-Agent vorgestellt, der auf den konzeptionellen Grundlagen von LIDA [\[7\]](#) aufbaut und diese in wesentlichen Aspekten erweitert. Zunächst werden zentrale Designentscheidungen der Umsetzung begründet, anschließend die vier Module des Agenten im Detail erläutert und die integrierten Evaluierungsmechanismen vorgestellt. Abschließend wird eine detaillierte Abgrenzung von LIDA vorgenommen, die Gemeinsamkeiten und Unterschiede systematisch herausarbeitet.

3.1. Umsetzung des Data-Science-Agenten

In diesem Abschnitt wird die Architektur und Funktionsweise des Data-Science-Agenten vorgestellt und auf zentrale Designentscheidungen eingegangen. Eine vollständige Dokumentation des Systems mit all seinen Komponenten ist dem öffentlich zugänglichen GitHub-Repository³ zu entnehmen.

Architektur und Framework-Wahl

Wie in [Abschnitt 2.5](#) dargelegt, lässt sich LIDA als implizites Multi-Agent-System einordnen, das Modularität, Spezialisierung und sequenzielle Koordination realisiert. Für den Data-Science-Agenten wurde diese Architekturentscheidung bewusst übernommen und explizit als Multi-Agent-System umgesetzt. Das Multi-Agent-System-Muster (vgl. [Abschnitt 2.3](#)) eignet sich für das vorliegende Problem – die automatisierte Analyse und Visualisierung beliebiger tabellarischer Datensätze – besonders gut, da sich die Aufgabe in klar abgegrenzte, spezialisierte Teilschritte zerlegen lässt, die sequenziell aufeinander aufbauen.

Als Implementierungs-Framework wurden **LangChain**⁴ und **LangGraph**⁵ für Python gewählt. LangChain abstrahiert die Interaktion mit verschiedenen LLM-Anbietern hinter einer einheitlichen Schnittstelle und erleichtert so den Austausch von Sprachmodellen für systematische Vergleichsexperimente.

LangGraph erweitert LangChain um zustandsbehaftete, zyklische Ausführungsgraphen. Ein LangGraph-Programm besteht aus **State** (geteilte Datenstruktur für den Systemzustand), **Nodes** (Python-Funktionen für Agentenlogik) und **Edges** (bedingte Verbindungen für den Kontrollfluss). Dieses Modell entspricht dem Shared-State-Prinzip des Multi-Agent-Musters (vgl. [Abschnitt 2.3](#))

³<https://github.com/Elsper01/data-science-agent>

⁴<https://www.langchain.com/>

⁵<https://www.langchain.com/langgraph>

und ermöglicht die direkte Implementierung von Reflexions-Schleifen, ReAct-Zyklen und Multi-Agent-Workflows als explizite Graphstruktur.

Für die Orchestrierung wurde ein deterministischer Graph definiert (vgl. [Abbildung 19](#)), in dem die einzelnen Knoten – die jeweils einem spezialisierten Agenten entsprechen – in vordefinierter Reihenfolge ausgeführt werden. Dies entspricht der in [Abschnitt 2.3](#) beschriebenen **layered**-Topologie, bei der jeder Agent sein Ergebnis an den nächsten weitergibt. Über den gesamten Graphen wird ein gemeinsamer Zustand geteilt, auf den alle Knoten lesend und schreibend zugreifen können – ein Mechanismus, der dem Shared-State-Prinzip des Multi-Agent-System-Patterns entspricht.

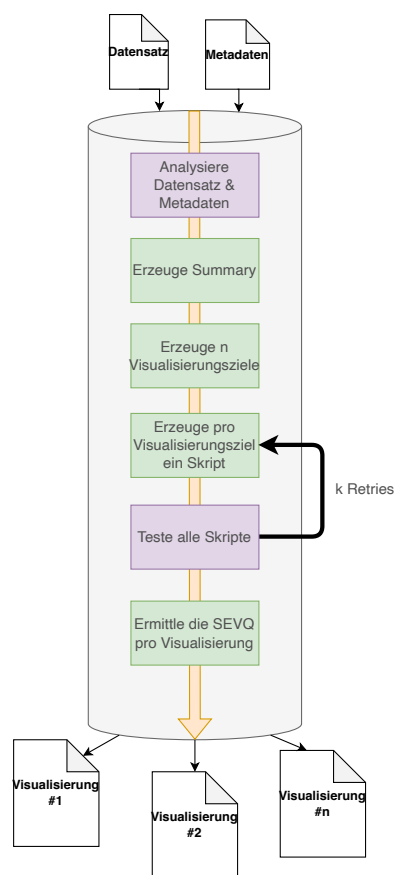


Abbildung 4: Die wichtigsten Schritte der Agenten-Pipeline des Data-Science-Agenten. Jeder Schritt stellt einen Agenten in einer Multi-Agent-System-Architektur mit layered-Topologie als Orchestrierungsansatz dar. Grün markierte Schritte bezeichnen Agenten, die ein LLM verwenden. Lila markierte Schritte arbeiten regelbasiert ohne Einsatz eines LLMs.

Konfiguration

Der Data-Science-Agent verfügt über mehrere konfigurierbare Parameter, die eine flexible Anpassung an unterschiedliche Experimentbedingungen ermöglichen. Erstens kann pro LLM-Schritt das verwendete Sprachmodell individuell festgelegt werden, was systematische Modellvergleiche ermöglicht. Zweitens unterstützt der Agent sowohl deutsche als auch englische Prompts; entsprechend werden die Prompt-Templates und die zugehörigen DTOs (Data Transfer Objects) dynamisch geladen. Die DTOs geben die Struktur der LLM-Antworten vor und fungieren damit als eine Form des indirekten Tool Use – wie in [Abschnitt 2.3](#) erläutert, ist die präzise Beschreibung der erwarteten Ausgabestruktur für die Qualität der LLM-Ergebnisse entscheidend. Drittens kann zwischen Python und R als Skriptsprache für die Codegenerierung gewählt werden. Schließlich sind auch die Anzahl n der zu generierenden Visualisierungsziele sowie die maximale Anzahl k der Reparaturversuche pro Visualisierung konfigurierbar.

Prompt-Design

Die Prompts der einzelnen Agenten folgen einem einheitlichen Aufbau: Jeder Agent verfügt über einen **System-Prompt**, der seine spezialisierte Expertenrolle präzise definiert – entsprechend dem Prinzip der Rollenspezialisierung im Multi-Agent-System-Muster (vgl. [Abschnitt 2.3](#)). Der **User-Prompt** formuliert die konkrete Aufgabe einschließlich aller relevanten Kontextinformationen.

Für alle Prompts wurde ein **Zero-Shot-Ansatz** gewählt. Diese Entscheidung ist durch die Heterogenität der Eingabedaten begründet: Da der Agent beliebige tabellarische Datensätze verarbeiten soll, wäre ein generisches Beispiel nicht repräsentativ und könnte zu einer Überanpassung an das Beispielformat führen (*example bias* [\[24\]](#)). Zudem kann die iterative Reparaturschleife des Viz Generators Qualitätsdefizite der Erstgenerierung kompensieren.

Da der Agent sowohl deutsche als auch englische Promptsprache unterstützt (vgl. Konfiguration), wurden sämtliche Prompts in beiden Sprachen mit identischem Inhalt angelegt. Zur Laufzeit werden die Prompt-Templates dynamisch anhand der konfigurierten Sprache geladen. Die vollständigen Prompts sind aufgrund ihres Umfangs nicht im Anhang aufgeführt, können jedoch dem öffentlich zugänglichen GitHub-Repository⁶ entnommen werden.

Module des Data-Science-Agenten

Die Modulstruktur des Data-Science-Agenten orientiert sich grundsätzlich an den vier Modulen von LIDA (vgl. [Abschnitt 2.5](#)), weicht jedoch in der konkreten Umsetzung und im Umfang ab. Auf den Infographer, der in LIDA aus Visualisierungen stilisierte Infografiken erzeugt, wurde verzichtet, da die vorliegende Arbeit auf die Qualität analytischer Visualisierungen fokussiert. Stattdessen wurde ein zusätzliches Modul zur systematischen Evaluation und Protokollierung integriert. [Abbildung 5](#) gibt einen Überblick über die vier Module und deren Zusammenspiel. Im Folgenden werden die Module entlang der in [Abbildung 4](#) dargestellten Pipeline-Schritte erläutert.

⁶<https://github.com/Elsper01/data-science-agent>

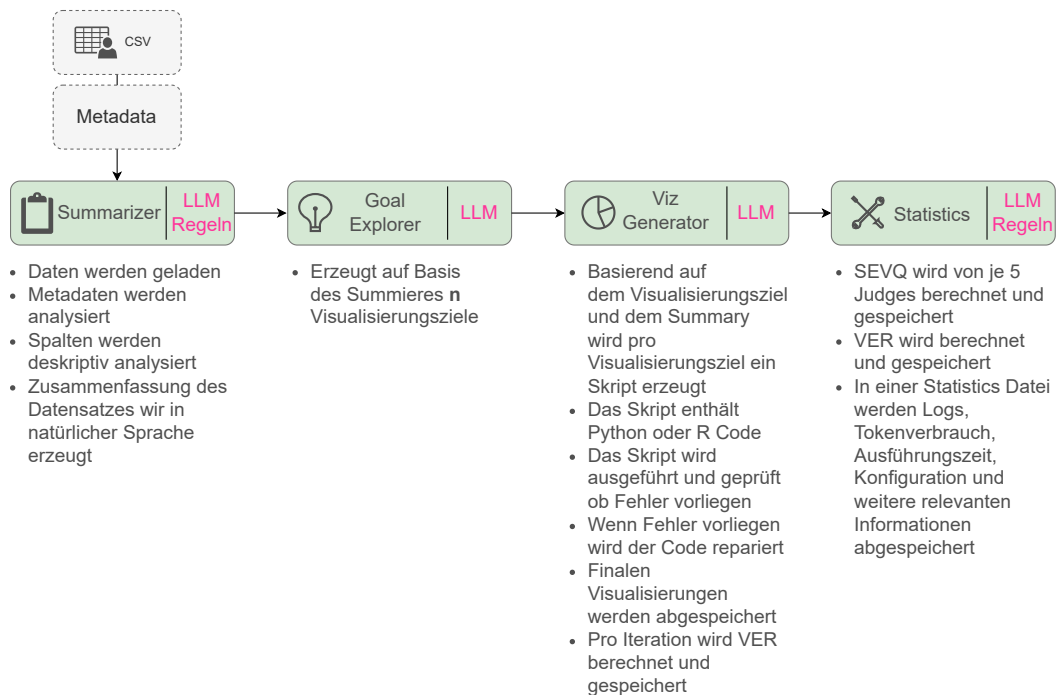


Abbildung 5: Übersicht über die vier Module des Data-Science-Agenten und deren Zusammenspiel. Ausgehend von einem tabellarischen Datensatz und zugehörigen Metadaten werden schrittweise Visualisierungen erzeugt und evaluiert. Jedes Modul ist mit seinen zentralen Verarbeitungsschritten dargestellt: Der Summarizer erzeugt eine natürlichsprachliche Zusammenfassung, der Goal Explorer leitet daraus Visualisierungsziele ab, der Viz Generator erzeugt und repariert Skripte iterativ, und das Statistics-Modul berechnet SEVQ und VER. Im Vergleich zu LIDA (vgl. [Abbildung 3](#)) entfällt der Infographer, während ein dediziertes Statistics-Modul für Evaluation und Protokollierung hinzukommt.

Summarizer

Das Summarizer-Modul transformiert einen tabellarischen Datensatz zusammen mit seinen Metadaten in eine kompakte natürlichsprachliche Zusammenfassung, die als Kontext für alle nachfolgenden Module dient. Konzeptuell entspricht dies dem Summarizer von LIDA (vgl. [Abschnitt 2.5](#)), jedoch wurde die Verarbeitung in mehrere spezialisierte Schritte unterteilt.

Im ersten Schritt werden die Eingabedaten geladen. Der Datensatz kann in einem beliebigen tabellarischen Format vorliegen, die Metadaten müssen jedoch im RDF-Format übergeben werden. Im zweiten Schritt erfolgt eine regelbasierte Analyse der Metadaten, bei der die relevantesten Textfragmente mit inhaltlicher Substanz extrahiert werden. Der dritte Schritt umfasst eine deskriptive Analyse der Datensatzspalten mittels `pandas.describe()`, die statistische Kennzahlen wie Mittelwert, Standardabweichung, Minimum, Maximum und Quartile berechnet. Im vierten und letzten Schritt wird ein LLM eingesetzt, um aus den Stichprobenzeilen des Datensatzes, der deskriptiven Spaltenanalyse und den extrahierten Metadaten eine kohärente Zusammenfassung in natürlicher Sprache zu generieren. Ein exemplarischer Output dieses Moduls ist in [Code-Fragment 1](#) zu finden.

Goal Explorer

Der Goal Explorer generiert auf Basis der vom Summarizer erzeugten Zusammenfassung n Visualisierungsziele, wobei n in der Konfiguration des Agenten festgelegt wird. Ein Visualisierungsziel besteht – analog zu LIDA (vgl. [Abschnitt 2.5](#)) – aus einer analytischen Fragestellung an die Daten, einem Vorschlag für den Visualisierungstyp und einer Begründung, warum diese Fragestellung relevant ist. Wie bereits in [Abschnitt 2.5](#) erläutert, hat sich das explizite Anfordern einer Begründung (Rationale) empirisch als entscheidend für die semantische Kohärenz der generierten Ziele erwiesen [\[7\]](#). Ein Beispiel-Output des Goal-Explorer-Moduls ist in [Code-Fragment 2](#) zu finden.

Viz Generator

Der Viz Generator bildet das komplexeste Modul des Data-Science-Agenten und erzeugt für jedes Visualisierungsziel ein ausführbares Skript. Basierend auf dem jeweiligen Visualisierungsziel und der Datensatz-Zusammenfassung generiert ein LLM ein Skript in der konfigurierten Programmiersprache (Python oder R).

Das generierte Skript wird anschließend ausgeführt, wobei sowohl die Standardausgabe als auch die Fehlerausgabe erfasst werden. Ein LLM prüft daraufhin, ob die Ausführung erfolgreich war oder ob eine Neugenerierung erforderlich ist. Dieser Mechanismus implementiert eine vereinfachte Variante des Reflexion-Patterns: Der Agent evaluiert seinen eigenen Output und verbessert ihn iterativ auf Basis des beobachteten Fehlerfeedbacks. Liegen Fehler vor, wird der Code repariert; maximal werden k Neuerzeugungsversuche unternommen, wobei k in der Konfiguration festgelegt ist. Erfolgreich erzeugte Visualisierungen werden abgespeichert.

Darüber hinaus wird pro Iteration die VER berechnet und protokolliert – eine Erweiterung gegenüber der einmaligen Berechnung in LIDA, die in [Abschnitt 3.2](#) definiert wird.

Statistics

Das Statistics-Modul stellt eine Erweiterung gegenüber LIDA dar und dient der systematischen Evaluation und Protokollierung. In diesem Modul wird die SEVQ – die von [V. Dibia \[7\]](#) eingeführte pointwise Evaluationsmetrik (vgl. [Abschnitt 2.4](#)) – für jede generierte Visualisierung berechnet. Dabei werden, dem in [Abschnitt 2.4](#) beschriebenen Multi-LLM-Evaluationsansatz folgend, fünf verschiedene LLM-Judges eingesetzt, um systematische Verzerrungen einzelner Modelle zu reduzieren (vgl. [Tabelle 2](#)). Zusätzlich wird die finale VER gemäß der ursprünglichen Definition aus dem LIDA-Paper berechnet und gespeichert.

Neben den Evaluationsmetriken protokolliert das Statistics-Modul umfassende Ausführungsinformationen: Logs, Token-Verbrauch pro Schritt im Graphen, Ausführungszeit pro Schritt, die verwendete Konfiguration sowie alle Zwischenergebnisse. Diese Protokollierung ist essenziell für die Reproduzierbarkeit der Experimente und ermöglicht eine detaillierte Analyse des Agentenverhaltens.

3.2. Evaluationsmetriken

Der Data-Science-Agent integriert die Evaluationsmetriken VER und SEVQ direkt in seine Pipeline, sodass jede Ausführung automatisch Qualitätskennzahlen erzeugt. Beide Metriken wurden ursprünglich von [V. Dibia \[7\]](#) eingeführt (vgl. [Abschnitt 2.5](#)) und für den Data-Science-Agenten angepasst. Im Folgenden werden sie definiert und kritisch diskutiert.

VER

Definition: Die **VER** misst den Anteil fehlgeschlagener Visualisierungsgenerierungen:

$$(2) \quad \text{VER} = \frac{E}{T} \cdot 100$$

mit

- E - Anzahl der fehlgeschlagenen Generierungen und
- T - Gesamtanzahl der generierten Visualisierungen.

Während [V. Dibia \[7\]](#) die VER einmalig am Ende einer Ausführung von LIDA berechnen, wird diese im Data-Science-Agenten nach jeder Iteration bestimmt, da der Codegenerierungsschritt bei Fehlern bis zu k -mal wiederholt wird. Zudem wird nicht die Kompilierbarkeit des Codes, sondern der tatsächliche Output bewertet: Eine Generierung gilt ausschließlich dann als erfolgreich, wenn tatsächlich eine Visualisierung erzeugt werden konnte.

Kritik: Die ursprüngliche VER misst nur die Ausführbarkeit des Codes. Moderne Sprachmodelle kompensieren Fehler jedoch häufig über Fehlerbehandlungskonstrukte (z.B. **try-catch**), sodass inhaltlich fehlerhafte Generierungen als erfolgreich bewertet werden können. Die hier vorgestellte Variante reduziert dieses Problem, erkennt jedoch weiterhin keine semantischen Fehler, beispielsweise leere oder inhaltlich falsche Visualisierungen. Ein vielversprechender Ansatz zur Erkennung solcher semantischen Fehler wäre der Einsatz von Visual Language Models (VLMs), die das erzeugte Bild direkt analysieren und auf inhaltliche Korrektheit prüfen könnten. Dieser Ansatz wird in [Abschnitt 4.4](#) als mögliche Weiterentwicklung diskutiert.

SEVQ

Definition: Die **SEVQ** ist ein LLM-as-a-Judge-Ansatz, der erstmals von [V. Dibia \[7\]](#) vorgestellt wurde. Dabei bewertet ein Sprachmodell den generierten Visualisierungscode zusammen mit dem jeweiligen Visualisierungsziel auf einer Skala von 1 (sehr schlecht) bis 10 (sehr gut) in sechs Dimensionen:

- **bugs:** Anzahl und Schwere von Fehlern im Code.
- **transformation:** Korrektheit und Sinnhaftigkeit der Datentransformation.
- **compliance:** Erfüllung des Visualisierungsziels durch den Code.
- **type:** Passung des Visualisierungstyps zu Ziel, Daten und Intention.
- **encoding:** Korrekte Kodierung der Daten.
- **aesthetics:** Ästhetische Qualität der Gestaltung.

Diese Dimensionen erfassen zentrale Qualitätsaspekte einer Visualisierung; eine ausführliche Begründung und Erklärung der Metrik liefert [V. Dibia \[7\]](#). Im Data-Science-Agenten werden – dem

in [Abschnitt 2.4](#) beschriebenen Multi-LLM-Evaluationsansatz folgend – fünf verschiedene LLM-Judges für die SEVQ-Berechnung eingesetzt, um die in [Tabelle 2](#) dokumentierten systematischen Verzerrungen einzelner Modelle zu reduzieren.

Die im Anhang aufgeführten Prompts ([Code-Fragment 3](#) und [Code-Fragment 4](#)) zeigen die deutschsprachigen System- und User-Prompts, die das Sprachmodell als Eingabe erhält, um als Judge zu bewerten. Die resultierende Bewertung anhand der sechs genannten Dimensionen wird als SEVQ bezeichnet.

Kritik: Da SEVQ nur den Code bewertet, kann es in Fällen fehlgeschlagener oder unvollständiger Visualisierungen zu überoptimistischen Bewertungen kommen – insbesondere in der Dimension **bugs**. Zudem deutet sich an, dass die Bewertung stark vom jeweils verwendeten Judge-Sprachmodell abhängt. Diese Beobachtung wird in Forschungsfrage RQ-1 untersucht.

3.3. Abgrenzung von LIDA

Der Data-Science-Agent übernimmt die konzeptionelle Grundidee von LIDA – Visualisierungsgenerierung als mehrstufiges Textgenerierungsproblem (vgl. [Abschnitt 2.5](#)) – und erweitert sie in drei wesentlichen Dimensionen: Erstens wird die implizite MAS-Struktur von LIDA als explizites LangGraph-System mit Shared State und layered-Topologie umgesetzt (vgl. [Abschnitt 3.1](#)). Zweitens wird die Evaluationsstrategie durch iterative VER-Berechnung pro Iteration und den Einsatz von fünf LLM-Judges für die SEVQ erweitert, um systematische Verzerrungen einzelner Modelle zu reduzieren. Drittens ermöglicht eine systematische Konfigurierbarkeit (LLM-Modell, Promptsprache, Skriptsprache, n , k) gezielte Experimente, die in LIDA nicht vorgesehen sind. Der Infographer von LIDA entfällt zugunsten eines dedizierten Statistics-Moduls für Evaluation und Protokollierung. Eine vollständige dimensionale Gegenüberstellung findet sich in [Tabelle 16](#) im Anhang.

4. Versuchsaufbau

Ziel dieser Arbeit ist die Evaluation des entwickelten Data-Science-Agenten. Untersucht wird, ob der Agent für beliebige tabellarische Datensätze sinnvolle Visualisierungen erzeugen kann und welche Systemkonfiguration die besten Ergebnisse liefert. Die empirische Untersuchung umfasst automatisierte Experimente sowie eine Online-Studie, in der erfahrene Teilnehmende die Agentenergebnisse beurteilen.

Die Untersuchung adressiert die folgenden, in der Einleitung eingeführten Forschungsfragen:

- **RQ-1 (Evaluierungsmetriken):** Eignen sich die von [V. Dibia \[7\]](#) eingeführten Metriken zur automatisierten Bewertung der Ergebnisqualität des Data-Science-Agenten?
- **RQ-2 (Konfiguration):** Welchen Einfluss haben die verwendete Promptsprache und die Programmiersprache im Codegenerierungsschritt auf die Ergebnisqualität?
- **RQ-3 (Generalisierbarkeit):** Ist der Data-Science-Agent für beliebige tabellarische Datensätze geeignet?

- **RQ-4 (Metadaten):** Verbessert die Integration von Metadaten die Qualität der erzeugten Visualisierungen?
- **RQ-5 (Stabilität):** Erzielt der Data-Science-Agent bei wiederholter Ausführung auf denselben Datensätzen konsistent gute Ergebnisse?

Zur Bewertung der Ergebnisqualität werden die Metriken VER und SEVQ eingesetzt, die in [Abschnitt 3.2](#) definiert und kritisch diskutiert werden. Im Folgenden werden der verwendete Datensatz ([Abschnitt 4.1](#)), das Studiendesign ([Abschnitt 4.2](#)), das Analyseverfahren ([Abschnitt 4.3](#)) zur Beantwortung der Forschungsfragen sowie Beeinträchtigungen der Validität ([Abschnitt 4.4](#)) beschrieben.

4.1. Datensatz

Dieser Abschnitt beschreibt die Datengrundlage der Evaluation. Da das GovData-Portal einen eigenen Datensatzbegriff verwendet, der von der Perspektive des Data-Science-Agenten abweicht, werden zunächst die relevanten Begriffe definiert.

Begriffsbestimmung

Das GovData-Portal setzt sich aus einer Menge von Dateien, welche als *Distributionen* bezeichnet werden, und einer Menge an Metadaten zusammen. Diese sind thematisch in sogenannten Datensätzen organisiert.

Ein Datensatz im Sinne des GovData-Portals ($D_{\text{Gov-Data}}$) besteht aus mindestens einer Distribution sowie den zugehörigen beschreibenden Metadaten. Die Metadaten charakterisieren das Thema des Datensatzes, während die Distributionen die eigentlichen Daten enthalten. Der Großteil der Distributionen umfasst tabellarisch strukturierte Daten.

Ein Datensatz im Sinne des Data-Science-Agenten ($D_{\text{Data-Science-Agent}}$) ist eine einzelne tabellarische Distribution mit den zugehörigen Metadaten. Für den Data-Science-Agenten sind somit nur tabellarische Distributionen eines GovData-Datensatzes relevant.

Evaluierungs-Datensatz

Die Datenerhebung wurde am 26. Januar 2026 um 15:30 CET durchgeführt. Zu diesem Zeitpunkt wurden sämtliche $D_{\text{Gov-Data}}$ Datensätze des GovData-Portals automatisiert erfasst:

- Erfasste $D_{\text{Gov-Data}}$ Datensätze: 147.259
- Enthaltene Distributionen: 364.192
- Davon CSV-Distributionen: 112.474
- $D_{\text{Gov-Data}}$ Datensätze mit CSV-Distributionen: 48.698 ($\approx 33,1$ % aller $D_{\text{Gov-Data}}$ Datensätze)

Für die Evaluation wurden 50 $D_{\text{Gov-Data}}$ Datensätze zufällig ausgewählt (im Folgenden D_{eval}), die jeweils mindestens eine CSV-Distribution enthalten. Für jeden D_{eval} Datensatz wurde eine der CSV-Distributionen zufällig bestimmt und heruntergeladen, um sie als $D_{\text{Data-Science-Agent}}$ Datensatz zu verwenden.

Das GovData-Portal basiert auf **CKAN** [\[44\]](#), einer Open-Source-Plattform für Datenportale.

CKAN⁷ stellt neben einem reichhaltigen Frontend eine vollständige REST-API bereit, über die Metadaten strukturiert abgerufen werden können. Über diese API wurden die Metadaten aller D_{eval} Datensätze heruntergeladen. Dies war für 47 der 50 D_{eval} Datensätze erfolgreich. Die verbleibenden drei Datensätze, bei denen der Abruf der Metadaten fehlschlug, verbleiben im Evaluierungs-Datensatz, da solche Ausfälle die reale Datenqualität offener Verwaltungsdaten widerspiegeln.

Für die weiteren Experimente wird die Menge aus 50 D_{eval} Datensätzen mit jeweils einem ausgewählten $D_{\text{Data-Science-Agent}}$ Datensatz und entsprechendem Metadatum als **Evaluierungs-Datensatz** bezeichnet.

4.2. Umsetzung der Studie

Ziel der Studie war die Bestimmung einer geeigneten Agentenkonfiguration, die Validierung des **LLM-as-a-Judge**-Ansatzes für die Metrik **SEVQ**, die Untersuchung der Generalisierbarkeit des Systems und die Analyse der Stabilität der Ergebnisse. Die Online-Erhebung wurde mit LamaPoll⁸ durchgeführt. Teilgenommen haben Personen mit Erfahrung in Datenverarbeitung und Visualisierung.

- Studiendauer: 25 Tage
- Teilnehmende: 14 Personen
- Vollständiger Fragebogen: Die Studie wurde als HTML Datei gespeichert und kann unter folgendem Link aufgerufen werden: [Github](#)

Die Studie umfasst vier Blöcke mit insgesamt 32 Fragen.

SEVQ

Im ersten Block (Fragen 1 - 12) bewerteten die Teilnehmenden durch den Agenten erzeugte Visualisierungen entlang der sechs **SEVQ**-Dimensionen (*bugs*, *transformation*, *compliance*, *type*, *encoding*, *aesthetics*) auf einer Skala von 1 (sehr schlecht) bis 10 (sehr gut). Zur Sicherstellung einer konsistenten Bewertung wurde den Teilnehmenden in der Studie ein Ankerbeispiel bereitgestellt, in welchem eine Visualisierung begründet hinsichtlich aller Dimensionen bewertet wurde. Die Erhebung erfolgte über eine Bewertungsmatrix. [Abbildung 6](#) zeigt eine Beispielfrage aus der Studie.

⁷<https://ckan.govdata.de>

⁸<https://survey.lamapoll.de/Data-Science-Agent>

Bewerten Sie die Visualisierung*

Ziel der Visualisierung:

Wie hat sich die Gamma-Ortsdosisleistung im Zeitverlauf an der Station Kornbach verändert?

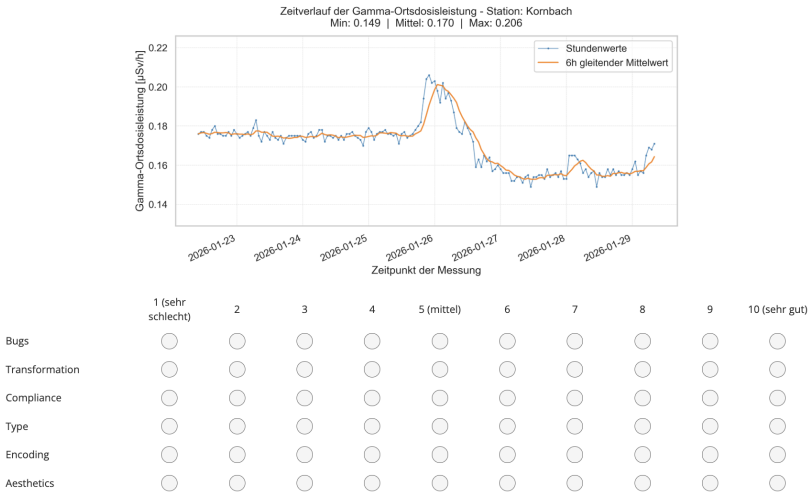


Abbildung 6: Bewertungsmatrix-Frage aus der Studie, in welcher Nutzende aufgefordert werden, alle Dimensionen mit einem Wert von 1 (sehr schlecht) bis 10 (sehr gut) zu bewerten. Die Bewertung entspricht der SEVQ-Bewertung, vorgestellt durch V. Dibia [7] mittels eines LLM-as-a-Judge.

Die Skripte umfassen teils bis zu 900 Zeilen Code und nicht alle Teilnehmenden sind mit R oder Python vertraut, deswegen wurden in der Befragung Visualisierungen anstelle des Codes präsentiert. Dies verbessert zugleich die Vergleichbarkeit, da mehrere **SEVQ**-Dimensionen (insb. *bugs* und *transformation*) direkt am Visualisierungsergebnis beurteilbar sind.

Es wurden Visualisierungen in allen vier Konfigurationen verwendet. Eine genaue Aufschlüsselung ist in Tabelle 4 zu finden.

Fragen	Sprache	Programmiersprache
1 - 3	Deutsch	R
4 - 6	Deutsch	Python
7 - 9	Englisch	R
10 - 12	Englisch	Python

Tabelle 4: Aufschlüsselung der SEVQ-Fragen in der Studie nach Konfiguration (RQ-1)

Der Evaluierungs-Datensatz wurde jeweils vollständig mit der angegebenen Kombination ausgeführt. Aus den erzeugten Ergebnissen wurden pro Bedingung zufällige Visualisierungen für die Befragung ausgewählt. Da auch die Promptsprache untersucht wurde, sind einige Visualisierungsziele auf Englisch formuliert. In der Instruktion wurde darauf hingewiesen, dass die Sprache kein Bewertungskriterium darstellt.

Sprache & Programmiersprache

Der zweite Block (Fragen 13 - 24) diente der Beantwortung von RQ-2, also der Wirkung von Promptsprache und Programmiersprache. Dazu wurden in A/B-Vergleichen jeweils zwei Konfigurationen gegenübergestellt. Eine genaue Aufschlüsselung welche Konfigurationen in welcher Frage verglichen wurden ist in [Tabelle 5](#) zu finden.

Fragen	Version A	Version B
13 - 15	Deutsch & R	Deutsch & Python
16 - 18	Deutsch & R	Englisch & R
19 - 21	Deutsch & Python	Englisch & Python
22 - 24	Englisch & R	Englisch & Python

Tabelle 5: Aufschlüsselung der Vergleichsfragen zur Beurteilung der Konfiguration (RQ-2)

Pro Konfiguration wurden drei Visualisierungen desselben Datensatzes gezeigt, um Reihenfolgen- und Positions-Bias zu vermeiden. Teilnehmende wählten die Variante, die sie fachlich überzeugender, thematisch passender und visuell ansprechender fanden. [Abbildung 7](#) ist ein Beispiel aus der Studie.

Wählen Sie die Ihrer Meinung nach passendere Variante aus*

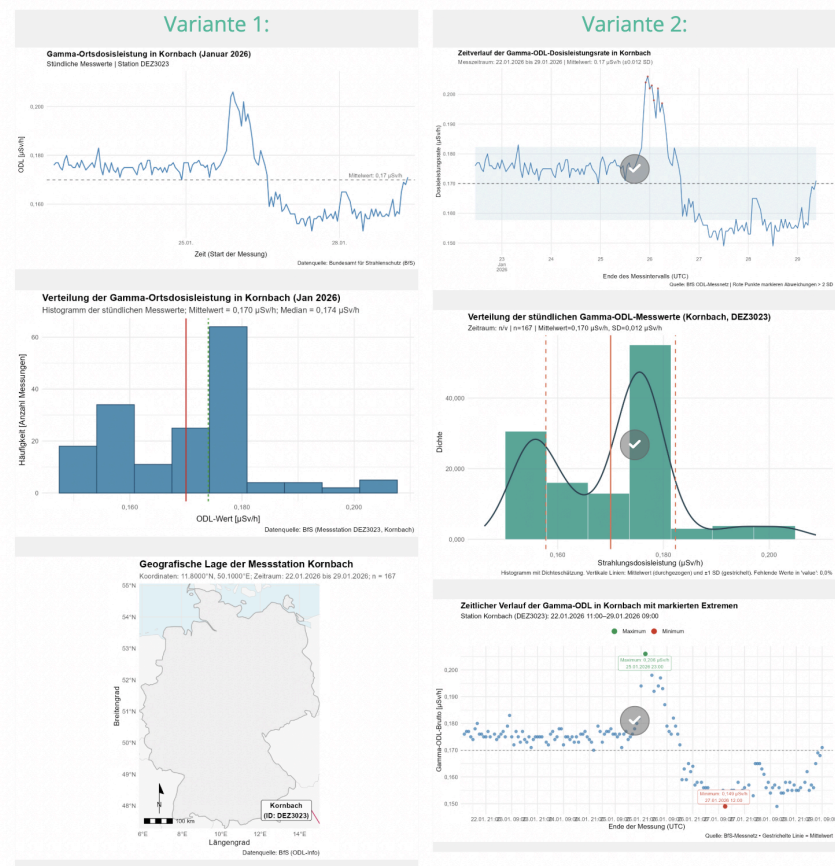


Abbildung 7: A/B-Frage aus der Studie, in welcher Nutzende zwischen den beiden Versionen auswählen müssen, welches Ergebnis sie passender finden. Im Hintergrund werden die Ergebnisse des Data-Science-Agenten mit verschiedenen Konfigurationen verglichen.

Metadaten

Zur Untersuchung von RQ-4 (Metadaten) wurde der Data-Science-Agent zweimal auf dem Evaluierungs-Datensatz ausgeführt – einmal **mit** und einmal **ohne** Metadatenintegration. Mit Metadatenintegration ist gemeint, dass im Agentensystem der Metadaten-Agent aktiviert wird, der bei der Generierung der Zusammenfassung zum Einsatz kommt und dadurch einen indirekten Einfluss auf alle weiteren Agentenergebnisse hat. Nur Datensätze mit erfolgreichen Ausführungen in beiden Bedingungen wurden berücksichtigt.

Auf Grundlage der **SEVQ**-Ergebnisse wurden zwei Konfigurationen gewählt: Deutsch & Python sowie Englisch & Python – jene Kombinationen, die in den vorangegangenen Analysen die besten Resultate zeigten. Insgesamt umfasste dieser Block sechs Fragen, welche in **Tabelle 6** nach Konfiguration aufgeschlüsselt sind und Auskunft gibt in welcher Version Metadaten verwendet wurde.

Fragen	Sprache	Programmiersprache	Version A	Version B
25 - 27	Deutsch	Python	Mit Metadaten	Ohne Metadaten
28 - 30	Englisch	Python	Mit Metadaten	Ohne Metadaten

Tabelle 6: Aufschlüsselung der Fragen nach Konfiguration und Angabe in welcher Version Metadaten verwendet wurden.

Für jede Konfiguration wurden aus der Schnittmenge der erfolgreichen Datensätze mit und ohne Metadaten drei Beispiele zufällig ausgewählt und im A/B-Design präsentiert. Der Aufbau der Fragen entspricht einer Frage im Block „Sprache & Programmiersprache“ (vgl. [Abbildung 7](#)), der zentrale Unterschied ist hier, dass die Konfiguration gleichbleibt, jedoch die Ausführung mit Metadaten-Schritt und ohne durchgeführt wird.

Gleichbleibende Qualität

Zur Beantwortung von RQ-5 wurde die Ergebnisstabilität untersucht. Hierfür wurde die zuvor als bestgeeignet identifizierte Konfiguration **Englisch & Python** verwendet. Aus den erfolgreichen Datensätzen wurden fünf zufällig ausgewählt und je fünfmal erneut verarbeitet. Für die Befragung wurden daraus zwei Datensätze herangezogen. Teilnehmende bewerteten die Konsistenz der jeweils fünf Visualisierungen auf einer 5-Punkt Likert Skala [\[45\]](#) entlang folgender Dimensionen:

- *Klarheit*: Sind die Visualisierungen über die fünf Durchläufe hinweg ähnlich klar und verständlich?
- *Aussagekraft / Relevanz*: Liefern die Visualisierungen stets vergleichbare Erkenntnisse zum Datensatz?
- *Gestalterische Qualität / Layout*: Bleibt die grafische Gestaltung (Farben, Achsenbeschriftung, Lesbarkeit) gleich hochwertig?
- *Informationsgehalt*: Bleibt der Umfang und die Tiefe der dargestellten Informationen konstant?
- *Interpretierbarkeit*: Sind die Visualisierungen über die Läufe hinweg gleich gut interpretierbar?
- *Plausibilität / Korrektheit*: Vermitteln die Visualisierungen über alle Durchläufe hinweg den gleichen Eindruck von Datenrichtigkeit?

In [Abbildung 8](#) ist eine der beiden Fragen aus der Studie zu sehen.

Bewerten Sie wie gleichbleibend die Qualität der einzelnen Durchläufe ist*

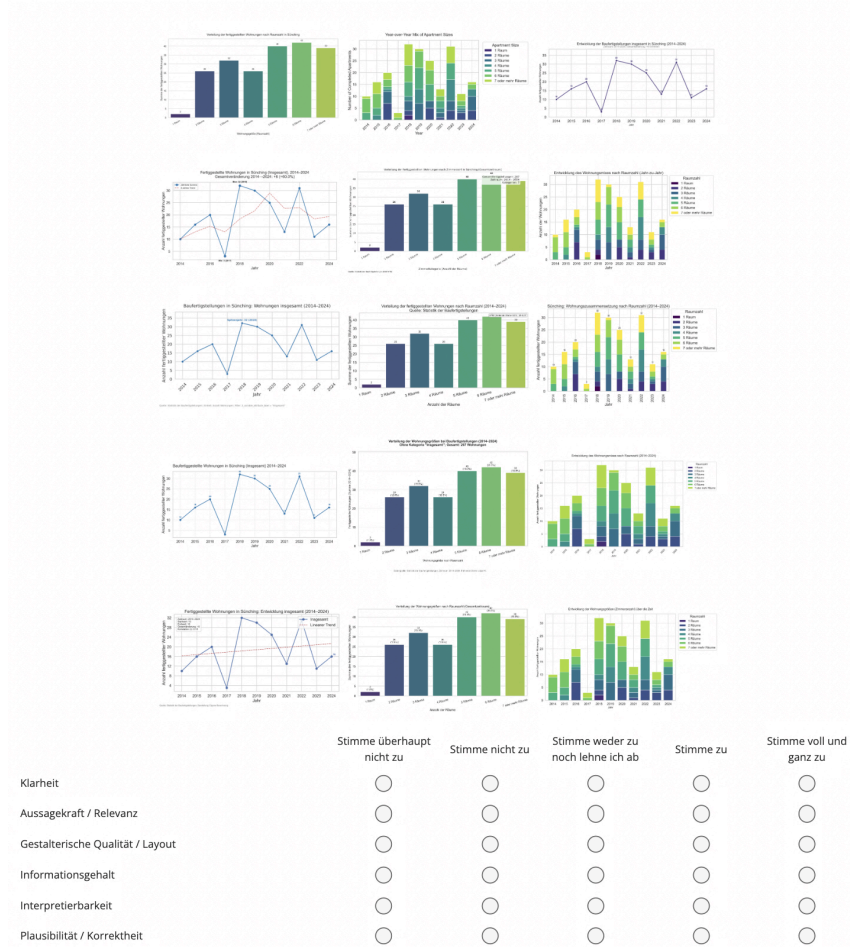


Abbildung 8: Bewertungsmatrix-Frage aus der Studie, in welcher Nutzende aufgefordert werden, alle Dimensionen mit einem Wert von 1 (sehr schlecht) bis 5 (sehr gut) zu bewerten. Ziel ist es zu bewerten, ob die Qualität der Ergebnisse des Data-Science-Agenten stabil ist.

4.3. Analyseverfahren

In diesem Abschnitt wird erläutert, wie die einzelnen Forschungsfragen analysiert und ausgewertet werden. Vorab wird die gewählte Basiskonfiguration begründet, welche für alle Analysen als Grundlage dient.

Die Signifikanzvergleiche erfolgen mittels gepaarter t-Tests [46] bei einem Signifikanzniveau von $\alpha = 0.05$. Da pro Forschungsfrage mehrere paarweise Vergleiche durchgeführt werden, wird eine Bonferroni-Korrektur [47] mit korrigiertem Signifikanzniveau $\alpha_{\text{korr}} = \frac{\alpha}{m}$ angewendet, wobei m die Anzahl der Tests innerhalb einer Vergleichsfamilie bezeichnet. Zur Bewertung der Effektgröße wird Cohen's d [48] berechnet mit den konventionellen Schwellenwerten: $|d| < 0.2$ (vernachlässigbar), $0.2 \leq |d| < 0.5$ (klein), $0.5 \leq |d| < 0.8$ (mittel) und $|d| \geq 0.8$ (groß). Bei signifikanten Unterschieden wird jeweils die Effektstärke ermittelt und interpretiert.

Bei ordinalskalierten oder nicht-normalverteilten Daten werden nicht-parametrische Verfahren eingesetzt: der Kruskal-Wallis-Test zum Vergleich von mehr als zwei unabhängigen Gruppen sowie paarweise Mann-Whitney-U-Tests mit Bonferroni-Korrektur als Post-hoc-Verfahren.

Inter-Rater-Agreement

Zur Quantifizierung der Übereinstimmung zwischen mehreren Ratern – sowohl zwischen LLM-Judges (RQ-1) als auch zwischen Studienteilnehmenden – werden zwei komplementäre Maße eingesetzt.

Der Intraclass Correlation Coefficient ICC(2,1) nach [P. E. Shrout und J. L. Fleiss \[49\]](#) quantifiziert die absolute Übereinstimmung unter der Annahme, dass sowohl Bewertungsobjekte als auch Rater zufällige Stichproben darstellen (Two-way Random, Single Measures):

$$(3) \quad \text{ICC}(2, 1) = \frac{\text{MS}_R - \text{MS}_E}{\text{MS}_R + (k - 1) \cdot \text{MS}_E + \frac{k \cdot (\text{MS}_C - \text{MS}_E)}{n}}$$

Dabei bezeichnet MS_R die mittleren Abweichungsquadrate zwischen den Bewertungsobjekten, MS_C zwischen den Ratern, MS_E die Residuen, k die Rateranzahl und n die Objektanzahl. Zur Interpretation dienen die Richtlinien von [T. K. Koo und M. Y. Li \[50\]](#): ICC < 0.50 gilt als schwach, 0.50–0.75 als moderat, 0.75–0.90 als gut und > 0.90 als exzellent.

Ergänzend wird Krippendorffs α [\[51\]](#) berechnet, das weniger Annahmen über die Datenstruktur voraussetzt:

$$(4) \quad \alpha_K = 1 - \frac{D_o}{D_e}$$

Werte von $\alpha_K \geq 0.667$ gelten als akzeptabel, $\alpha_K \geq 0.800$ als reliabel [\[51\]](#). Während der ICC systematische Niveauunterschiede zwischen Ratern bestraft, erfasst α_K die ordinale Konsistenz. Beide Maße werden komplementär berichtet, um Niveau- und Rangeffekte zu trennen.

Kriteriumsvalidität und Agreement-Metrik

Zur Überprüfung der Kriteriumsvalidität der LLM-Judges gegenüber menschlichen Urteilen wird die Agreement-Metrik nach [\[32\]](#) berechnet:

$$(5) \quad \text{Agreement} = \frac{\sum_{i \in D} I(|S_{\text{llm},i} - S_{\text{human},i}| \leq \tau)}{|D|}$$

Dabei bezeichnet $S_{\text{llm},i}$ die gemittelte SEVQ-Bewertung der Judges für Visualisierung i , $S_{\text{human},i}$ die gemittelte Bewertung der Studienteilnehmenden und τ einen Toleranzparameter, da auf einer 60-Punkte-Skala exakte Übereinstimmung unrealistisch ist. Ergänzend werden Spearman- ρ , Pearson- r , MAE und Mean Bias berichtet. Zur visuellen Beurteilung systematischer Verzerrungen wird der Bland-Altman-Plot [\[52\]](#) eingesetzt, der die Differenz zweier Messungen gegen deren Mittelwert aufträgt und 95%-Übereinstimmungsgrenzen ausweist.

Basiskonfiguration

Wie in [Abschnitt 3.1](#) beschrieben, kann für jeden Schritt im LLM-Pipeline-Prozess unabhängig ein Sprachmodell konfiguriert werden. Um mit der Analyse beginnen zu können, wurde zunächst eine Basiskonfiguration bestimmt. Vorexperimente zeigten, dass die Ergebnisqualität des Data-Science-Agenten vor allem vom Codegenerierungsschritt abhängt. Für die übrigen Schritte wurde daher **Gemini** (für die genaue Modellspezifikation siehe [Tabelle 7](#)) ausgewählt - ein modernes (Veröffentlichung im Dezember 2025), leistungsfähiges und zuverlässiges Modell mit kurzen Ausführungszeiten und konsistenten Ergebnissen. Die einzige Ausnahme bildet der *decide_regeneration*-Schritt. Hier wurde das ältere Sprachmodell **GPT-4o** (vgl. [Tabelle 7](#)) gewählt, da die Aufgabe darin besteht, anhand von *std_out* und *std_error* zu entscheiden, ob eine erneute Generierung nötig ist. Bei der Implementierung wurde festgestellt, dass modernere Modelle mit umfangreichen Thinking-Schritten schlechtere Ergebnisse liefern als GPT-4o.

Zur Bestimmung des optimalen Sprachmodells im Codegenerierungsschritt wurde folgendes Vorgehen gewählt:

- Auswahl von 10 zufälligen Tupeln aus dem Evaluierungs-Datensatz
- Konfiguration: Deutsch & R
- getestete Modelle: alle in [Tabelle 7](#) aufgelisteten Kandidaten
- Durchführung des Data-Science-Agenten je Modell auf den 10 Samples
- Erfassung der Metriken **VER** und **SEVQ**. Ohne RQ-1 evaluiert zu haben wird angenommen, dass SEVQ und VER, so wie sie von [V. Dibia \[7\]](#) vorgestellt wurden, aussagekräftige Metriken sind.

Die Stichprobengröße von $n = 10$ wurde bewusst gewählt, da jede vollständige Ausführung des Data-Science-Agenten mit erheblichen API-Kosten und langen Ausführungszeiten verbunden ist. Pro Modell und Datensatz erfordert ein einzelner Run mehrere aufeinanderfolgende LLM-Aufrufe, sodass die Gesamtkosten für die Modellauswahl mit der Anzahl der Kandidatenmodelle multipliziert linear skalieren. Die gewählte Stichprobengröße stellt somit einen Kompromiss zwischen ökonomischen Rahmenbedingungen und statistischer Aussagekraft dar. Es ist zu beachten, dass bei $n = 10$ die statistische Power gepaarter t-Tests gering ist, weshalb nicht-signifikante p -Werte nicht als Evidenz für die Abwesenheit eines Effekts interpretiert werden sollten. Zur Absicherung der Modellwahl werden daher ergänzend Effektstärken (Cohen's d) herangezogen, die weniger anfällig gegenüber kleinen Stichprobenumfängen sind.

Die Konfiguration *Deutsch & R* wurde auf Grundlage einer Vorstudie als Basiskonfiguration für die Modellauswahl festgelegt. In dieser Vorstudie wurde der Data-Science-Agent mit verschiedenen Kombinationen aus Promptsprache und Programmiersprache auf einer Teilmenge des Evaluierungs-Datensatzes ausgeführt. Die Ergebnisse deuteten darauf hin, dass *Deutsch & R* eine geeignete Konfiguration darstellt. Da die Modellauswahl zeitlich vor der systematischen Untersuchung aller Konfigurationen (RQ-2) erfolgte, wurde diese Konfiguration als Ausgangspunkt verwendet. Die möglichen Auswirkungen dieser Entscheidung auf die Modellwahl werden in [Abschnitt 4.4](#) diskutiert.

Wie [Abbildung 9](#) zeigt, erzielte **GPT-5** die höchsten mittleren SEVQ-Werte. Obwohl die Unterschiede zu anderen Modellen ($p_{\text{claude}} = 0.39$, $p_{\text{gemini}} = 0.34$, $p_{\text{grok}} = 0.41$), bis auf GPT-4o ($p_{\text{gpt-4o}} = 0.01$), bei dem gewählten Stichprobenumfang statistisch nicht signifikant waren, weisen die

berechneten Effektstärken auf mittlere bis starke Effekte hin ($d_{\text{claude}} = 0.55$, $d_{\text{gemini}} = 0.57$, $d_{\text{gpt-4o}} = 1.11$, $d_{\text{grok}} = 0.54$). Die konsistent positiven Effektstärken zugunsten von GPT-5 über alle Vergleiche hinweg stützen die Modellwahl trotz der begrenzten statistischen Aussagekraft. Für **Mistral** liegen aufgrund häufiger Kommunikationsfehler mit dem Sprachmodell keine validen Ergebnisse vor. Das Modell wurde daher ausgeschlossen.

Kurzname	Offizieller Modellname	Veröffentlichungsdatum
Mistral	mistralai/mistral-large-2412	Dez 2025
GPT-5	openai/gpt-5	Aug 2025
GPT-4o	openai/gpt-4o	Mai 2024
Grok	xai/grok-4.1-fast	Nov 2025
Gemini	google/gemini-3-flash	Dez 2025
Claude Sonnet	anthropic/claude-sonnet-3.5	Sep 2025

Tabelle 7: Übersicht der verwendeten Sprachmodelle

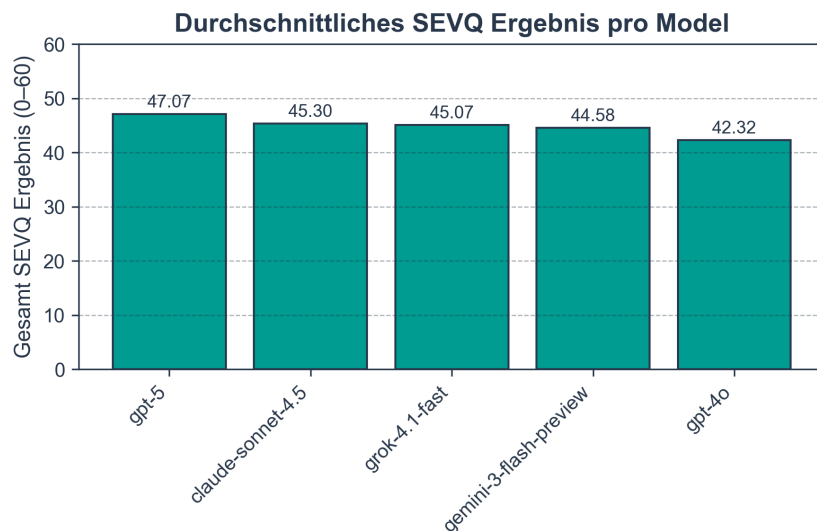


Abbildung 9: Durchschnittliche SEVQ-Gesamtbewertung (Summe der sechs Dimensionen, Skala 0–60) über alle Visualisierungen nach Ausführung der einzelnen Sprachmodelle im Codegenerierungsschritt auf einem Sample ($n = 10$) des Evaluierungs-Datensatzes mit der Basiskonfiguration *Deutsch & R*.

Reproduzierbarkeit

Alle Experimente wurden unter dokumentierten Bedingungen durchgeführt. Zur Sicherstellung der Reproduzierbarkeit wurden die folgenden Maßnahmen ergriffen: Für alle LLM-Aufrufe wurde `temperature = 0` konfiguriert, um deterministische Ausgaben zu erzielen. Seeds wurden, soweit von der jeweiligen API unterstützt, explizit gesetzt. Die verwendeten Python-Version, Package-Abhängigkeiten, API-Endpunkte und -Modellversionen (vgl. [Tabelle 7](#)) sowie die eingesetzte

Hardware sind im zugehörigen GitHub-Repository vollständig spezifiziert⁹. Sämtliche Rohdaten der Experimente, die vollständigen Studienergebnisse sowie die Auswertungsskripte sind ebenfalls im Repository hinterlegt, sodass eine vollständige Reproduktion der Ergebnisse möglich ist.

RQ-1 - Evaluierungsmetriken

Ziel ist die Untersuchung, wie zuverlässig die automatisierte SEVQ-Bewertung durch LLMs als „Judge“ funktioniert. Da vermutet wird, dass die Urteile stark vom verwendeten Judge-Modell abhängen, wurden fünf verschiedene LLM-Judges integriert, die sich ausschließlich im verwendeten Sprachmodell unterscheiden (siehe [Tabelle 7](#), ohne **Mistral**).

Zur Quantifizierung der Übereinstimmung zwischen den Judges wird das Inter-Rater-Agreement über den ICC bestimmt. Darüber hinaus wird geprüft, ob eine Kombination mehrerer Judges (Schnitt- oder Mittelwertbildung) konsistentere SEVQ-Ergebnisse liefert. Abschließend erfolgt ein Vergleich der automatisierten Bewertungen mit den menschlichen Einschätzungen aus der Studie.

RQ-2 - Einfluss der Konfiguration

Analysiert wird, welche Konfiguration des Data-Science-Agenten die besten SEVQ- und VER-Ergebnisse liefert. Hierzu werden die in [Abschnitt 4.2](#) aufgelisteten Kombinationen (Sprach- und Programmiersprache) verglichen und mit den Präferenzen der Studienteilnehmenden abgeglichen.

RQ-3 - Generalisierbarkeit

Zur Beurteilung der Generalisierbarkeit wird der Agent in allen vier Konfigurationen auf dem vollständigen Evaluierungs-Datensatz ($N = 50$) ausgeführt. Die VER allein erfasst nur die Fehlerrate innerhalb einzelner Visualisierungen und setzt voraus, dass die Visualisierungsphase überhaupt erreicht wird. Für eine umfassende Bewertung der Generalisierbarkeit werden daher drei ergänzende Metriken eingeführt.

Die **Dataset Coverage (DC)** misst den Anteil der Datensätze, bei denen der Agent die Visualisierungsphase erreicht:

$$(6) \quad DC = \frac{D_{\text{viz}}}{D_{\text{gesamt}}} \cdot 100$$

Dabei bezeichnet D_{viz} die Anzahl der Datensätze, bei denen mindestens eine VER-Iteration durchlaufen wurde, und $D_{\text{gesamt}} = 50$ die Gesamtanzahl im Experiment.

Die **Convergence Error Rate (CER)** quantifiziert den Anteil der Datensätze, die die Visualisierungsphase erreichten, aber nicht alle Visualisierungen erfolgreich abschlossen:

$$(7) \quad CER = \frac{D_{\text{viz}} - D_{\text{conv}}}{D_{\text{viz}}} \cdot 100$$

⁹<https://github.com/Elsper01/data-science-agent>

Hierbei ist D_{conv} die Anzahl der Datensätze, bei denen in der letzten Iteration alle geplanten Visualisierungen erzeugt wurden.

Die **Average Iterations Required (AIR)** gibt den mittleren Korrekturaufwand über die konvergierten Datensätze an:

$$(8) \quad \text{AIR} = \frac{1}{|D_{\text{conv}}|} \sum_{d \in D_{\text{conv}}} I_d$$

mit I_d als Anzahl der Iterationen bis zur vollständigen Konvergenz von Datensatz d . Die AIR ist nur über konvergierte Datensätze definiert, da nicht-konvergierte Datensätze kein eindeutiges Ende besitzen.

Ergänzend wird die VER auf Basis der ersten Iteration (VER_0) berichtet, da nur diese den unkorrigierten Erstversuch des Agenten widerspiegelt. Die gemeldete VER_0 ist der Mittelwert der datensatzspezifischen Fehlerraten über alle Datensätze, die die Visualisierungsphase erreicht haben.

RQ-4 - Einfluss von Metadaten

Für diese Forschungsfrage wird eine Ablationsstudie durchgeführt, bei der Runs **mit** und **ohne** Metadatenintegration verglichen werden. Hierfür werden VER und SEVQ untersucht und mit den Nutzerentscheidungen aus der Studie verglichen.

RQ-5 - Stabilität

Der Data-Science-Agent wird auf fünf zufällig gewählten Samples aus dem Evaluierungs-Datensatz je fünfmal ausgeführt. Untersucht wird, wie stabil sich **SEVQ** und **VER** über mehrere Runs verhält. Zusätzlich werden die Nutzerentscheidungen aus der Studie ausgewertet, in welchen die Qualität der Ergebnisse der einzelnen Runs untereinander bewertet wurden.

4.4. Beeinträchtigung der Validität

Interne Validität

Vier Faktoren können die interne Validität beeinträchtigen. Erstens wurden in der Studie ausschließlich Visualisierungen bewertet, nicht der Quellcode – dies hielt die Einstiegshürde niedrig, schränkte aber die Perspektive auf die visuelle Darstellung ein. Zweitens wurde die Basiskonfiguration (*Deutsch & R*) auf Grundlage einer Vorstudie festgelegt, bevor RQ-2 alle Konfigurationen systematisch untersuchte; spätere Analysen zeigten, dass *Englisch & Python* gemäß SEVQ und VER besser abschneidet, weshalb die Modellwahl im Kontext der gewählten Basiskonfiguration zu interpretieren ist. Drittens basieren die Prompts auf den 2025 gängigen Designprinzipien; zwischenzeitlich vorgestellte Methoden könnten höhere Konsistenz erreichen. Viertens bewertet der LLM-as-a-Judge ausschließlich den Code, nicht das visuelle Ergebnis – der Einsatz von VLMs zur direkten Bildbeurteilung wäre ein vielversprechender Weiterentwicklungsschritt.

Externe Validität

Die Teilnehmenden verfügten über Grundkenntnisse in Datenanalyse und Visualisierung; eine Generalisierung auf Nutzende ohne Datenhintergrund ist daher nur bedingt möglich. Der Evaluierungs-Datensatz entstammt ausschließlich dem GovData-Portal (deutschsprachige, öffentlich-administrative Tabellen) und wurde ohne domänenspezifische Schichtung ausgewählt – ein domänenstratifiziertes Sampling hätte die thematische Repräsentativität erhöht. Die eingesetzten Sprachmodelle entsprechen dem Versionsstand von Dezember 2025 bis Januar 2026; alle Befunde gelten relativ zu diesem technischen Stand. Schließlich wurden die Experimente in einem kontrollierten Umfeld durchgeführt; Replikationen unter realen Einsatzbedingungen mit Zeitdruck und Nutzerinteraktion sind empfehlenswert.

5. Ergebnisse

Dieses Kapitel präsentiert die empirischen Ergebnisse entlang einer aufeinander aufbauenden Argumentationskette. Zunächst wird die Validität des Evaluationsframeworks selbst geprüft (RQ-1), da alle nachfolgenden Analysen auf den automatisierten SEVQ-Bewertungen basieren. Anschließend werden die Konfigurationseffekte (RQ-2) und die Generalisierbarkeit (RQ-3) des Agenten untersucht, gefolgt vom Einfluss der Metadatenintegration (RQ-4). Abschließend quantifiziert RQ-5 die stochastische Stabilität der Ergebnisse und gibt damit eine Einordnung der in RQ-2 bis RQ-4 beobachteten Effektgrößen.

5.1. RQ-1: Validität der automatisierten Evaluation

Insgesamt wurden 1320 Einzelbewertungen über 264 Visualisierungen erhoben, erzeugt durch fünf LLM-Judges (GPT-5, GPT-4o, Gemini, Grok, Claude Sonnet). 14 Studienteilnehmende bewerteten jeweils 12 Visualisierungen auf denselben sechs SEVQ-Dimensionen.

Inter-Rater-Agreement

Tabelle 8 zeigt das Inter-Rater-Agreement der fünf Judges. Der globale ICC(2,1) (vgl. [Gleichung 3](#)) von 0.120 liegt weit unter der Schwelle für schwache Übereinstimmung, während $\alpha_K = 0.592$ (vgl. [Gleichung 4](#)) die Akzeptanzschwelle von 0.667 verfehlt, aber deutlich über dem Zufallsniveau liegt. Die Diskrepanz ist aufschlussreich: Der ICC bestraft systematische Niveauunterschiede, α_K erfasst die ordinale Konsistenz – die Judges operieren auf unterschiedlichen Skalenniveaus, bewahren aber eine vergleichbare Rangfolge. Konfigurationsspezifisch erreicht *Englisch & Python* mit $\alpha_K = 0.723$ den höchsten Wert.

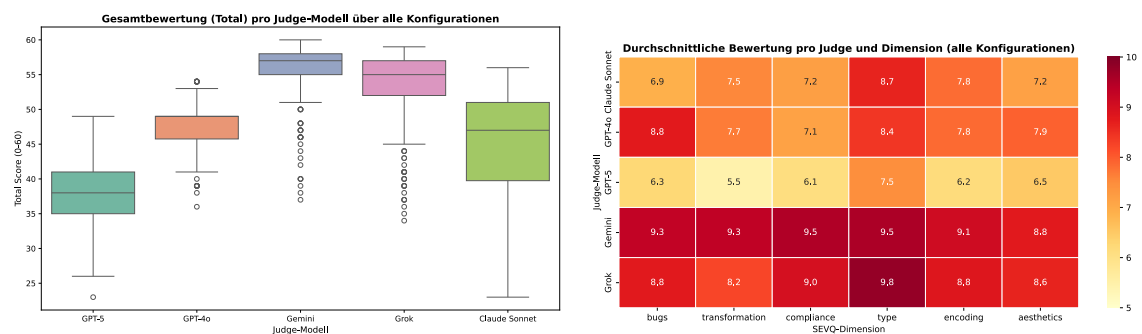
Konfiguration	n	ICC(2,1)	α_K
Global (alle)	264	0.120	0.592
Deutsch & R	63	0.079	0.675
Deutsch & Python	69	0.137	0.630
Englisch & R	69	0.132	0.676
Englisch & Python	63	0.063	0.723

Tabelle 8: Inter-Rater-Agreement der fünf LLM-Judges (SEVQ-Total), aufgeschlüsselt nach globaler und konfigurationspezifischer Betrachtung. ICC(2,1) nach [P. E. Shrout und J. L. Fleiss \[49\]](#), α_K nach [\[51\]](#).

Dieses Muster bestätigt sich im Vergleich mit den menschlichen Ratern: Für die 12 Studienvisualisierungen erreichen die Judges ICC = 0.086 bei $\alpha_K = 0.607$, die Studienteilnehmenden ICC = 0.365 bei $\alpha_K = 0.194$. Die Judges sind ordinal konsistenter, weisen aber größere absolute Niveauunterschiede auf.

Systematische Verzerrungen

Die Ursache der Niveauunterschiede zeigen [Abbildung 10a](#) und [Abbildung 10b](#). Die Mediane des SEVQ-Total-Scores liegen zwischen 38 (GPT-5) und 57 (Gemini) – ein Unterschied von 19 Punkten auf einer 60er-Skala. Aus diesem Grund fällt auch das Inter-Rater-Agreement so gering aus. Gemini zeigt eine inflationäre Tendenz ($M \geq 8.8$ in allen Dimensionen), GPT-5 ist der strengste Judge. Die vollständige Aufschlüsselung nach Dimension und Konfiguration findet sich in [Tabelle 17](#) und [Abbildung 20](#) im Anhang.



(a) Verteilung des SEVQ-Total-Scores pro Judge als Boxplot ($n = 264$). (b) Mittlere SEVQ-Bewertung pro Judge und Dimension als Heatmap, gemittelt über alle Konfigurationen.

Abbildung 10: Systematische Bias-Muster der fünf LLM-Judges ($n = 264$). Links ([Abbildung 10a](#)): Verteilung des SEVQ-Total-Scores pro Judge. Rechts ([Abbildung 10b](#)): Mittlere Bewertung pro Judge und Dimension.

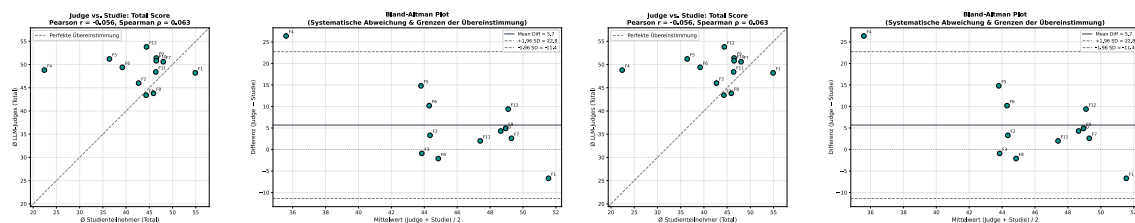
Kriteriumsvalidität

[Tabelle 9](#) stellt den Einzelvergleich für die 12 Studienvisualisierungen dar. Die Judges bewerten bei den Fragen 4–6 (*Deutsch & Python*) systematisch 10 bis 26 Punkte höher als die Studienteilnehmenden, während die Differenzen für die übrigen Konfigurationen im Bereich von ± 7 Punkten liegen.

Frage	Konfig	Dataset	Vis	Ø Judge	Ø Studie	Diff
1	Deutsch & R	139606	0	48.2	54.9	−6.7
2	Deutsch & R	104336	0	46.0	42.7	+3.3
3	Deutsch & R	103779	1	43.4	44.3	−0.9
4	Deutsch & Python	16317	0	48.8	22.4	+26.4
5	Deutsch & Python	108398	1	51.2	36.4	+14.8
6	Deutsch & Python	103779	2	49.4	39.2	+10.2
7	Englisch & R	137336	0	50.6	48.0	+2.6
8	Englisch & R	104336	0	43.8	45.9	−2.1
9	Englisch & R	103779	1	51.4	46.5	+4.9
10	Englisch & Python	13979	0	50.8	46.5	+4.3
11	Englisch & Python	104336	0	48.4	46.4	+2.0
12	Englisch & Python	103779	1	53.8	44.4	+9.4

Tabelle 9: Mittlerer SEVQ-Total-Score im Vergleich: LLM-Judges vs. 14 Studienteilnehmende ($n = 12$), aufgeschlüsselt nach Frage, Konfiguration und Datensatz. Fett markierte Zeilen kennzeichnen Abweichungen von mehr als 10 Punkten. *Diff* = Judge – Studie.

Das Agreement (vgl. Gleichung 5) liegt bei $\tau = 5$ bei 0.583 und erreicht erst bei $\tau = 7$ den Wert 0.667. Spearman $\rho = 0.065$ ($p = 0.838$) bestätigt, dass keine Übereinstimmung in der Rangfolge besteht; der Mean Bias beträgt +5.68. Abbildung 11a und Abbildung 11b visualisieren dies: Die weiten 95%-Übereinstimmungsgrenzen (−11.1 bis +22.8) belegen eine systematische Überbewertung, getrieben durch F4–F6. Deutsch & Python wird von den Judges in der Rangfolge sehr gut eingestuft, die Studienteilnehmenden stufen es jedoch als sehr schlecht ein.



(a) Scatter-Plot der mittleren Judge-Scores gegen die mittleren Studien-Scores ($n = 12$) mit eingezeichneter Winkelhalbierender. (b) Bland-Altman-Plot mit eingezeichnetem Mean Bias und 95%-Übereinstimmungsgrenzen ($n = 12$).

Abbildung 11: Kriteriumsvalidität der LLM-Judges gegenüber den Studienteilnehmenden ($n = 12$). Links (Abbildung 11a): Scatter-Plot der Bewertungspaare. Rechts (Abbildung 11b): Bland-Altman-Plot der Differenzen.

Die Differenzierung nach einzelnen Judges (vgl. Tabelle 19 im Anhang) offenbart ein zentrales Dilemma: Judges mit hoher Rangkorrelation (Gemini: $\rho = 0.523$, Grok: $\rho = 0.413$) weisen den größten absoluten Bias auf, während Judges mit geringem Bias kaum mit der menschlichen Rangfolge korrelieren. Kein einzelner Judge vereint beide Qualitäten – dies motiviert die nachfolgende Ensemble-Analyse.

Ensemble-Strategien

Vier Strategiekategorien wurden untersucht, deren repräsentative Ergebnisse [Tabelle 10](#) zusammenfasst; die vollständige Ergebnistabelle findet sich in [Tabelle 20](#) im Anhang. Das Ziel der Ensemble-Strategie-Untersuchung ist es, Ansätze zu vergleichen, um aus den Judge-Bewertungen eine Bewertung abzuleiten, welche in Inter-Rater-Agreement und Rangkorrelation möglichst ähnlich zu den Studienergebnissen ist.

Bei der **Basis-Aggregation** werden die rohen Judge-Scores direkt aggregiert, etwa als Mittelwert oder Median über alle fünf Judges. Der Mittelwert aller Judges erzielt $MAE = 7.30$ bei $\rho = 0.063$ – eine solide Baseline, die jedoch keine Bias-Korrektur vornimmt und damit die systematischen Niveauunterschiede der Judges (vgl. [Abbildung 10](#)) unbereinigt lässt.

Die **Bias-Korrektur** zieht für jeden Judge dessen modellspezifischen Mittelwertversatz vom Gesamtmittel ab, bevor aggregiert wird. Dadurch bleiben die Rangurteile erhalten, während die absoluten Niveauunterschiede reduziert werden. Gemini nach Bias-Korrektur erreicht $\rho = 0.523$ bei $MAE = 6.53$ – die beste Kombination aus Rangkorrelation und absolutem Fehler unter allen Einzeljudges. Die Mittelung aller fünf bias-korrigierten Judges senkt ρ allerdings auf 0.063, da gegenläufige Rangurteile sich gegenseitig neutralisieren.

Bei der **Z-Score-Normalisierung** wird jeder Judge-Score auf seine modellspezifische Standardabweichung normiert, bevor die Scores aggregiert werden. Das 2er-Subset Gemini+Grok nach Z-Score-Normalisierung erzielt $\rho = 0.526$ bei $MAE = 6.58$ und übertrifft damit die 5er-Mittelung deutlich. Die Auswahl der Judges ist folglich wichtiger als ihre Anzahl.

Die **LOO-Kalibrierung** (Leave-One-Out Cross-Validation) passt für jeden Judge eine lineare Funktion an, die dessen Scores auf den menschlichen Maßstab kalibriert, wobei die Kalibrierparameter jeweils auf den verbleibenden $n - 1$ Beobachtungen geschätzt werden. Diese Strategie minimiert den MAE (6.03) und erzielt das höchste Agreement ($Agr(\tau = 5) = 0.667$), weist jedoch $\rho = -0.392$ auf – ein klassisches Overfitting-Artefakt bei $n = 12$ Kalibrierungsbeobachtungen.

Strategie	Kategorie	Spearman ρ	MAE	Bias	$Agr(\tau = 5)$
Ø Alle Judges (roh)	Basis	0.063	7.30	+5.67	0.583
Median (roh)	Basis	-0.158	8.04	+5.94	0.417
Getrimmtes Mittel (roh)	Basis	0.077	7.96	+6.66	0.417
Gemini (bias-korr.)	Bias-Korr.	0.523	6.53	+5.19	0.583
Ø Bias-korrigiert	Bias-Korr.	0.063	7.30	+5.67	0.583
Gemini+Grok (z-norm.)	Z-Score	0.526	6.58	+1.01	0.500
Ø Z-Score-norm.	Z-Score	0.007	6.96	+1.48	0.417
Ø Kalibriert (LOO)	LOO-Kalib.	-0.392	6.03	-0.31	0.667
GPT-4o (kalibriert LOO)	LOO-Kalib.	0.081	6.15	+0.03	0.667

Tabelle 10: Repräsentative Ensemble-Strategien im Vergleich ($n = 12$). Spearman ρ : Rangkorrelation mit menschlichen Urteilen; MAE: mittlerer absoluter Fehler; Bias: systematische Über- oder Unterbewertung; $Agr(\tau = 5)$: Anteil der Bewertungspaare mit Abweichung ≤ 5 Punkten.

Die zentralen Befunde sind:

1. **Bias-Korrektur bewahrt die Rangkorrelation.** Die Mittelung aller fünf Judges senkt ρ auf 0.063, da gegenläufige Rangurteile sich neutralisieren. Gemini allein nach Bias-Korrektur erzielt $\rho = 0.523$ bei $\text{MAE} = 6.53$.
2. **Kalibrierung optimiert den MAE, nicht die Rangfolge.** LOO-Kalibrierung erreicht den niedrigsten MAE (6.03) und das höchste Agreement, weist aber $\rho = -0.390$ auf – ein Overfitting-Artefakt bei $n = 12$.
3. **Weniger Judges können mehr sein.** Das 2er-Subset Gemini+Grok nach Z-Score-Normalisierung ($\rho = 0.526$, $\text{MAE} = 6.58$) übertrifft die 5er-Mittelung deutlich. Die Auswahl der Judges ist wichtiger als ihre Anzahl.

Abbildung 12 zeigt den vollständigen Vergleich der Strategien.

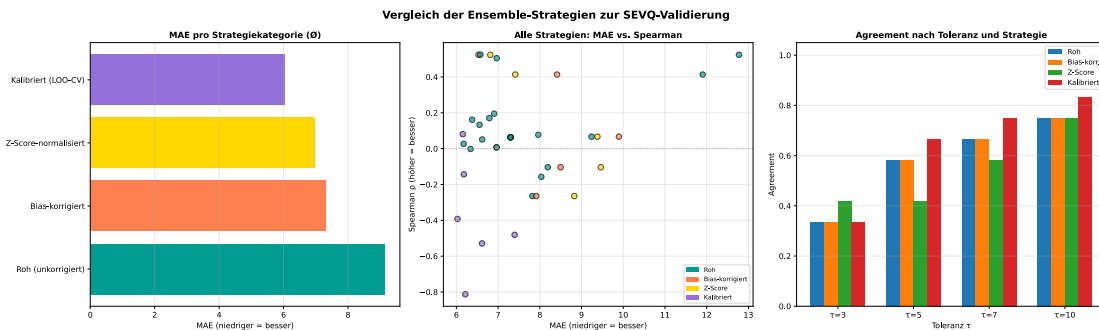


Abbildung 12: Vergleich der Ensemble-Strategien zur SEVQ-Validierung (vollständige Ergebnistabelle in [Tabelle 20](#) im Anhang).

Links: MAE pro Strategiekategorie.
Mitte: MAE vs. Spearman ρ aller untersuchten Strategien.
Rechts: Agreement bei verschiedenen Toleranzwerten τ .

5.2. RQ-2: Konfigurationseffekte

Die Analyse basiert auf 264 Visualisierungen über vier Konfigurationen (Promptsprache $\times$ Programmiersprache) sowie den A/B-Präferenzen der 14 Studienteilnehmenden.

SEVQ-Gesamtbewertung

[Tabelle 11](#) und [Abbildung 13](#) zeigen die SEVQ-Verteilungen. *Deutsch & R* erzielt mit $M = 45.22$ den niedrigsten Wert, die übrigen drei Konfigurationen liegen eng beieinander ($M = 48.35\text{--}49.21$). Der Kruskal-Wallis-Test bestätigt signifikante Unterschiede ($H = 59.73$, $p < 0.001$); paarweise Tests zeigen, dass ausschließlich *Deutsch & R* signifikant abfällt ($p_{\text{adj}} < 0.001$, $|d| > 0.9$), während die drei übrigen Konfigurationen nicht unterscheidbar sind.

Konfiguration	n	M	SD	$\tilde{x}$	CI_{95}
Deutsch & R	63	45.22	3.20	46.00	[44.42; 46.03]
Deutsch & Python	69	48.35	3.50	49.40	[47.51; 49.19]
Englisch & R	69	48.90	3.36	49.60	[48.09; 49.71]
Englisch & Python	63	49.21	2.54	49.40	[48.57; 49.85]

Tabelle 11: SEVQ-Gesamtbewertung (0–60) pro Konfiguration mit Mittelwert, Standardabweichung, Median und 95%-Konfidenzintervall, gemittelt über alle Judges.

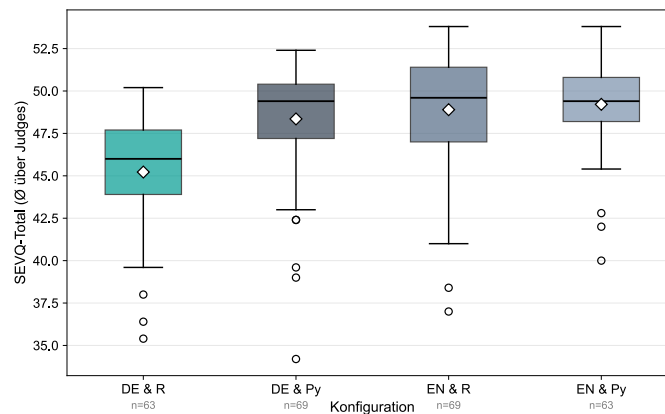


Abbildung 13: SEVQ-Gesamtbewertung (0–60) pro Konfiguration als Boxplot, gemittelt über alle Judges (n je Konfiguration vgl. [Tabelle 11](#)). Paarweise Signifikanztests in [Tabelle 22](#) im Anhang.

Die Dimension *bugs* ist die Hauptursache: *Deutsch & R* erzielt hier nur $M = 6.35$, während die übrigen Konfigurationen $M = 8.40$ – 8.59 erreichen. Die vollständige Dimensionsaufschlüsselung findet sich in [Tabelle 21](#) und [Abbildung 22](#) im Anhang.

VER und Haupteffekte

Die R-Konfigurationen erzielen höhere VER Success Rates (*DE & R*: 65.2%, *EN & R*: 72.5%) als Python (*DE & Py*: 49.3%, *EN & Py*: 48.5%), der Unterschied ist jedoch nicht signifikant ($H = 4.12$, $p = 0.249$).

Isoliert betrachtet zeigt die Promptsprache einen signifikanten SEVQ-Haupteffekt zugunsten englischer Prompts ($d = -0.65$, $p < 0.001$), ebenso die Programmiersprache zugunsten von Python ($d = -0.47$, $p < 0.001$). Beide Effekte werden hauptsächlich durch die niedrigen *bugs*-Werte von *Deutsch & R* getrieben. Für die VER zeigt sich kein signifikanter Spracheffekt ($p = 0.668$); die Programmiersprache zeigt einen marginalen Trend zugunsten von R ($p = 0.055$, $d = 0.50$) – gegenläufig zum SEVQ-Ergebnis. Die VER Success Rates der einzelnen Konfigurationen werden in [Abbildung 14b](#) im mittleren Boxplot abgebildet.

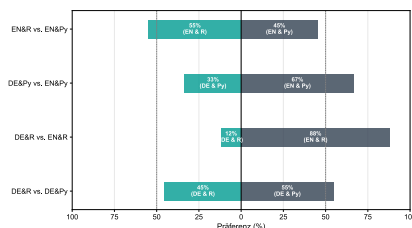
A/B-Vergleiche der Studienteilnehmenden

Die 14 Studienteilnehmenden bewerteten die vier verschiedenen Konfigurationen (vgl. [Tabelle 5](#)) in jeweils drei paarweisen Vergleichen ($3 \times 14 = 42$ Stimmen pro Konfiguration). Die Ergebnisse sind in [Tabelle 12](#) dargestellt und werden in [Abbildung 14a](#) visualisiert.

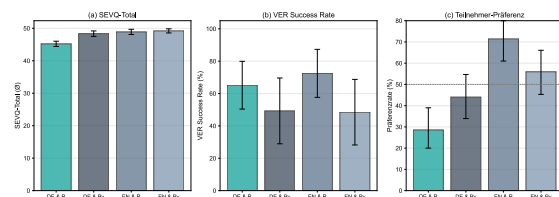
Vergleich	Konfig. 1	n_1	Konfig. 2	n_2	n
DE & R vs. DE & Py	DE & R	19 (45.2%)	DE & Py	23 (54.8%)	42
DE & R vs. EN & R	DE & R	5 (11.9%)	EN & R	37 (88.1%)	42
DE & Py vs. EN & Py	DE & Py	14 (33.3%)	EN & Py	28 (66.7%)	42
EN & R vs. EN & Py	EN & R	23 (54.8%)	EN & Py	19 (45.2%)	42

Tabelle 12: A/B-Präferenzen der 14 Studienteilnehmenden, aufgeschlüsselt nach den vier paarweisen Konfigurationsvergleichen ($n = 42$ Stimmen pro Vergleich). Fett markierte Werte kennzeichnen die jeweils bevorzugte Konfiguration.

Der deutlichste Befund: Bei gleicher Programmiersprache werden die englischsprachigen Konfigurationen klar bevorzugt (88.1% bzw. 66.7%), während der Programmiersprach-Einfluss bei gleicher Promptsprache gering ist (je 55% vs. 45%).



(a) A/B-Präferenzen der Studienteilnehmenden als horizontales Balkendiagramm, aufgeschlüsselt nach den vier Vergleichspaaren.



(b) Zusammenfassung der Konfigurationseffekte über die drei Evaluationsperspektiven SEVQ, VER und Teilnehmerpräferenz.

Abbildung 14: Konfigurationseffekte aus zwei Perspektiven. Links ([Abbildung 14a](#)): A/B-Präferenzen der Studienteilnehmenden. Rechts ([Abbildung 14b](#)): Zusammenfassung über SEVQ, VER und Nutzerpräferenz.

In [Abbildung 14b](#) werden die Ergebnisse der SEVQ-, VER- und Teilnehmer-Präferenz-Auswertung nebeneinander dargestellt. Die Promptsprache Englisch wird über alle drei Metriken hinweg besser als Deutsch eingestuft. Hinsichtlich der Programmiersprachen ist kein klarer Trend erkennbar. Die Programmiersprache ist von der Promptsprache abhängig.

5.3. RQ-3: Generalisierbarkeit

Datensatzabdeckung

In [Abbildung 15](#) wird die Datensatzabdeckung pro Konfiguration abgebildet. Von $N = 50$ Datensätzen konnten 47 geladen werden (drei scheiterten beim Datenabruf von GovData oder CKAN). Je nach Konfiguration erreichten 22–23 die Visualisierungsphase (Dataset Coverage (DC, vgl. [Gleichung 6](#)) ≈ 44 –46%), wobei 24–25 Läufe vorgelagert abbrechen – ein über alle Konfigurationen nahezu konstanter Anteil, der auf strukturelle Datensatzeigenschaften genauer proprietäre und nicht standardisierte Formate zurückzuführen war.

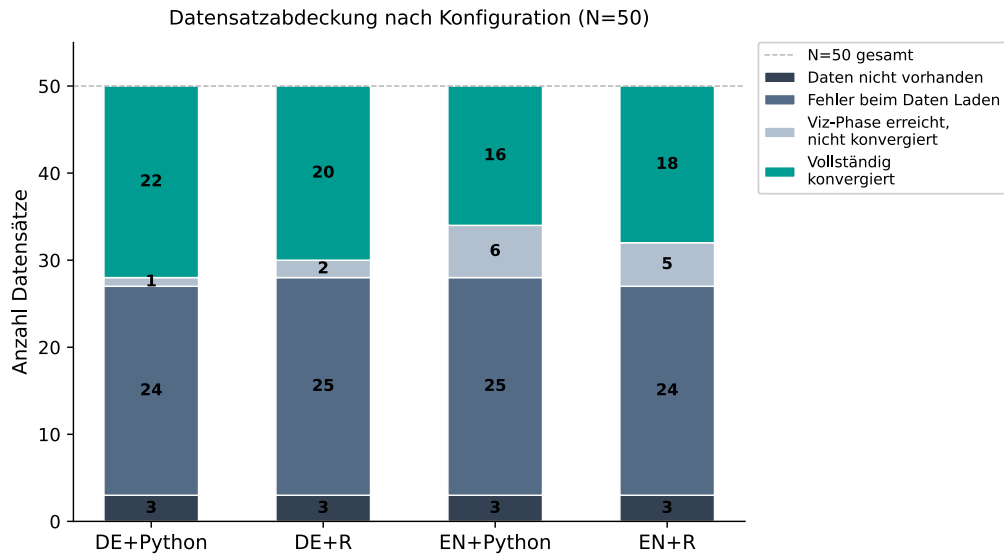


Abbildung 15: Datensatzabdeckung pro Konfiguration ($N = 50$), aufgeschlüsselt nach Verlustursache: Daten fehlen im Evaluierungs-Datensatzes, Daten konnten nicht geladen werden, Visualisierungsphase erreicht aber nicht konvergiert, vollständig konvergiert.

Fehlerrate und Konvergenz

Tabelle 13 fasst die Generalisierbarkeitskennzahlen zusammen. Die Python-Konfigurationen weisen mit $VER_0 \approx 51\%$ höhere Erstversuch-Fehlerraten auf als R (27.5–34.8%). Die Convergence Error Rate (CER, vgl. [Gleichung 7](#)) zeigt ein auffälliges Muster: Die deutschen Konfigurationen konvergieren mit 4.3% (*DE & Py*) und 9.1% (*DE & R*) deutlich zuverlässiger als die englischen (21.7–27.3%). Die mittlere Average Iterations Required (AIR, vgl. [Gleichung 8](#)) liegt zwischen 1.56 und 1.75 Iterationen, was auf einen effizienten Korrekturmechanismus hindeutet.

Konfiguration	DC [%]	VER_0 MW [%]	VER_0 SD [%]	CER [%]	AIR MW	AIR SD
DE & Python	46,0	50,7	47,0	4,3	1,68	0,65
DE & R	44,0	34,8	33,3	9,1	1,75	0,79
EN & Python	44,0	51,5	45,7	27,3	1,56	0,63
EN & R	46,0	27,5	34,3	21,7	1,56	0,62

Tabelle 13: Generalisierbarkeitskennzahlen pro Konfiguration. DC: Dataset Coverage; VER_0 : Fehlerrate im ersten Generierungsversuch (Mittelwert und Standardabweichung); CER: Convergence Error Rate; AIR: Average Iterations Required (Mittelwert und Standardabweichung).

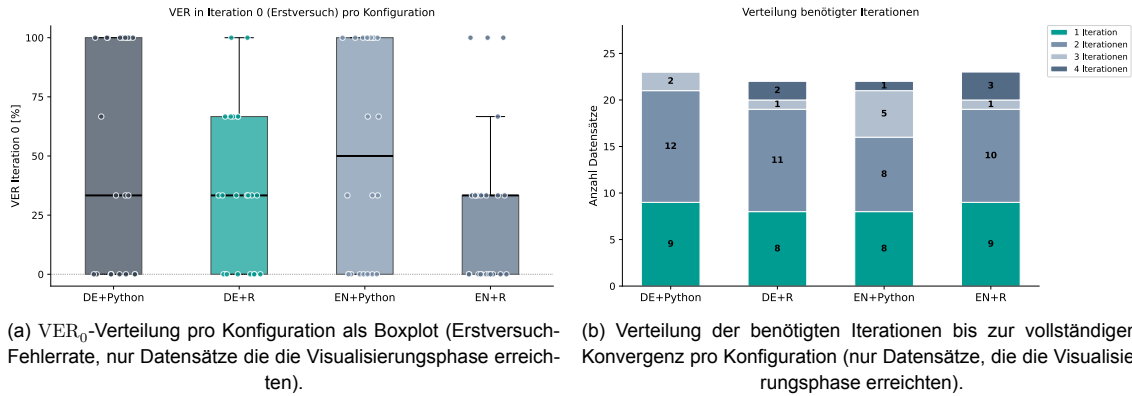


Abbildung 16: Fehllerate und Konvergenzverhalten des Data-Science-Agenten auf dem Evaluierungs-Datensatz. Links (Abbildung 16a): VER_0 -Verteilung pro Konfiguration. Rechts (Abbildung 16b): Anzahl benötigter Iterationen bis zur Konvergenz.

Die Hauptverlustquelle ist nicht die Visualisierungsphase selbst, sondern die vorgelagerte Datenverarbeitung, welche Probleme mit proprietären und nicht standardisierten Dateiformaten hat. Innerhalb der Visualisierungsphase zeigen die R-Konfigurationen niedrigere Erstversuch-Fehlerraten, die deutschen Konfigurationen hingegen höhere Konvergenzraten.

5.4. RQ-4: Einfluss von Metadaten

Die Ablationsstudie vergleicht den Agenten *mit* und *ohne* Metadaten-Schritt auf den Konfigurationen *DE & Python* und *EN & Python*.

SEVQ und VER

In beiden Konfigurationen liegt der mittlere SEVQ mit Metadaten geringfügig höher ($\Delta M_{DE} = 0.28$, $d = 0.18$; $\Delta M_{EN} = 0.69$, $d = 0.32$), keiner der Unterschiede erreicht jedoch das Bonferroni-korrigierte Signifikanzniveau ($\alpha_{\text{kor}} = 0.025$). Die VER zeigt keinen einheitlichen Trend ($\Delta_{DE} = +1.4\%$, $\Delta_{EN} = -4.5\%$; jeweils $p > 0.4$).

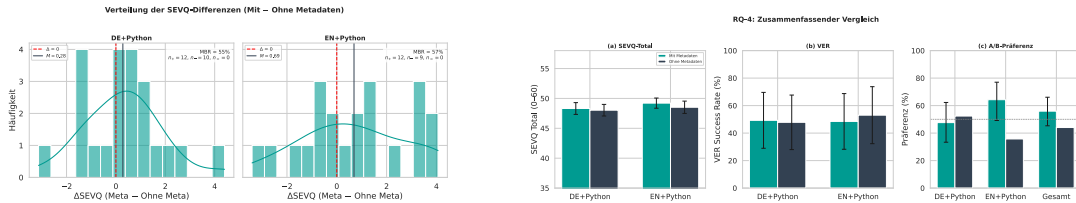
Konfig.	n	M_{meta}	M_{ohne}	ΔM	t	p	d	CI_{95}
DE & Py	22	48.31	48.03	+0.28	0.82	0.419	0.18	$[-0.43; 0.99]$
EN & Py	21	49.21	48.52	+0.69	1.48	0.156	0.32	$[-0.29; 1.67]$

Tabelle 14: SEVQ-Total (0–60) mit und ohne Metadatenintegration sowie gepaarte t-Tests pro Konfiguration. ΔM : Differenz der Mittelwerte (Mit – Ohne), d : Cohens Effektstärke, CI_{95} : 95%-Konfidenzintervall von ΔM .

Die Metadata Benefit Rate (MBR) beträgt 54.5% (*DE & Py*) bzw. 57.1% (*EN & Py*) – Metadaten verbessern die Qualität bei einer leichten Mehrheit der Datensätze, der Effekt ist aber zu klein für statistische Signifikanz. Die Dimensionsanalyse (vgl. Tabelle 24 im Anhang) deutet darauf hin, dass der größte Effekt in *type* für *EN & Python* liegt ($\Delta = +0.27$, $d = 0.37$), was inhaltlich plausibel ist: Die Metadaten liefern Informationen über Spaltentypen, die die Wahl geeigneter Diagrammtypen unterstützen.

A/B-Vergleiche und Zusammenfassung

Die Studienteilnehmenden bevorzugen für *EN & Python* tendenziell die Metadaten-Variante (64.3%, $p = 0.088$), für *DE & Python* ist die Präferenz ausgeglichen (47.6% vs. 52.4%).



(a) Gepaarte SEVQ-Differenzen ($\Delta\text{SEVQ} = \text{Mit} - \text{Ohne Metadaten}$) pro Datensatz und Konfiguration als Histogramm mit eingezeichnetem Mittelwert.

(b) Zusammenfassung des Metadaten-Effekts über die drei Evaluationsperspektiven SEVQ, VER und Teilnehmerpräferenz, aufgeschlüsselt nach Konfiguration.

Abbildung 17: Einfluss der Metadatenintegration auf die Visualisierungsqualität. Links (Abbildung 17a): Gepaarte SEVQ-Differenzen pro Datensatz. Rechts (Abbildung 17b): Zusammenfassung über SEVQ, VER und Teilnehmerpräferenz.

Insgesamt ergibt sich ein schwacher, konsistenter Trend zugunsten der Metadatenintegration, der jedoch in keiner Analyse statistische Signifikanz erreicht.

5.5. RQ-5: Ergebnisstabilität

Die Konfiguration *Englisch & Python* wurde auf fünf Datensätzen je fünfmal ausgeführt (insgesamt 25 Runs).

SEVQ-Stabilität

Der mittlere Variationskoeffizient beträgt $CV = 2.45\%$, die mittlere Standardabweichung $SD = 1.18$ Punkte und die mittlere Spannweite $R = 2.95$ Punkte (Tabelle 15). Der ICC(2,1) (vgl. Gleichung 3) über die fünf Runs beträgt 0.678 (95%-KI: [0.280, 0.950], $p < 0.001$) – moderate Übereinstimmung. Die zwischen-Datensatz-Varianz dominiert gegenüber der Run-Varianz.

Datensatz	M	SD	CV (%)	Min	Max	R
DS 1	51.0	1.02	2.0	49.5	52.1	2.7
DS 2	46.1	1.16	2.5	44.2	47.1	2.9
DS 3	48.8	1.56	3.2	47.1	50.8	3.7
DS 4	47.9	1.68	3.5	46.2	50.3	4.1
DS 5	50.1	0.50	1.0	49.4	50.7	1.3
Mittel	48.8	1.18	2.45			2.95

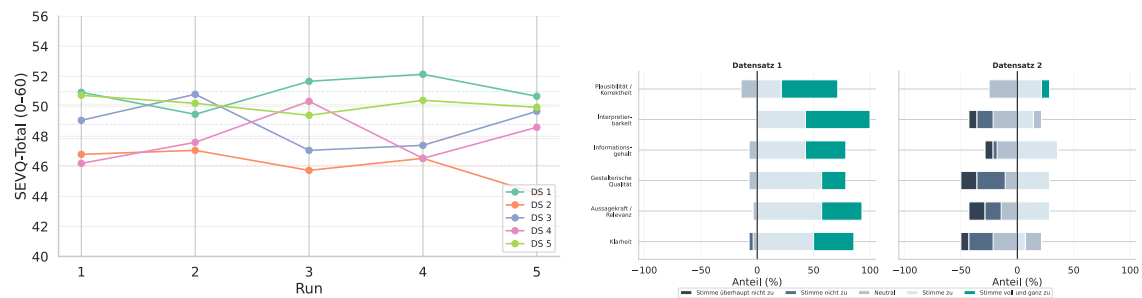
Tabelle 15: SEVQ-Total-Stabilität pro Datensatz über $k = 5$ Runs, angegeben als Mittelwert, Standardabweichung, Variationskoeffizient (CV), Minimum, Maximum und Spannweite (R).

Auf Dimensionsebene weisen *compliance* ($ICC = 0.848$) und *transformation* ($ICC = 0.844$) die höchste Stabilität auf, während *encoding* (0.172) und *aesthetics* (0.285) stärker variieren – gestalterische Aspekte unterliegen größeren Run-zu-Run-Schwankungen als inhaltlich-strukturelle.

VER-Stabilität und subjektive Bewertung

Zwei von fünf Datensätzen erreichen in allen Runs eine VER von 100%; bei den übrigen weicht jeweils ein einzelner Run ab (VER = 66.7%). Die mittlere finale VER liegt bei 96.0% – die Abweichungen betreffen isolierte Runs.

Die Studienteilnehmenden bewerteten die Stabilität auf einer 5-Punkt-Likert-Skala datensatzabhängig unterschiedlich: Datensatz 1 ($M = 4.20$, $SD = 0.79$) wird positiv, Datensatz 2 ($M = 2.77$, $SD = 0.94$) als unzureichend eingestuft. Die Diskrepanz deckt sich mit den objektiven Stabilitätsmaßen (vgl. [Tabelle 28](#) und [Abbildung 29](#) im Anhang).



(a) SEVQ-Total pro Run und Datensatz als Liniendiagramm. Gestrichelte Linien kennzeichnen die datensatzspezifischen Mittelwerte über alle fünf Runs.

(b) Likert-Bewertungen der wahrgenommenen Ergebnisstabilität durch die Studienteilnehmenden (5-Punkt-Skala) für zwei Datensätze, aufgeschlüsselt nach den sechs Bewertungsdimensionen.

Abbildung 18: Ergebnisstabilität des Data-Science-Agenten über $k = 5$ Runs auf fünf Datensätzen (Konfiguration *Englisch & Python*). Links ([Abbildung 18a](#)): SEVQ-Total pro Run und Datensatz. Rechts ([Abbildung 18b](#)): Subjektive Stabilitätsbewertungen der Studienteilnehmenden.

6. Diskussion

Die folgende Diskussion greift die im Ergebniskapitel etablierte Argumentationskette auf: Zunächst wird die Tragfähigkeit des Evaluationsframeworks bewertet (RQ-1), da diese die Interpretierbarkeit aller nachfolgenden Befunde bestimmt. Darauf aufbauend werden die inhaltlichen Ergebnisse zu Konfiguration (RQ-2), Generalisierbarkeit (RQ-3), Metadaten (RQ-4) und Stabilität (RQ-5) eingeordnet und quervernetzt.

6.1. RQ-1: Validität der automatisierten Evaluation

Die Ergebnisse zeigen ein ambivalentes Bild: Moderate ordinale Konsistenz ($\alpha_K = 0.592$) bei schwacher absoluter Übereinstimmung ($ICC = 0.120$) und geringer Kriteriumsvalidität ($\rho = 0.065$).

Inter-Rater-Agreement und Bias-Muster

Die Diskrepanz zwischen ICC und α_K erklärt sich durch die systematischen Niveauunterschiede (Gemini: $\hat{x} = 57$, GPT-5: $\hat{x} = 38$), die den ICC drücken, die Rangübereinstimmung aber weniger beeinträchtigen. Der Vergleich mit den menschlichen Ratern ($ICC = 0.365$, $\alpha_K = 0.194$) ist bemerkenswert.

kenswert: Die LLM-Judges operieren auf divergenten Skalenniveaus, ordnen Visualisierungen aber konsistenter in eine Rangfolge ein als die Studienteilnehmenden.

Die Judge-spezifischen Verzerrungen – Geminis *Leniency Bias* und GPT-5s *Severity Bias* [34] – bestätigen die Notwendigkeit einer Multi-LLM-Ensemble-Strategie H. Li *et al.* [35]. Allerdings übersetzt sich die ordinale Konsistenz unter den Judges nicht in Kriteriumsvalidität, was sich mit der von J. Gu *et al.* [34] beobachteten Modellabhängigkeit deckt.

Der Data-Science-Agent verwendet aktuell Zero-Shot Prompting für die Evaluation, durch einen Few-Shot-Prompting-Ansatz mit klarer Referenzbewertung könnte das Problem der divergenten Skalenniveaus minimiert werden.

Kriteriumsvalidität und Ensemble-Strategien

Die extreme Diskrepanz bei F4–F6 (*Deutsch & Python*: +10 bis +26 Punkte) erklärt sich durch einen fundamentalen Unterschied: LLM-Judges bewerten den generierten Code, nicht die resultierende Visualisierung. Syntaktisch korrekter, aber visuell suboptimaler Code führt zu hohen Judge-Scores bei niedrigen menschlichen Bewertungen. Für die englischsprachigen Konfigurationen (F7–F12) liegen die Differenzen bei ± 5 Punkten – dort korreliert Code-Qualität stärker mit visueller Qualität.

Die Ensemble-Analyse bestätigt, dass Post-hoc-Korrekturen Absolutwerte verbessern, die Rangfolge aber problematisch bleibt. Gemini nach Bias-Korrektur liefert den besten Kompromiss ($\rho = 0.523$, $MAE = 6.53$) L. Zheng *et al.* [32], während LOO-Kalibrierung klassisches Overfitting zeigt ($\rho = -0.390$ bei $n = 12$). Entscheidend: Das Hinzufügen weiterer Judges verschlechtert die Rangkorrelation – das Multi-LLM-Ensemble ist nur vorteilhaft, wenn die Einzeljudges hinreichende Basiskorrelation mit dem Goldstandard aufweisen H. Li *et al.* [35]. Die Ensemble-Strategie-Untersuchung ist auf einer Menge von $n = 12$ menschlichen Referenzbewertungen durchgeführt worden. Diese Menge ist sehr gering, daher sollte die Untersuchung nochmal auf einer größeren Referenzmenge durchgeführt werden, um aussagekräftigere Erkenntnisse ableiten zu können. Die Studienteilnehmenden haben nur die Visualisierung und nicht den Code bewertet, es wäre für zukünftige Untersuchungen sinnvoll zu untersuchen, ob Code gleich wie die entsprechenden Visualisierungen bewertet werden. Falls nicht sollte für die Untersuchung der Ensemble-Strategie eine Referenzmenge von Code-Bewertungen in Betracht gezogen werden.

Implikationen

Die SEVQ eignet sich im Multi-Judge-Setting als Indikator für relative Qualitätsunterschiede zwischen Konfigurationen, nicht aber als Ersatz für menschliche Bewertungen. Visual Language Models (VLMs), die das erzeugte Bild direkt analysieren, könnten die Lücke zwischen Code- und visueller Qualität schließen. Im Vergleich zu textbasierten Evaluationsaufgaben (bis 80% Agreement L. Zheng *et al.* [32]) zeigt die Visualisierungsbewertung deutlich niedrigere Übereinstimmung – plausibel, da sie multimodal ist und LLM-Judges nur die Code-Ebene adressieren.

Diese Einordnung ist für alle folgenden RQs relevant: Die relativen Unterschiede zwischen Konfigurationen sind interpretierbar, da die moderate ordinale Konsistenz alle Gruppen gleichermaßen betrifft; Absolutwerte sind jedoch mit Vorsicht zu behandeln.

6.2. RQ-2: Konfigurationseffekte

Hauptbefund: Promptsprache dominiert

Der stärkste Einzelbefund ist die Unterlegenheit von *Deutsch & R* ($M = 45.22$, $p < 0.001$, $|d| > 0.9$), getrieben durch niedrigere *bugs*-Scores ($M = 6.35$ vs. $M \geq 8.40$). Die A/B-Präferenzen bestätigen dies: 88.1% bevorzugen *Englisch & R* über *Deutsch & R*. LLMs mit englischsprachigen Trainingsdaten erzeugen bei englischen Prompts präzisere Codeergebnisse [53], woraus sich die klare Empfehlung englischer Prompts ergibt.

Die Programmiersprache zeigt hingegen kein einheitliches Muster: Python erzielt höhere SEVQ-Scores ($d = -0.47$), R höhere VER Success Rates ($p = 0.055$, $d = 0.50$). Dies erklärt sich dadurch, dass LLM-Judges Python-Code besser bewerten (dominierende Trainingssprache), während ggplot2 [54] bei Standardvisualisierungen robuster ist. Die ausgeglichenen A/B-Ergebnisse (55% vs. 45%) zeigen, dass Teilnehmende keinen konsistenten visuellen Qualitätsunterschied wahrnehmen.

Alle drei Perspektiven konvergieren: Promptsprache dominiert, Programmiersprache ist untergeordnet. *Englisch & Python* erzielt den höchsten SEVQ bei geringster Streuung; die Unterschiede zu *Englisch & R* sind nicht signifikant ($p_{\text{adj}} = 1.0$).

Limitationen

Die SEVQ-Bewertung basiert auf LLM-Judges mit eingeschränkter Validität (vgl. [Abschnitt 6.1](#)); da die ordinale Konsistenz alle Konfigurationen gleichermaßen betrifft, sind relative Unterschiede interpretierbar. Die A/B-Studie ($n = 14$) begrenzt die statistische Belastbarkeit, die konsistente Richtung über alle Vergleiche verleiht dem Befund jedoch Robustheit.

6.3. RQ-3: Generalisierbarkeit

Datensatzabdeckung und Fehlerrate

Die DC von 44–46% liegt deutlich unter einer wünschenswerten Zielmarke. Die Ausfallanalyse zeigt zwei strukturelle Probleme, welche vor allem auf den Evaluierungs-Datensatz zurückzuführen sind: Drei Datensätze konnten entweder nicht aus dem GovData-Portal oder CKAN vollständig bezogen werden, 24–25 brechen in der Analyse-/Beschreibungsphase ab – konsistent über alle Konfigurationen, was auf proprietäre und nicht standardisierte Formate zurückzuführen ist. Eine Vorvalidierung des Evaluierungs-Datensatzes könnte dies adressieren.

R-Konfigurationen zeigen konsistent niedrigere VER_0 (27.5–34.8% vs. 50.7–51.5%), plausibel durch ggplot2s deklarative API mit weniger Freiheitsgraden bei der Code-Generierung [54]. Die hohe Standardabweichung (33–47 PP) signalisiert, dass die VER_0 primär durch den Datensatz bestimmt wird.

Konvergenz

Die niedrige CER bei *DE & Python* (4.3%) versus *EN & Python* (27.3%) zeigt, dass Promptsprache die Konvergenzrate unabhängig von der Programmiersprache beeinflusst – ein auf den ersten Blick überraschender Befund, der darauf hindeutet, dass der deutschsprachige Systemprompt präzisere Fehlerkorrektur-Anweisungen enthält. Die mittlere AIR von 1.56–1.75 bestätigt die Effizienz des iterativen Korrekturmechanismus.

Die Stichprobe ($N = 50$, Subgruppen $n \approx 22$) und die Beschränkung auf GovData begrenzen die Generalisierbarkeit. VER_0 misst Code-Ausführbarkeit, nicht semantische Qualität – ein Aspekt, den die in RQ-1 diskutierte Limitation der Code-basierten Evaluation widerspiegelt.

6.4. RQ-4: Einfluss von Metadaten

Die Ablationsstudie zeigt einen konsistenten, aber kleinen und nicht signifikanten Qualitätsgewinn ($\Delta SEVQ = +0.48$, $d = 0.18-0.32$), während die VER unbeeinflusst bleibt. Das Fehlen eines VER-Effekts ist erwartungskonform, da Metadaten die Zusammenfassungsphase bereichern, nicht die Code-Korrektheit. Die Dimension *type* profitiert für *EN+Python* am stärksten ($\Delta = +0.27$, $d = 0.37$; vgl. [Tabelle 24](#)), was inhaltlich plausibel ist: Metadaten liefern semantischen Kontext für die Diagrammtypwahl.

Die A/B-Präferenzen bestätigen ein konfigurationsabhängiges Muster: Bei *EN & Python* bevorzugen 64.3% die Metadaten-Variante ($p = 0.088$), bei *DE & Python* ist die Präferenz ausgeglichen. Diese Interaktion lässt sich dadurch erklären, dass englischsprachige LLMs die Metadaten-Zusammenfassung effektiver propagieren.

Die Abwesenheit statistischer Signifikanz schließt einen echten Effekt nicht aus: Bei $d = 0.32$ und $n = 21$ liegt die Power bei lediglich $\approx 25\%$; $n \approx 100$ wäre für 80% Power erforderlich. Die Richtungskonsistenz und der geringe Aufwand (ein zusätzlicher LLM-Aufruf) sprechen dafür, den Metadaten-Schritt als Standardkonfiguration beizubehalten.

6.5. RQ-5: Ergebnisstabilität

Metrische Stabilität

Der mittlere CV von 2.45% und die SD von 1.18 Punkten ($< 2\%$ des Skalenbereichs) zeigen hohe Stabilität. Der ICC(2,1) von 0.678 (moderate Übereinstimmung nach [T. K. Koo und M. Y. Li \[50\]](#)) bestätigt, dass die Variation zwischen Datensätzen die Run-Variation dominiert. Die Dimensionsanalyse (vgl. [Abbildung 28](#)) differenziert: *Compliance* (ICC = 0.848) und *transformation* (0.844) sind hochstabil, *encoding* (0.172) und *aesthetics* (0.285) variieren stärker – plausibel, da gestalterische Entscheidungen stärker von der Sampling-Temperatur beeinflusst werden.

Die VER erreicht eine mittlere finale Success Rate von 96.0%, vereinzelte Abweichungen betreffen isolierte Runs und sind durch den stochastischen Charakter der Codegenerierung erklärbar.

Subjektive Stabilitätswahrnehmung

Die Likert-Bewertungen zeigen einen markanten Datensatz-Unterschied: Datensatz 1 ($M = 4.20$) wird positiv bewertet, Datensatz 2 ($M = 2.77$) als unzureichend. Dieser scheinbare Widerspruch zu den stabilen automatisierten Metriken erklärt sich durch die unterschiedlichen Bewertungsebenen: Die SEVQ bewertet die technische Codequalität (stabil), Teilnehmende die visuelle Erscheinung. Der Agent kann bei wiederholter Ausführung strukturell unterschiedliche Visualisierungstypen wählen, die den SEVQ-Score kaum beeinflussen, aber als inkonsistent wahrgenommen werden – ein Phänomen, das die in RQ-1 identifizierte Diskrepanz zwischen Code- und visueller Qualität widerspiegelt.

Implikationen

Der Agent ist für einmalige Anwendung hinreichend stabil (geringe metrische Variabilität, hohe VER). Die Reproduzierbarkeit der visuellen Ergebnisse ist jedoch datensatzabhängig: Bei Datensätzen mit mehreren plausiblen Visualisierungsstrategien kann die wahrgenommene Konsistenz sinken. Diese Befunde geben den in RQ-2 bis RQ-4 beobachteten Effektgrößen ($\Delta\text{SEVQ} = 0.28 - 3.99$) eine untere Schranke der Interpretierbarkeit: Da die Run-zu-Run-Variation bei $SD \approx 1.2$ Punkten liegt, sind Konfigurationseffekte ab $\Delta \approx 2-3$ Punkten als substantiell einzustufen.

Die Aussagekraft unterliegt der kleinen Stichprobe ($k = 5$ Runs, $n = 5$ Datensätze, zwei subjektiv bewertet); eine Erweiterung wäre für belastbarere Aussagen notwendig.

7. Schlussfolgerung

Die vorliegende Arbeit hatte zum Ziel, ein LLM-basiertes Multi-Agent-System zur autonomen Exploration tabellarischer Datensätze zu konzipieren, zu implementieren und empirisch zu evaluieren. Der entwickelte Data-Science-Agent erweitert die konzeptionellen Grundlagen von LIDA in den Dimensionen Architektur, Evaluationsstrategie und Konfigurierbarkeit und wurde auf realen Open-Data-Datensätzen des GovData-Portals untersucht.

Die empirischen Ergebnisse zeigen, dass der Agent auf standardisierten Datensätzen konsistent gute Visualisierungen erzeugt. Fälle, in denen die Pipeline vorzeitig abbrach, ließen sich nahezu ausschließlich auf proprietäre oder nicht-standardkonforme Dateiformate zurückführen - ein strukturelles Problem der Datenqualität im GovData-Portal, nicht ein Versagen des Agenten selbst. Innerhalb der erreichten Visualisierungsphase waren die Ergebnisse stabil ($CV = 2,45\%$, mittlere finale VER = 96,0 %) und über wiederholte Ausführungen hinweg reproduzierbar.

Hinsichtlich der Konfiguration hat sich die Promptsprache als dominierender Einflussfaktor herausgestellt: Englischsprachige Prompts erzielten signifikant bessere Ergebnisse, was auf die englischsprachige Trainingsdatenbasis moderner LLMs zurückzuführen ist. Die Wahl der Programmiersprache ist demgegenüber nachrangig - Python erzielt höhere SEVQ-Scores, R niedrigere Erstversuch-Fehlerraten, ohne dass Studienteilnehmende einen konsistenten visuellen Qualitätsunterschied wahrnahmen. Die Metadatenintegration zeigt einen schwachen, konsisten-

ten, aber nicht signifikanten Qualitätsgewinn; angesichts des geringen zusätzlichen Aufwands empfiehlt sich ihre Beibehaltung als Standardkonfiguration.

Die automatisierte SEVQ-Bewertung durch LLM-Judges eignet sich als Indikator für relative Qualitätsunterschiede zwischen Konfigurationen, nicht jedoch als vollwertiger Ersatz menschlicher Bewertung. Die moderate ordinale Konsistenz bei geringer Kriteriumsvalidität ist darauf zurückzuführen, dass LLM-Judges ausschließlich den generierten Code bewerten, nicht das visuelle Ergebnis.

Aus den Ergebnissen ergeben sich mehrere vielversprechende Richtungen für zukünftige Arbeiten. Der offensichtlichste Ansatzpunkt ist der Einsatz von Visual Language Models (VLMs) zur direkten Bildbeurteilung: Statt nur den Visualisierungscode zu bewerten, könnten VLMs zusätzlich die erzeugte Grafik selbst analysieren und damit die identifizierte Lücke zwischen Code- und visueller Qualität schließen. Dies würde auch semantische Fehler erfassen, die der aktuelle VER-Mechanismus nicht erkennt.

Im Bereich der Metadatenverarbeitung besteht weiteres Optimierungspotenzial: Eine verbesserte Extraktionslogik, die relevante Metadatenfelder gezielter selektiert und aufbereitet, könnte den bislang kleinen Effekt verstärken. Besonders aufschlussreich wäre zudem eine gezielte Evaluation auf GovData-Datensätzen mit nachweislich hochwertigen Metadaten, um den tatsächlichen Mehrwert semantisch reichhaltiger Kontextinformationen isoliert zu untersuchen.

Die LLM-Konfiguration des Agenten wurde in dieser Arbeit nur auf einer kleinen Stichprobe ($n = 10$) von Datensätzen und unter der Basiskonfiguration Deutsch & R vorselektiert. Eine systematische Evaluation über alle Konfigurationskombinationen und eine größere Datenbasis ist notwendig, um die optimale Modellwahl belastbar zu begründen. Darüber hinaus bietet weiteres Prompt Engineering - etwa durch Few-Shot-Beispiele aus einer kuratierten Evaluationsdatenbank erfolgreicher Visualisierungen - ein konkretes Mittel zur Qualitätssteigerung, das innerhalb des bestehenden Systemrahmens umsetzbar ist.

Schließlich ist während der Entstehung dieser Arbeit ein breiterer Trend zu beobachten gewesen: Deep-Agent-Systeme und noch autonomere Multi-Agent-Architekturen sind zunehmend in den Fokus der Forschung gerückt. Für den Data-Science-Agenten ließe sich dieser Ansatz weiterdenken - etwa durch einen vorgelagerten Agenten, der eigenständig die Datensätze lädt und dadurch auch mit nicht-standardkonformen Datensätzen umgehen kann. Ein solches System würde den manuellen Einstiegspunkt weiter reduzieren und die ursprüngliche Vision einer vollautomatischen Vorschaufunktion für Open-Data-Plattformen konsequent weiterführen.

Literatur

- [1] Europäisches Parlament und Rat, „Richtlinie (EU) 2019/1024“. Zugegriffen: 12. März 2026. [Online]. Verfügbar unter: <https://eur-lex.europa.eu/legal-content/DE/TXT/PDF/?uri=CELEX:32019L1024>
- [2] Bundesregierung, „Open Data Strategie der Bundesregierung“. Zugegriffen: 12. März 2026. [Online]. Verfügbar unter: <https://www.bundesregierung.de/resource/blob/992814/1940386/1d269a2ad1b6346fcf60663bdea9c9f8/2021-07-07-open-data-strategie-data.pdf>
- [3] M. Wooldridge und N. R. Jennings, „Intelligent agents: Theory and practice“, *The knowledge engineering review*, Bd. 10, Nr. 2, S. 115–152, 1995.
- [4] X. Huang u. a., „Understanding the planning of llm agents: A survey“, *arXiv preprint arXiv:2402.02716*, 2024.
- [5] D. B. Acharya, K. Kuppan, und B. Divya, „Agentic AI: Autonomous intelligence for complex goals—A comprehensive survey“, *IEEE Access*, Bd. 13, S. 18912–18936, 2025.
- [6] T. Guo u. a., „Large language model based multi-agents: A survey of progress and challenges“, *arXiv preprint arXiv:2402.01680*, 2024.
- [7] V. Dibia, „LIDA: A tool for automatic generation of grammar-agnostic visualizations and infographics using large language models“, in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations)*, 2023, S. 113–126.
- [8] A. Vaswani u. a., „Attention is all you need“, *Advances in neural information processing systems*, Bd. 30, 2017.
- [9] L. Ouyang u. a., „Training language models to follow instructions with human feedback“, *Advances in neural information processing systems*, Bd. 35, S. 27730–27744, 2022.
- [10] T. Schick u. a., „Toolformer: Language models can teach themselves to use tools“, *Advances in neural information processing systems*, Bd. 36, S. 68539–68551, 2023.
- [11] T. Brown u. a., „Language models are few-shot learners“, *Advances in neural information processing systems*, Bd. 33, S. 1877–1901, 2020.
- [12] A. Holtzman, J. Buys, L. Du, M. Forbes, und Y. Choi, „The curious case of neural text degeneration“, *arXiv preprint arXiv:1904.09751*, 2019.
- [13] N. Paulsen, „Context Is What You Need: The Maximum Effective Context Window for Real World Limits of LLMs“, *arXiv preprint arXiv:2509.21361*, 2025.
- [14] U. Nisa, M. Shirazi, M. A. Saip, und M. S. M. Pozi, „Agentic AI: The age of reasoning—A review“, *Journal of Automation and Intelligence*, 2025.
- [15] Z. Milosevic und F. Rabhi, „Architecting Agentic Communities using Design Patterns“, *arXiv preprint arXiv:2601.03624*, 2026.
- [16] M. Fowler, *Analysis patterns: reusable object models*. Addison-Wesley Professional, 1997.

- [17] M. Fowler, *Patterns of enterprise application architecture*. Addison-Wesley, 2012.
- [18] A. Singh, A. Ehtesham, S. Kumar, und T. T. Khoei, „Enhancing AI systems with agentic workflows patterns in large language model“, in *2024 IEEE World AI IoT Congress (AlloT)*, 2024, S. 527–532.
- [19] A. Madaan u. a., „Self-refine: Iterative refinement with self-feedback“, *Advances in neural information processing systems*, Bd. 36, S. 46534–46594, 2023.
- [20] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, und S. Yao, „Reflexion: Language agents with verbal reinforcement learning“, *Advances in neural information processing systems*, Bd. 36, S. 8634–8652, 2023.
- [21] Z. Gou u. a., „Critic: Large language models can self-correct with tool-interactive critiquing“, *arXiv preprint arXiv:2305.11738*, 2023.
- [22] Anthropic, „Introducing the model context protocol“. Zugegriffen: 1. April 2026. [Online]. Verfügbar unter: <https://www.anthropic.com/news/model-context-protocol>
- [23] S. Yao u. a., „React: Synergizing reasoning and acting in language models“, in *The eleventh international conference on learning representations*, 2022.
- [24] J. Wei u. a., „Chain-of-thought prompting elicits reasoning in large language models“, *Advances in neural information processing systems*, Bd. 35, S. 24824–24837, 2022.
- [25] Q. Wu u. a., „Autogen: Enabling next-gen LLM applications via multi-agent conversations“, in *First conference on language modeling*, 2024.
- [26] A. Fournay u. a., „Magentic-one: A generalist multi-agent system for solving complex tasks“, *arXiv preprint arXiv:2411.04468*, 2024.
- [27] A. Gatt und E. Krahmer, „Survey of the state of the art in natural language generation: Core tasks, applications and evaluation“, *Journal of Artificial Intelligence Research*, Bd. 61, S. 65–170, 2018.
- [28] K. Papineni, S. Roukos, T. Ward, und W.-J. Zhu, „Bleu: a method for automatic evaluation of machine translation“, in *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, 2002, S. 311–318.
- [29] C.-Y. Lin, „Rouge: A package for automatic evaluation of summaries“, in *Text summarization branches out*, 2004, S. 74–81.
- [30] S. Banerjee und A. Lavie, „METEOR: An automatic metric for MT evaluation with improved correlation with human judgments“, in *Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization*, 2005, S. 65–72.
- [31] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, und C. Zhu, „G-eval: NLG evaluation using gpt-4 with better human alignment“, in *Proceedings of the 2023 conference on empirical methods in natural language processing*, 2023, S. 2511–2522.
- [32] L. Zheng u. a., „Judging llm-as-a-judge with mt-bench and chatbot arena“, *Advances in neural information processing systems*, Bd. 36, S. 46595–46623, 2023.

- [33] S. Kadavath u. a., „Language models (mostly) know what they know“, *arXiv preprint arXiv:2207.05221*, 2022.
- [34] J. Gu u. a., „A survey on llm-as-a-judge“, *The Innovation*, 2024.
- [35] H. Li u. a., „Llms-as-judges: a comprehensive survey on llm-based evaluation methods“, *arXiv preprint arXiv:2412.05579*, 2024.
- [36] A. Satyanarayan, D. Moritz, K. Wongsuphasawat, und J. Heer, „Vega-lite: A grammar of interactive graphics“, *IEEE transactions on visualization and computer graphics*, Bd. 23, Nr. 1, S. 341–350, 2016.
- [37] J. VanderPlas u. a., „Altair: Interactive statistical visualizations for python“, *Journal of open source software*, Bd. 3, Nr. 32, S. 1057, 2018.
- [38] B. Saket, D. Moritz, H. Lin, V. Dibia, C. Demiralp, und J. Heer, „Beyond heuristics: Learning visualization design“, *arXiv preprint arXiv:1807.06641*, 2018.
- [39] V. Dibia und Ç. Demiralp, „Data2vis: Automatic generation of data visualizations using sequence-to-sequence recurrent neural networks“, *IEEE computer graphics and applications*, Bd. 39, Nr. 5, S. 33–46, 2019.
- [40] Y. Tian u. a., „Chartgpt: Leveraging llms to generate charts from abstract natural language“, *IEEE Transactions on Visualization and Computer Graphics*, Bd. 31, Nr. 3, S. 1731–1745, 2024.
- [41] A. Narechania, A. Srinivasan, und J. Stasko, „NL4DV: A toolkit for generating analytic specifications for data visualization from natural language queries“, *IEEE Transactions on Visualization and Computer Graphics*, Bd. 27, Nr. 2, S. 369–379, 2020.
- [42] K. Wongsuphasawat, D. Moritz, A. Anand, J. Mackinlay, B. Howe, und J. Heer, „Voyager: Exploratory analysis via faceted browsing of visualization recommendations“, *IEEE transactions on visualization and computer graphics*, Bd. 22, Nr. 1, S. 649–658, 2015.
- [43] K. Wongsuphasawat u. a., „Voyager 2: Augmenting visual analysis with partial view specifications“, in *Proceedings of the 2017 chi conference on human factors in computing systems*, 2017, S. 2648–2659.
- [44] Open Knowledge Foundation und Beitragende, „CKAN“. Zugegriffen: 27. März 2026. [Online]. Verfügbar unter: <https://docs.ckan.org/en/2.11/user-guide.html>
- [45] A. Joshi, S. Kale, S. Chandel, und D. K. Pal, „Likert scale: Explored and explained“, *British journal of applied science & technology*, Bd. 7, Nr. 4, S. 396–403, 2015.
- [46] Student, „The probable error of a mean“, *Biometrika*, S. 1–25, 1908.
- [47] E. W. Weisstein, „Bonferroni correction“, <https://mathworld.wolfram.com/>, 2004.
- [48] J. Cohen, „The statistical power of abnormal-social psychological research: a review.“, *The Journal of Abnormal and Social Psychology*, Bd. 65, Nr. 3, S. 145, 1962.
- [49] P. E. Shrout und J. L. Fleiss, „Intraclass correlations: uses in assessing rater reliability.“, *Psychological bulletin*, Bd. 86, Nr. 2, S. 420, 1979.

- [50] T. K. Koo und M. Y. Li, „A guideline of selecting and reporting intraclass correlation coefficients for reliability research“, *Journal of chiropractic medicine*, Bd. 15, Nr. 2, S. 155–163, 2016.
- [51] K. Krippendorff, „Computing Krippendorff's alpha-reliability“, 2011.
- [52] J. M. Bland und D. Altman, „Statistical methods for assessing agreement between two methods of clinical measurement“, *The lancet*, Bd. 327, Nr. 8476, S. 307–310, 1986.
- [53] L. Qin u. a., „Multilingual large language model: A survey of resources, taxonomy and frontiers“, *arXiv preprint arXiv:2404.04925*, 2024.
- [54] R. A. M. Villanueva und Z. J. Chen, „ggplot2: elegant graphics for data analysis“. Taylor & Francis, 2019.

Anhang A: Data-Science-Agent

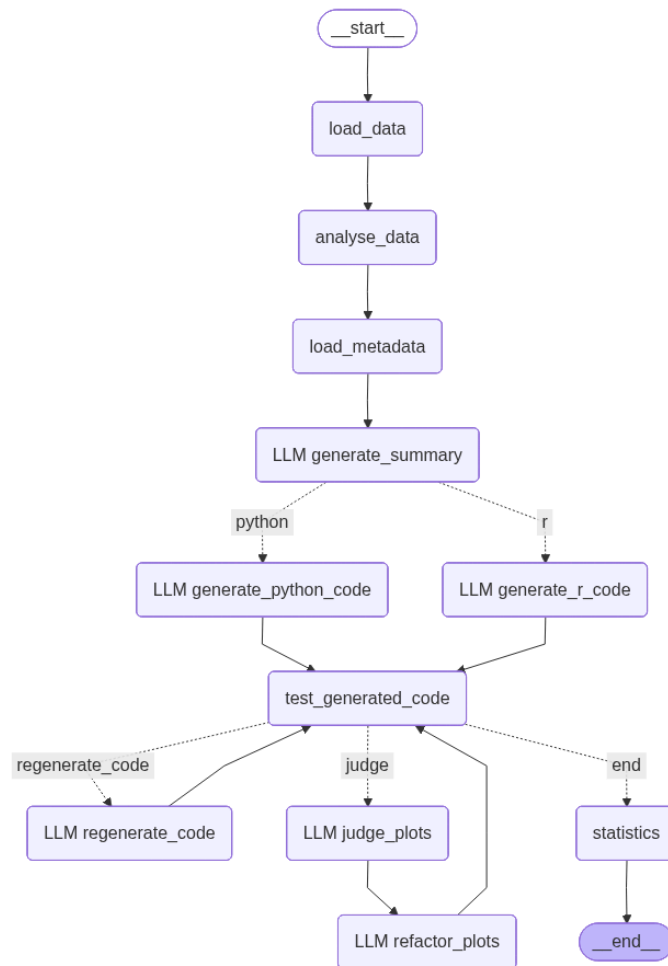


Abbildung 19: Orchestrierungsgraph des Data-Science-Agenten in LangGraph. Knoten repräsentieren Verarbeitungsschritte der einzelnen Module; Knoten mit Präfix *LLM* verwenden ein Sprachmodell. Kanten zeigen den Kontrollfluss einschließlich bedingter Verzweigungen. Der vollständige Quellcode ist dem GitHub-Repository zu entnehmen: github.com/Elisper01/data-science-agent.

```

summary = (
    'Dieser Datensatz dokumentiert die Tourismusentwicklung der Gemeinde '
    'Rettenberg ueber einen Zeitraum von 11 Jahren (2014-2024). Er basiert '
    'auf der amtlichen Monatserhebung im Tourismus und differenziert zwischen '
    'Gaesteankunften und Gaesteuebernachtungen, jeweils unterteilt nach der '
    'Herkunft der Gaeste (Inland vs. Ausland). Mit insgesamt 66 Datenpunkten '
    'bietet er eine solide Basis fuer Zeitreihenanalysen. Auffaellig ist die '
    'starke Dominanz des Inlandstourismus und signifikante Schwankungen in den '
    'Pandemiejahren (2020/2021). Der Maximalwert von 102.156 Uebernachtungen '
    'im Jahr 2023 deutet auf eine starke Erholung des Sektors hin.'
)

columns = [
    Column(
        name='statistics_code',

```

```

        unit='Code',
        description='Interner statistischer Schluessel der Erhebung.',
        missing_values=0, unique_values=1, most_frequent_value=45511,
    ),
    Column(
        name='statistics_label',
        unit='Text',
        description='Bezeichnung der statistischen Erhebung.',
        missing_values=0, unique_values=1,
        most_frequent_value='Monatserhebung im Tourismus',
    ),
    Column(
        name='time_code',
        unit='Code',
        description='Zeitliche Granularitaet der Daten (hier: Jaehrlich).',
        missing_values=0, unique_values=1, most_frequent_value='JAHR',
    ),
    Column(
        name='time',
        unit='Jahr (JJJJ)',
        description='Berichtsjahr der Datenerhebung (Zeitraum 2014 bis 2024).',
        missing_values=0, unique_values=11, most_frequent_value=2019,
    ),
    Column(
        name='1_variable_attribute_code',
        unit='AGS-Code',
        description='Amtlicher Gemeindeschluessel (AGS) fuer Rettenberg.',
        missing_values=0, unique_values=1, most_frequent_value=9780137,
    ),
    Column(
        name='1_variable_attribute_label',
        unit='Text',
        description='Name der untersuchten Gemeinde.',
        missing_values=0, unique_values=1, most_frequent_value='Rettenberg',
    ),
    Column(
        name='2_variable_attribute_label',
        unit='Kategorie',
        description='Klassifizierung der Gaeste nach ihrer Herkunft.',
        missing_values=0, unique_values=3, most_frequent_value='Inland',
    ),
    Column(
        name='value',
        unit='Anzahl (Gaeste/Naechte)',
        description='Der numerische Messwert der jeweiligen Kennzahl.',
        missing_values=0, unique_values=66, most_frequent_value=None,
    ),
    Column(
        name='value_variable_label',
        unit='Indikator',
        description='Unterscheidung zwischen Gaesteankunefte und Uebernachtungen.',
        missing_values=0, unique_values=2,
        most_frequent_value='Gaesteankunefte in Beherbergungsbetrieben',
    ),
]

```

Code-Fragment 1: Beispielausgabe des Summarizer-Moduls für den Tourismus-Datensatz der Gemeinde Rettenberg (Datensatz-ID: 15653). Das Objekt enthält eine natürlichsprachliche Zusammenfassung des Datensatzes sowie eine strukturierte Beschreibung aller Spalten mit Datentyp, Einheit, fehlenden Werten und häufigsten Werten. Diese Ausgabe dient als Kontext für alle nachfolgenden Module des Data-Science-Agenten.

```

question='Wie haben sich die Gästeübernachtungen und Gästeankünfte im Zeitverlauf von 2014 bis 2024 entwickelt?'
visualization="Liniendiagramm mit 'time' auf der x-Achse, 'value' auf der y-Achse und einer

```

Farbtrennung nach 'value_variable_label'."

rationale="Ein Liniendiagramm ist ideal, um Trends über 11 Jahre hinweg darzustellen. Durch die Verwendung von 'time' auf der x-Achse und 'value' auf der y-Achse, gefiltert nach den verschiedenen Indikatoren in 'value_variable_label', kann der Nutzer sofort das Wachstum, den Einbruch während der Pandemiejahre und die anschließende Erholung des Tourismus in Rettenberg erkennen."

Code-Fragment 2: Beispielausgabe des Goal-Explorer-Moduls für den Tourismus-Datensatz der Gemeinde Rettenberg (Datensatz-ID: 15653). Das Visualisierungsziel besteht aus einer analytischen Fragestellung, einem Vorschlag für den Visualisierungstyp und einer Begründung, warum diese Fragestellung für den Datensatz relevant ist.

Dimension	LIDA	Data-Science-Agent
Architektur	Implizites MAS; prozedurales Python-Skript als Orchestrierung	Explizites MAS; LangGraph mit Shared State und layered-Topologie (vgl. Abschnitt 2.3)
Framework	Eigenständige Python-Implementierung	LangChain + LangGraph (vgl. Abschnitt 3.1)
Module	Summarizer, Goal Explorer, VizGenerator, Infographer (vgl. Abbildung 3)	Summarizer, Goal Explorer, Viz Generator, Statistics (vgl. Abbildung 5)
Metadaten	Nicht unterstützt	RDF-Metadaten als zusätzlicher Input
Codegenerierung	n Kandidaten parallel; Self-Consistency-Selektion	Iterative Reparatur mit max. k Versuchen; Reflexion-Muster
Skriptsprachen	Nur Python	Python und R konfigurierbar
Visualisierungsbibliotheken	Matplotlib, Seaborn	Abhängig von Skriptsprache; flexibel durch LLM-Codegenerierung
SEVQ	1 LLM-Judge; einmalig am Ende	5 LLM-Judges; Multi-LLM-Evaluation (vgl. Abschnitt 2.4)
VER	Einmalig am Ende; misst Code-Kompilierbarkeit	Pro Iteration; misst tatsächliche Visualisierungserzeugung (vgl. Abschnitt 3.2)
Protokollierung	Nicht vorgesehen	Token-Verbrauch, Laufzeit, Konfiguration, Zwischenergebnisse
Konfigurierbarkeit	Fest konfiguriert	LLM-Modell, Promptsprache, Skriptsprache, n , k (vgl. Abschnitt 3.1)
Promptsprache	Nur Englisch	Deutsch und Englisch; dynamische DTO-Auswahl
Infografiken	Ja (Infographer mit IGM)	Nein; Fokus auf analytische Visualisierungen

Tabelle 16: Gegenüberstellung von LIDA [7] und dem Data-Science-Agenten hinsichtlich Architektur, Modulstruktur, Evaluation und Konfigurierbarkeit.

```

system_prompt = """
Du bist ein kritischer, erfahrener Experte für Datenvisualisierung
und bewertest Code, der Visualisierungen erstellt. Deine Aufgabe
ist es, den Code objektiv, streng und differenziert danach zu
bewerten, wie gut er das beschriebene Ziel der Visualisierung
erfüllt.

Gehe besonders kritisch vor – vermeide zu wohlwollende
Beurteilungen. Beziehe dich auf nachvollziehbare Standards der

```

Datenvisualisierung und logisch begründete Urteile. Wenn ein Aspekt unklar, fehlerhaft oder suboptimal ist, bewerte ihn deutlich unter 8 und erkläre dies präzise.

Verwende die folgenden sechs LIDA-Bewertungsdimensionen:

1. *bugs* – Enthält der Code syntaktische oder logische Fehler, die die Ausführung verhindern oder das Ergebnis verfälschen? Falls der Code nicht lauffähig ist oder falsche Ergebnisse produziert, bewerte zwingend unter 5.
2. *transformation* – Ist die Datenvorbereitung (Filtern, Aggregieren, Gruppieren etc.) korrekt, logisch und zweckmäßig für das Ziel? Eine nur teilweise sinnvolle Transformation darf maximal eine mittlere Bewertung erhalten.
3. *compliance* – Wie gut erfüllt der Code das formulierte Visualisierungsziel und zeigt die intendierten Zusammenhänge? Wenn das Ziel nur teilweise erreicht wird, sollte der Wert im Bereich 4–7 liegen.
4. *type* – Passt der gewählte Visualisierungstyp (z.B. Balken-, Linien-, Streudiagramm) wirklich zu den Daten und zur Fragestellung? Wenn der Typ missverständlich, unpassend oder unklar ist, bewerte unter 6.
5. *encoding* – Werden Variablen korrekt und effektiv über visuelle Kanäle (Achsen, Farben, Formen, Größen) dargestellt? Fehlerhafte oder inkonsistente Achsenbeschriftungen, Farben oder Skalierungen führen zu einer niedrigen Bewertung (<6).
6. *aesthetics* – Ist das Erscheinungsbild (Layout, Farben, Lesbarkeit, Proportionen) ansprechend, klar und professionell? Schlechtes Layout, übermäßiger Text oder unklare Farben mindern die Punktzahl.

Gib für jede Dimension eine Punktzahl von 1 (sehr schlecht) bis 10 (ausgezeichnet) und eine präzise, nachvollziehbare Begründung, warum du genau diesen Wert gewählt hast. Eine kurze Gesamteinschätzung am Ende (1–3 Sätze) ist erwünscht, aber keine Gesamtnote nötig.

"""

Code-Fragment 3: System-Prompt des Evaluator-Moduls. Der Prompt instruiert ein Sprachmodell als kritischen Bewertungsexperten für Datenvisualisierungen anhand der sechs LIDA-Bewertungsdimensionen: *bugs*, *transformation*, *compliance*, *type*, *encoding* und *aesthetics*. Jede Dimension wird auf einer Skala von 1 bis 10 bewertet.

```
user_prompt = """
Bewerte den folgenden Python-Visualisierungscode in Hinblick auf
das angegebene Ziel. Verwende die sechs LIDA-Dimensionen (bugs,
transformation, compliance, type, encoding, aesthetics). Sei
kritisch und objektiv – vermeide übermäßig hohe Bewertungen,
wenn der Code Schwächen hat oder unklar ist.

VISUALISIERUNGSZIEL:
{goal_description}

Generierter Code:
```{programming_language}
{code}
```

Gib für jede Dimension eine Bewertung (1–10) mit klarer
Begründung und schließe mit einer kurzen Gesamteinschätzung ab.
"""
```

Code-Fragment 4: User-Prompt des Evaluator-Moduls. Der Prompt übergibt dem Sprachmodell das Visualisierungsziel sowie den generierten Code und fordert eine dimensionsweise Bewertung anhand der sechs LIDA-Kriterien. Die Platzhalter `goal_description`, `programming_language` und `code` werden zur Laufzeit mit den jeweiligen Werten befüllt.

Anhang B: Ergebnisse

1. Ergänzende Materialien zu den Ergebnissen

1.1. RQ-1: Detailanalysen der LLM-Judges

| Judge | bugs | transf. | compl. | type | encod. | aesth. | Total |
|---------------|------|---------|--------|------|--------|--------|-------|
| Claude Sonnet | 6.9 | 7.5 | 7.2 | 8.7 | 7.8 | 7.2 | 45.3 |
| GPT-4o | 8.8 | 7.7 | 7.1 | 8.4 | 7.8 | 8.0 | 47.8 |
| GPT-5 | 6.3 | 5.5 | 6.1 | 7.5 | 6.2 | 6.5 | 38.0 |
| Gemini | 9.3 | 9.3 | 9.5 | 9.6 | 9.1 | 8.8 | 55.5 |
| Grok | 8.8 | 8.2 | 9.0 | 9.8 | 8.8 | 8.6 | 53.3 |

Tabelle 17: Durchschnittliche Bewertung pro LLM-Judge und SEVQ-Dimension über alle Konfigurationen ($n = 264$). Die Spalte *Total* summiert die sechs Dimensionen (0–60).

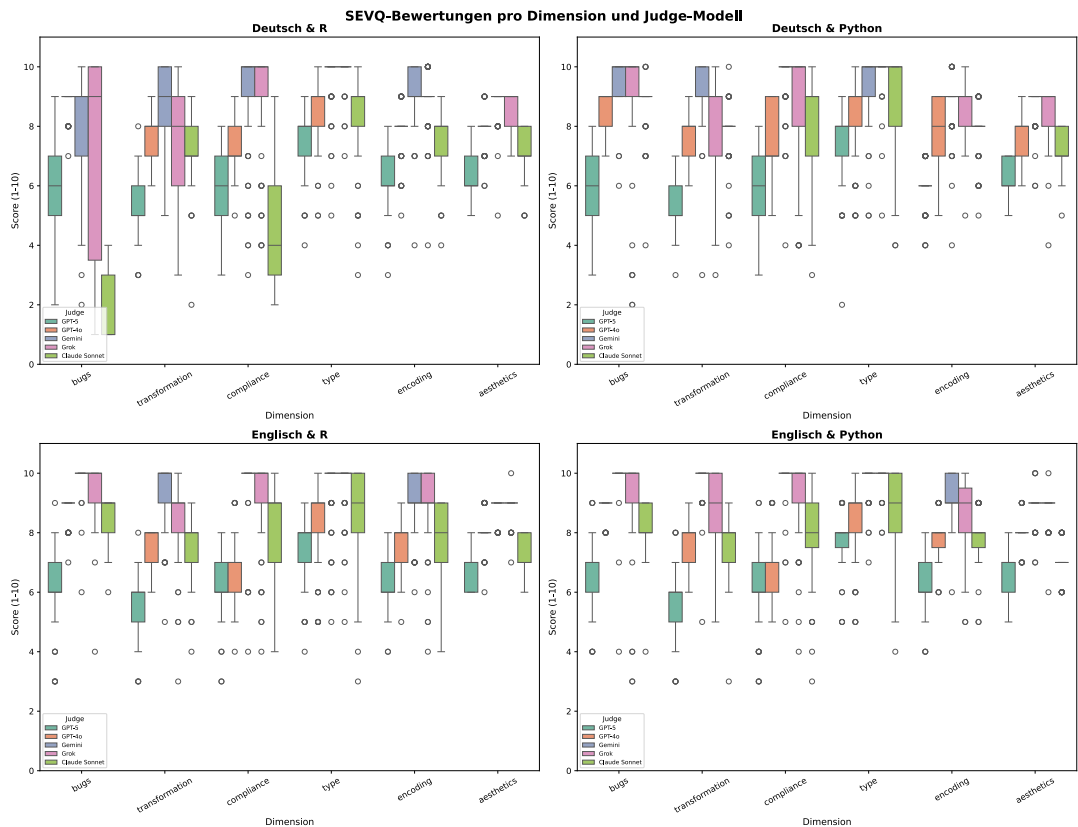


Abbildung 20: SEVQ-Bewertungsverteilungen pro Dimension und Judge-Modell, aufgegliedert nach Konfiguration.

| τ | Agreement | Übereinstimmend |
|--------|-----------|-----------------|
| 0 | 0.000 | 0 / 12 |
| 1 | 0.083 | 1 / 12 |
| 3 | 0.333 | 4 / 12 |
| 5 | 0.583 | 7 / 12 |
| 7 | 0.667 | 8 / 12 |
| 10 | 0.750 | 9 / 12 |

Tabelle 18: Agreement bei verschiedenen Toleranzwerten τ zwischen $\emptyset$ LLM-Judge und $\emptyset$ Studienteilnehmende ($n = 12$).

| Judge | $\text{Agr}_{\tau=3}$ | $\text{Agr}_{\tau=5}$ | $\text{Agr}_{\tau=7}$ | MAE | Bias | ρ |
|---------------|-----------------------|-----------------------|-----------------------|-------|-------|--------------|
| Claude Sonnet | 0.167 | 0.250 | 0.417 | 9.24 | +3.2 | 0.067 |
| GPT-4o | 0.333 | 0.417 | 0.667 | 7.82 | +6.1 | -0.265 |
| GPT-5 | 0.250 | 0.500 | 0.500 | 8.19 | -4.7 | -0.104 |
| Gemini | 0.083 | 0.083 | 0.167 | 12.77 | +12.8 | 0.523 |
| Grok | 0.083 | 0.167 | 0.250 | 11.90 | +11.0 | 0.413 |

Tabelle 19: Agreement und Korrelation pro LLM-Judge gegenüber dem Mittel der 14 Studienteilnehmenden. Agr_{τ} : Agreement bei Toleranz τ . Bias = Judge – Studie.

| Strategie | ρ | MAE | Bias | $\text{Agr}_{\tau=5}$ | $\text{Agr}_{\tau=7}$ |
|--------------------------------|--------------|-------------|------|-----------------------|-----------------------|
| <i>Basis-Aggregation (roh)</i> | | | | | |
| $\emptyset$ aller Judges | 0.065 | 7.30 | +5.7 | 0.583 | 0.667 |
| Median | -0.143 | 8.04 | +5.9 | 0.417 | 0.583 |
| Getrimmtes Mittel | 0.079 | 7.96 | +6.7 | 0.417 | 0.667 |
| <i>Bias-korrigiert</i> | | | | | |
| Gemini (bc) | 0.523 | 6.53 | +5.2 | 0.583 | 0.667 |
| Gemini + Grok (bc) | 0.507 | 6.97 | +5.5 | 0.500 | 0.667 |
| <i>Z-Score-normalisiert</i> | | | | | |
| $\emptyset$ aller Judges (zn) | 0.009 | 6.96 | +1.5 | 0.417 | 0.583 |
| Gemini + Grok (zn) | 0.526 | 6.58 | +1.0 | 0.500 | 0.667 |
| <i>Kalibriert (LOO-CV)</i> | | | | | |
| $\emptyset$ aller Judges (LOO) | -0.390 | 6.03 | -0.3 | 0.667 | 0.750 |
| GPT-4o (LOO) | 0.084 | 6.15 | +0.0 | 0.667 | 0.750 |
| <i>Subset-Selektion (roh)</i> | | | | | |
| GPT-5 + Grok | 0.009 | 6.34 | +3.2 | 0.583 | 0.583 |
| GPT-4o + GPT-5 + Grok | 0.033 | 6.17 | +4.1 | 0.583 | 0.583 |

Tabelle 20: Vollständiger Vergleich der Ensemble-Strategien ($n = 12$). Kategorien: roh, bias-korrigiert (bc), Z-Score-normalisiert (zn), kalibriert (LOO-CV).

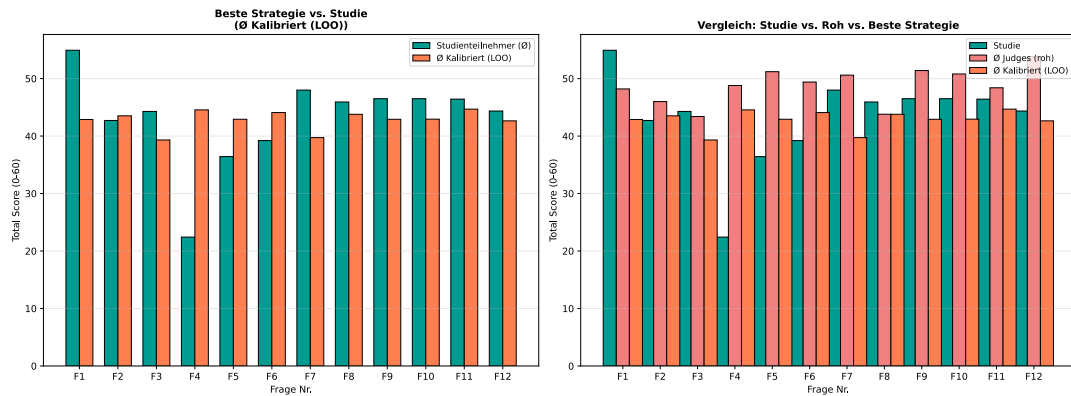


Abbildung 21: SEVQ-Total pro Visualisierung für drei Bewertungsquellen: Studienteilnehmende, LOO-kalibrierte beste Ensemble-Strategie und rohe Judge-Mittelung ($n = 12$ Visualisierungen).

1.2. RQ-2: Detailanalysen der Konfigurationseffekte

| Konfiguration | bugs | transf. | compl. | type | encod. | aesth. |
|-------------------|-------------|---------|--------|------|--------|--------|
| Deutsch & R | 6.35 | 7.38 | 7.09 | 8.74 | 7.93 | 7.74 |
| Deutsch & Python | 8.40 | 7.66 | 7.97 | 8.80 | 7.85 | 7.69 |
| Englisch & R | 8.59 | 7.72 | 7.99 | 8.65 | 8.00 | 7.95 |
| Englisch & Python | 8.56 | 7.82 | 8.06 | 8.91 | 8.02 | 7.84 |

Tabelle 21: Mittlerer SEVQ-Score pro Dimension und Konfiguration, gemittelt über alle LLM-Judges. Jeder Wert ist der Durchschnitt über alle Visualisierungen der jeweiligen Konfiguration.



Abbildung 22: Heatmap der mittleren SEVQ-Bewertung pro Dimension und Konfiguration, gemittelt über alle LLM-Judges. Die Farbskala reicht von 5 (rot) bis 10 (grün).

| Vergleich | U | z | p_{adj} | d |
|-----------------------------------|--------------|--------------|-------------------|--------------|
| DE & R vs. DE & Py | 937.5 | −5.63 | < 0.001 | −0.93 |
| DE & R vs. EN & R | 811.0 | −6.21 | < 0.001 | −1.12 |
| DE & R vs. EN & Py | 550.0 | −7.00 | < 0.001 | −1.38 |
| DE & Py vs. EN & R | 2129.0 | −1.07 | 1.000 | −0.16 |
| DE & Py vs. EN & Py | 1948.0 | −1.03 | 1.000 | −0.28 |
| EN & R vs. EN & Py | 2171.0 | −0.01 | 1.000 | −0.11 |

Tabelle 22: Paarweise Mann-Whitney-U-Tests für den SEVQ-Total-Score mit Bonferroni-korrigierten p -Werten ($k = 6$ Vergleiche). Cohens d als Effektstärke: $|d| < 0.2$ trivial, $0.2\text{--}0.5$ klein, $0.5\text{--}0.8$ mittel, > 0.8 groß. Signifikante Unterschiede ($p_{\text{adj}} < 0.05$) sind fett markiert.

| Konfiguration | n_{DS} | M | SD | $\tilde{x}$ | CI_{95} |
|-------------------|-----------------|------|------|-------------|------------------|
| Deutsch & R | 22 | 65.2 | 33.3 | 66.7 | [50.5; 79.8] |
| Deutsch & Python | 23 | 49.3 | 47.0 | 66.7 | [29.0; 69.5] |
| Englisch & R | 23 | 72.5 | 34.3 | 66.7 | [57.7; 87.2] |
| Englisch & Python | 22 | 48.5 | 45.7 | 50.0 | [28.3; 68.6] |

Tabelle 23: VER Success Rate (0–100%) in Iteration 0 pro Konfiguration. n_{DS} ist die Anzahl der Datensätze, die die Visualisierungsphase erreichten. CI_{95} ist das 95%-Konfidenzintervall des Mittelwerts.

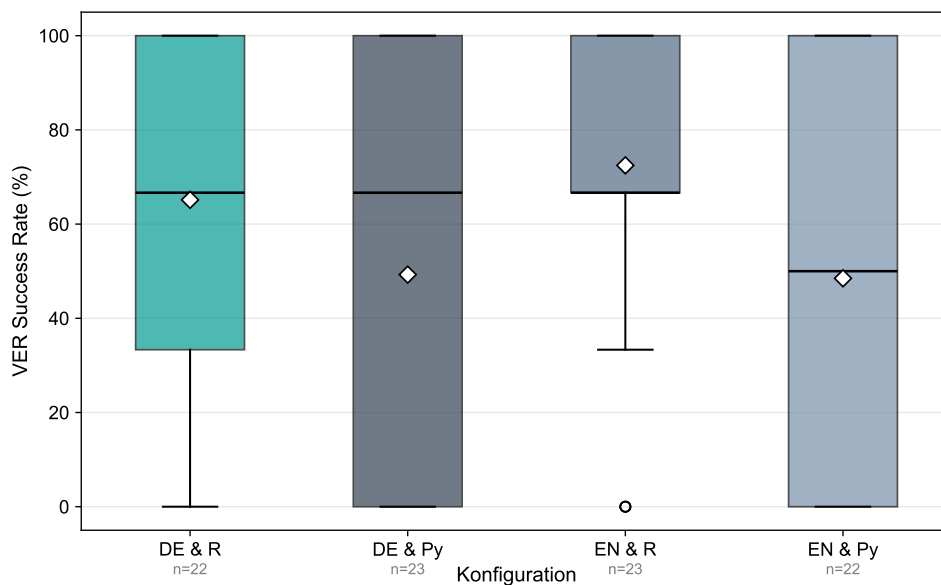


Abbildung 23: Boxplot der VER Success Rate (Iteration 0) pro Konfiguration. Rauten markieren den Mittelwert, horizontale Linien den Median.

1.3. RQ-4: Dimensionsanalyse der Metadaten-Ablation

| Dim. | DE+Python | | | | | EN+Python | | | | |
|---------|-----------|------|----------|-------|--|-----------|------|----------|------|--|
| | Meta | Ohne | Δ | d | | Meta | Ohne | Δ | d | |
| bugs | 8.40 | 8.30 | +0.10 | 0.17 | | 8.56 | 8.45 | +0.10 | 0.23 | |
| transf. | 7.63 | 7.54 | +0.09 | 0.22 | | 7.82 | 7.75 | +0.08 | 0.17 | |
| compl. | 7.95 | 7.95 | +0.00 | 0.01 | | 8.06 | 8.01 | +0.04 | 0.08 | |
| type | 8.80 | 8.73 | +0.07 | 0.14 | | 8.91 | 8.64 | +0.27 | 0.37 | |
| encod. | 7.85 | 7.81 | +0.03 | 0.10 | | 8.02 | 7.90 | +0.12 | 0.23 | |
| aesth. | 7.68 | 7.70 | -0.02 | -0.06 | | 7.84 | 7.77 | +0.08 | 0.23 | |

Tabelle 24: Mittlerer SEVQ-Score pro Dimension, Bedingung und Konfiguration. Δ = Mit – Ohne Metadaten, d = Cohens Effektstärke. Signifikanzniveau: $\alpha_{\text{korr}} = 0.025$.

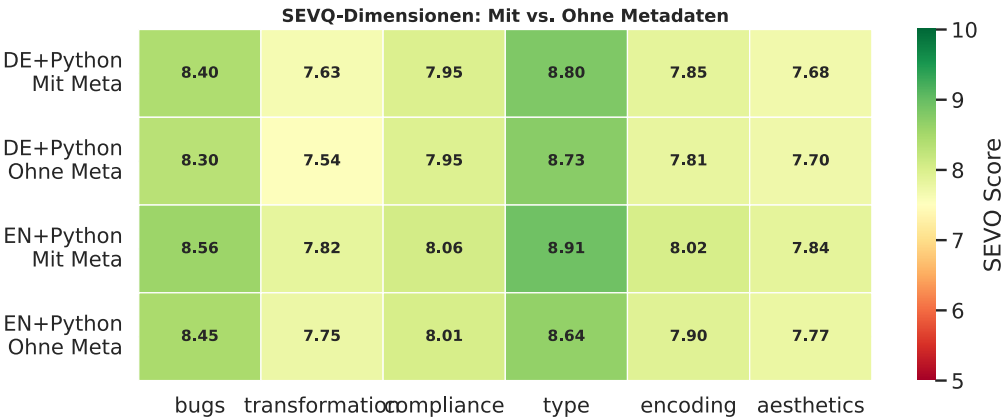


Abbildung 24: Heatmap der mittleren SEVQ-Bewertung pro Dimension, aufgeschlüsselt nach Bedingung (mit/ohne Metadaten) und Konfiguration. Die Farbskala reicht von 5 (rot) bis 10 (grün).

| Konfiguration | Mit Meta | Ohne Meta | n | p | CI ₉₅ |
|---------------|-------------------|------------|-----|-------|------------------|
| DE+Python | 20 (47,6%) | 22 (52,4%) | 42 | 0.878 | [33.4; 62.3]% |
| EN+Python | 27 (64,3%) | 15 (35,7%) | 42 | 0.088 | [49.2; 77.0]% |
| Gesamt | 47 (56,0%) | 37 (44,0%) | 84 | 0.326 | |

Tabelle 25: A/B-Präferenzen der 14 Studienteilnehmenden für den Metadatenvergleich ($3 \times 14 = 42$ Stimmen pro Konfiguration).

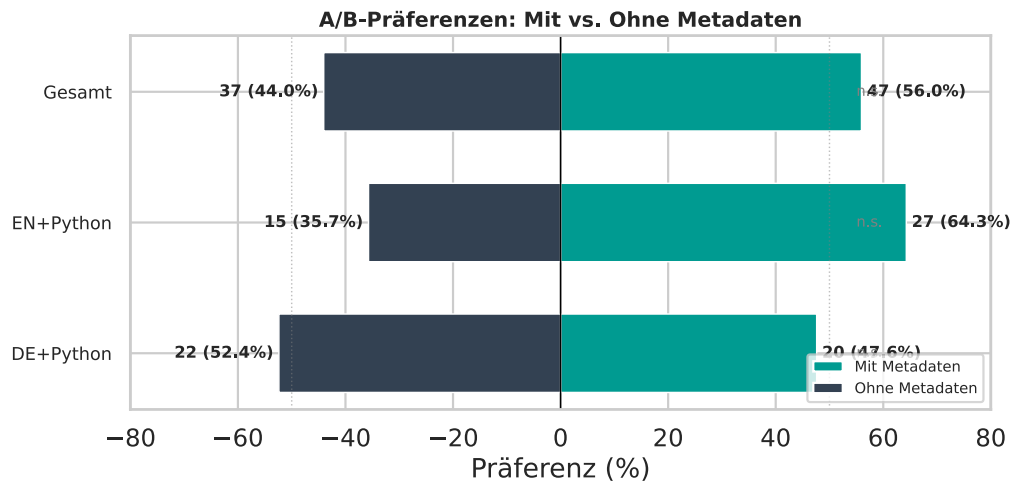


Abbildung 25: Balkendiagramm der A/B-Präferenzen (mit vs. ohne Metadaten) pro Konfiguration, basierend auf den Bewertungen der 14 Studienteilnehmenden.

| Konfiguration | n | M_{meta} | SD_{meta} | M_{ohne} | SD_{ohne} | t | p |
|---------------|-----|-------------------|--------------------|-------------------|--------------------|--------|-------|
| DE+Python | 23 | 49.3 | 47.0 | 47.8 | 45.9 | 0.439 | 0.665 |
| EN+Python | 22 | 48.5 | 45.7 | 53.0 | 46.8 | -0.826 | 0.418 |

Tabelle 26: Gepaarter Vergleich der VER Success Rate pro Bedingung (mit/ohne Metadaten) und Konfiguration mit t -Statistik und p -Wert.

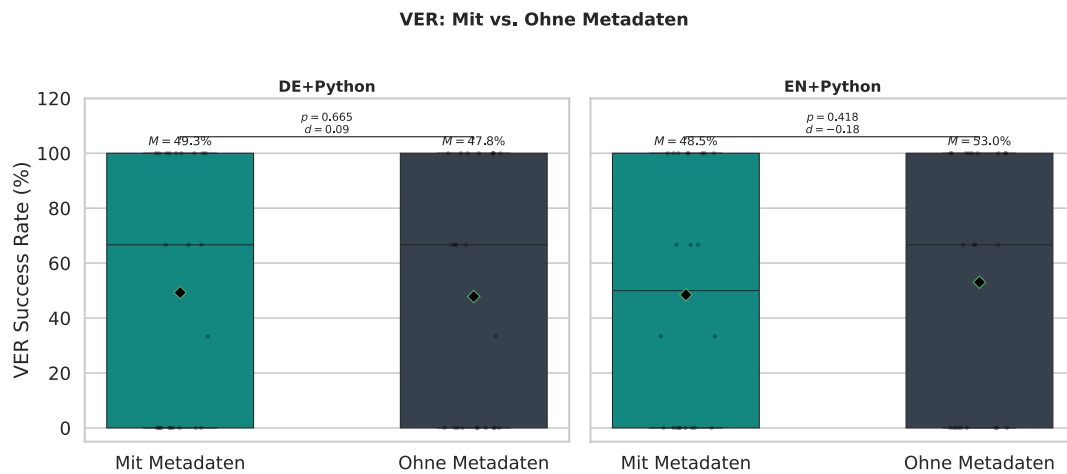


Abbildung 26: Boxplot der VER Success Rate, aufgeschlüsselt nach Bedingung (mit/ohne Metadaten) und Konfiguration.

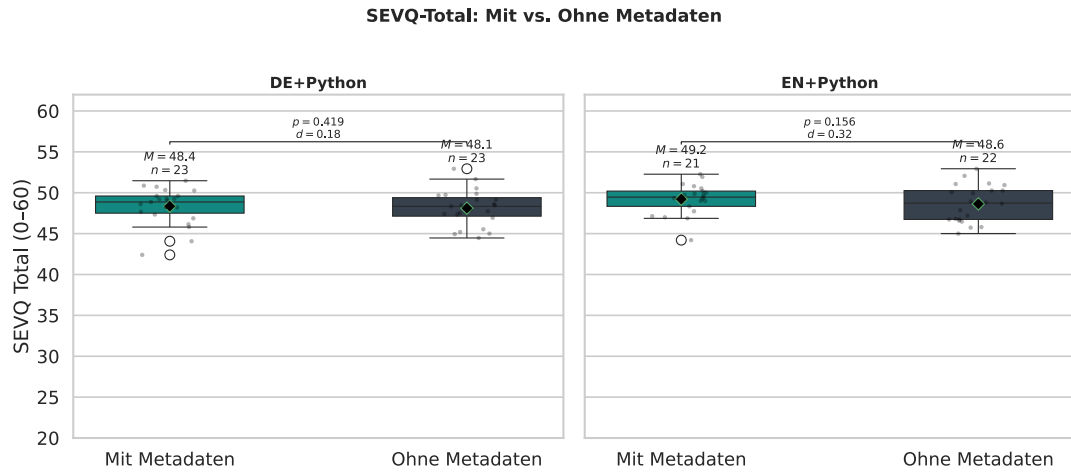


Abbildung 27: Boxplot des SEVQ-Total-Scores, aufgeschlüsselt nach Bedingung (mit/ohne Metadaten) und Konfiguration.

1.4. RQ-5: Ergänzende Stabilitätsanalysen

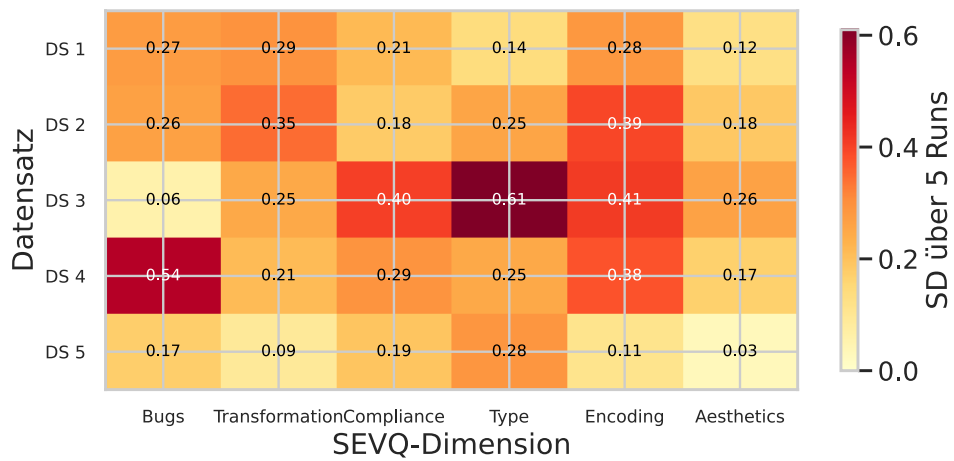


Abbildung 28: Heatmap der Standardabweichung des SEVQ-Scores pro Dimension und Datensatz über $k = 5$ Runs.

| Datensatz | Run 1 | Run 2 | Run 3 | Run 4 | Run 5 | M | SD |
|-----------|-------|-------|-------|-------|-------|-------|------|
| DS 1 | 100 | 100 | 100 | 100 | 100 | 100.0 | 0.0 |
| DS 2 | 100 | 66.7 | 100 | 100 | 100 | 93.3 | 14.9 |
| DS 3 | 100 | 100 | 100 | 100 | 100 | 100.0 | 0.0 |
| DS 4 | 100 | 100 | 100 | 100 | 66.7 | 93.3 | 14.9 |
| DS 5 | 66.7 | 100 | 100 | 100 | 100 | 93.3 | 14.9 |

Tabelle 27: VER Success Rate (finale Iteration, %) pro Datensatz über $k = 5$ Runs.

| Dimension | Datensatz 1 (Q31) | | | Datensatz 2 (Q32) | | |
|-----------------------------|-------------------|-------------|-------------|-------------------|-------------|-------------|
| | <i>M</i> | <i>SD</i> | $\tilde{x}$ | <i>M</i> | <i>SD</i> | $\tilde{x}$ |
| Klarheit | 4.14 | 0.86 | 4.0 | 2.50 | 0.76 | 2.5 |
| Aussagekraft / Relevanz | 4.29 | 0.61 | 4.0 | 2.71 | 1.07 | 3.0 |
| Gestalterische Qualität | 3.93 | 0.83 | 4.0 | 2.64 | 1.08 | 2.5 |
| Informationsgehalt | 4.07 | 0.92 | 4.0 | 3.00 | 0.96 | 3.0 |
| Interpretierbarkeit | 4.57 | 0.51 | 5.0 | 2.64 | 0.84 | 3.0 |
| Plausibilität / Korrektheit | 4.21 | 0.89 | 4.5 | 3.14 | 0.86 | 3.0 |
| Gesamt | 4.20 | 0.79 | 4.0 | 2.77 | 0.94 | 3.0 |

Tabelle 28: Likert-Bewertung der wahrgenommenen Ergebnisstabilität (5-Punkt-Skala). Frage 31: Datensatz 1, Frage 32: Datensatz 2.

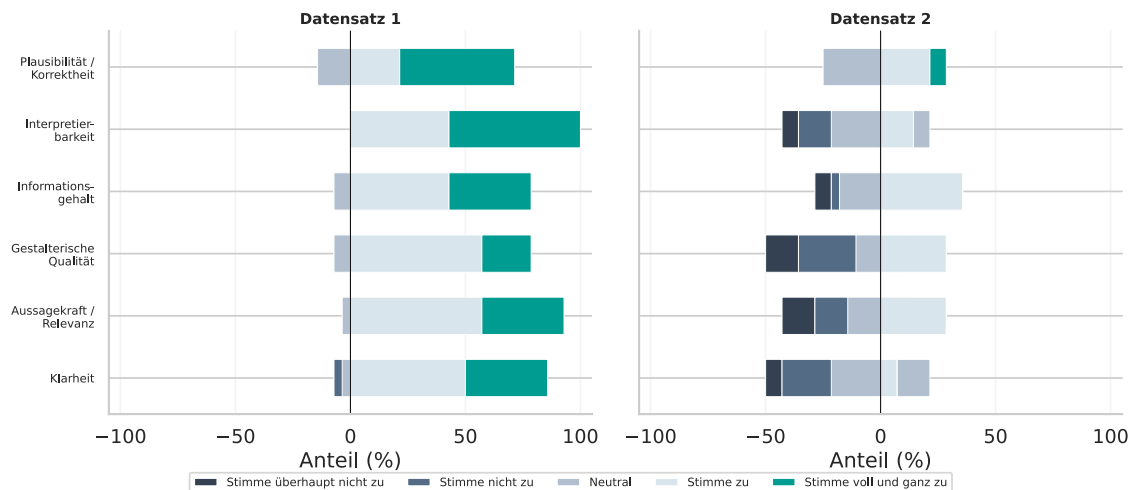


Abbildung 29: Divergierendes Balkendiagramm der Likert-Bewertungen zur Ergebnisstabilität, getrennt nach Datensatz.